VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

REPORT
CAPSTONE PROJECT
SEMESTER 252 ACADEMIC YEAR 2025-2026

DEVELOPING A SMART MONITORING TOOL FOR HIGH-PERFORMANCE COMPUTING SYSTEMS

Major: Computer Science

COUNCIL:

SUPERVISOR(s): THOAI NAM, Assoc. Prof., Dr.
DIEP THANH DANG, Ph.D.

——o0o——

STUDENTS: DANG NGOC PHU - 2252617
PHAM THAI AN - 2252011
NGUYEN VIET HUNG - 2252272

HO CHI MINH CITY, May 2026

Supervisor's Signature

_____ Date: _____

Assoc. Prof. Thoai Nam (Supervisor)

Associate Professor

Faculty of Computer Science and Engineering

_____ Date: _____

Dr. Diep Thanh Dang (Supervisor)

Doctoral of Natural Sciences

Faculty of Computer Science and Engineering

Declaration of Authenticity

Our group confirms that we solely carried out this capstone project under the guidance of Assoc. Prof. Thoai Nam and Dr. Diep Thanh Dang, from the Faculty of Computer Science and Engineering at Vietnam National University - Ho Chi Minh City University of Technology.

We have ensured that all external references and sources used in the project are properly cited and documented.

If there is any instance of plagiarism be found, we are fully prepared to face the consequences. Ho Chi Minh City University of Technology - Vietnam National University bears no liability for any copyright infringements that may have occurred during the course of our research.

Ho Chi Minh City, May 2026

Authors,

Acknowledgement

We would like to express our deep appreciation to Assoc. Prof. Thoai Nam, Dr. Diep Thanh Dang. Their expertise, thoughtful feedback, and continuous support have been instrumental in guiding our research. We could not truly have done it without their insight and encouragement.

Besides, we are grateful for the help from Mr. La Quoc Nhut Huan and Mr. Le Bao Khanh for their valuable comments for research aspect and our proposed pipeline, including pointing out strengths as well as weakness of previous works on multimodal models so that we can elaborate on our ideas more effectively.

In addition, we want to say a big thank you to our families. Their unwavering belief in our abilities and constant encouragement have been what kept us going, especially through the tough times. Their support has meant the world to us, and we are forever grateful for their love and motivation.

Abstract

High-Performance Computing (HPC) systems have become an essential infrastructure in modern academic and research institutions, providing the computational capacity required for large-scale simulations, data-intensive experiments and complex scientific workloads. As these systems are typically shared among multiple users and research groups, effective resource management is critical to ensuring fair allocation and operational efficiency. In practice, however, resource utilization in such environments is frequently imbalanced, there are some users over-request resources far beyond their actual needs, while others underestimate their requirements, resulting in idle nodes alongside heavily contended ones. A major reason behind this inefficiency is the lack of fine-grained, real-time visibility into how resources are being consumed across users, applications and processes.

Existing HPC monitoring solutions only address this problem at a certain level. Most tools operate at the node level, reporting hardware metrics without correlating them to the specific users or jobs responsible for resource consumption. Furthermore, these tools typically present data through static, pre-configured dashboards that require great technical expertise to interpret, creating a significant barrier for non-expert users who need to understand system behavior without deep knowledge of the underlying infrastructure.

To address these limitations, this thesis proposes a comprehensive monitoring framework designed for HPC environments. The system employs a lightweight Compute Node Agent on each physical machine, leveraging eBPF for kernel-level, low-overhead collection of CPU, memory, disk I/O and network metrics at the process level, alongside nvidia-smi for GPU telemetry. Collected metrics are continuously streamed to a Collect

Agent via gRPC, where they pass through a structured preprocessing pipeline, including validation, filtering and enrichment before being published to Apache Kafka for reliable, high-throughput delivery to downstream services. The overall architecture is designed with modularity as a central principle, allowing individual components to be replaced, extended or reconfigured independently without disrupting the rest of the system.

Beyond data collection and transport, the framework introduces an LLM-powered Visualization Chatbot that changes how users interact with monitoring data. Rather than navigating static dashboards, users can submit natural language queries and receive interactive Grafana dashboards generated automatically through a seven-stage pipeline encompassing query transformation, analytical question generation, SQL synthesis, chart recommendation and zero-configuration dashboard deployment. This capability makes the system accessible to a wide range of users, enabling both system administrators and non-expert researchers to obtain actionable insights from the cluster without requiring knowledge of database schemas or visualization tooling.

Glossary

This section provides the full form and explanation of all abbreviations and acronyms used throughout the thesis.

Abbreviation	**Explanation**
API	Application Programming Interface: A set of protocols that allow different software systems to communicate.
CPU	Central Processing Unit: The main component in a computer responsible for executing instructions.
eBPF	Extended Berkeley Packet Filter: A technology in the Linux kernel that allows efficient packet filtering and system monitoring.
GPU	Graphics Processing Unit: A processor specialized for rendering images and performing parallel computations.
HPC	High-Performance Computing: The use of supercomputers and parallel processing techniques to solve complex computational problems.
MQ	Message Queue: An asynchronous communication mechanism that allows applications to exchange data via a queue, decoupling producers from consumers to ensure system scalability.
RAM	Random Access Memory: A type of volatile memory that stores data temporarily during computation.

Contents

1 Introduction **1**

1.1 Context 1

1.2 Problem Statement 2

1.3 Objectives 4

1.4 Scope 5

1.5 Thesis Outline 6

2 Related Works **8**

2.1 Academic Research Tools 8

2.1.1 Data Center Data Base (DCDB) 8

2.1.2 Exascale Monitoring (Examon) 10

2.1.3 Lightweight Distributed Metric Service (LDMS) 13

2.1.4 Ganglia Monitoring System (Ganglia) 15

2.1.5 Monitoring System for HPC Clusters (MonSTer) 18

2.1.6 Cluster Administration Using Relational Databases (CARD) . . 21

2.1.7 A High-Speed Cluster Monitoring System (Supermon) 23

2.1.8 Network Performance Monitoring Tool (NWPerf) 26

2.1.9 End-to-End I/O Monitoring And Diagnosis (Beacon) 28

2.1.10 Center-Wide and Job-Aware Cluster Monitoring (PIKA) 30

2.2 Industrial Tools 33

2.2.1 Zabbix 33

2.2.2 Prometheus 36

2.2.3 Nagios 38

2.3 Research Gaps and Evaluation Criteria 41

2.3.1 Research Gap Analysis 41
2.3.2 Evaluation Criteria 43
2.4 Visualization Chatbot 47
2.4.1 Natural-Language Interfaces to Databases 48
2.4.2 LLM-Based Text-to-SQL Techniques 49
2.4.3 Natural-Language Interfaces to Data Visualization 51
2.4.4 The Gap and Our Position 52

3 System Requirements **56**
3.1 Functional Requirements 56
3.2 Non-Functional Requirements 59

4 System Architecture **63**

5 System Design **69**

6 Visualization Chatbot **87**
6.1 Introduction and Motivation 87
6.2 Two Pipelines, One API 89
6.2.1 One Sentence In, One Artefact Out 89
6.2.2 Why Two Pipelines Instead of One 90
6.2.3 Why a Unified API Nonetheless 92
6.2.4 Pipeline Overview 92
6.3 Schema Introspection on Every Request 94
6.3.1 The Schema-Drift Problem 95
6.3.2 Historical-Tier Introspection 96
6.3.3 Realtime-Tier Introspection 96
6.3.4 Cost Analysis 97
6.3.5 Privacy and Cardinality Bounds 98
6.4 Pipeline Classification with a Safe Default 99
6.4.1 The Routing Problem 100
6.4.2 Why an LLM Rather than a Rule 100
6.4.3 Inputs and Outputs 101

6.4.4 Failure Modes and Defaults . . . 102
6.4.5 Why Historical Is the Safe Default . . . 102
6.5 Historical Pipeline . . . 104
6.5.1 Pipeline Overview and Artefact Justification . . . 105
6.5.2 Retrieval-Augmented SQL Generation . . . 106
6.5.3 SQL Execution and Isolation . . . 108
6.5.4 SQL Auto-Fix . . . 108
6.5.5 DataFrame Profile, Not Raw Rows . . . 109
6.5.6 Hardened Chart-Code Generation . . . 110
6.5.7 Chart-Code Auto-Fix . . . 111
6.5.8 Pipeline Output . . . 112
6.6 Realtime Pipeline . . . 112
6.6.1 Pipeline Overview and Artefact Justification . . . 113
6.6.2 Why Structured Configuration Rather Than a Raw Query . . . 114
6.6.3 The Panel Configuration Schema . . . 115
6.6.4 The Deterministic Flux Builder . . . 116
6.6.5 Grafana Panel Push . . . 117
6.6.6 Pipeline Output . . . 118
6.7 Caching and Freshness Strategy . . . 119
6.7.1 Per-Request Schema Introspection . . . 119
6.7.2 RAG Corpus Persistence . . . 119
6.7.3 Grafana Datasource Discovery . . . 120
6.7.4 Summary of Caching Decisions . . . 121
6.8 Security as a Design Property . . . 121
6.8.1 Threat Surface . . . 122
6.8.2 T1: SQL Injection by String Concatenation Is Structurally Absent 123
6.8.3 T2: Realtime-Path Injection Is Bounded by Indirection . . . 123
6.8.4 T3: Arbitrary Code Execution Is the Weakest Boundary . . . 124
6.8.5 Threat Summary . . . 126
6.9 Summary . . . 127

7 Implementation 130

8 Evaluation **149**

8.1 Monitoring Substrate Overhead 149

8.2 Chart-Generation Core on VisEval 158

9 Conclusion and Future Work **161**

9.1 Conclusion 161

9.2 Current Limitations 163

9.3 Future Work 164

References **167**

List of Tables

2.1 Comparative Summary of HPC Monitoring Tools based on Evaluation Criteria 47
2.2 Comparison of prior natural-language visualization work and the Visualization Chatbot. 54

5.1 Comparison of Linux process-level metric collection approaches. eBPF's event-driven model, per-PID BPF map attribution, and kernel safety guarantee make it the most suitable choice for continuous, low-overhead HPC monitoring **Cassagnes2020**, **Dai2025eHashPipe**. 72
5.2 Summary of technology stack across all system components. 86

6.1 Structural contrast between the historical and realtime pipelines. The two pipelines share only the schema-introspection and classification stages; after the classifier they diverge in data tier, query language, output artefact, freshness semantics, persistence, and the typical kind of question they answer. 94
6.2 Schema introspection at the two storage tiers. Both tiers are queried on every request; the result feeds the classifier and the per-pipeline generators. The realtime tier additionally returns a bounded sample of tag values, capped to limit token cost and to keep accidental disclosure of high-cardinality tags within configured limits. 99
6.3 Classifier failure modes and default behaviours. Each failure is handled deterministically rather than propagated to the caller, so that an operator's question is answered as long as any pipeline is reachable. 102

6.4 Panel configuration schema emitted by the LLM in the realtime pipeline. The six fields are sufficient to describe any panel the pipeline currently produces, and each is constrained enough that downstream validation reduces to per-field checks. The deterministic builder compiles this configuration into Flux; the LLM never emits Flux text directly. 116

6.5 Threats introduced by LLM-mediated query and code generation, and the mechanisms that mitigate each. T1 and T2 are bounded by structural design choices; T3 is the weakest of the three boundaries and the principal hardening item identified for future work. 126

8.1 VisEval single-table split (2,064 queries = 688 samples $\times$ 3 paraphrases), all systems under `gpt-4o-mini`, temperature 0, 180-second timeout. Quality = Pass Rate $\times$ Readability (VisEval composite). *Tok/q* is the sum of prompt and completion tokens per query. ΔQ is the quality delta relative to the GPT-4o-mini plain-prompt baseline. Best value per column is bold; the proposed system is shaded. 159

List of Figures

2.1 DCDB overall architecture. 9
2.2 Examon overall architecture. 12
2.3 LDMS overall architecture. 14
2.4 Ganglia overall architecture. 17
2.5 MonSTer overall architecture. 20
2.6 CARD overall architecture. 22
2.7 Supermon overall architecture. 25
2.8 NWPerf overall architecture. 27
2.9 Beacon overall architecture. 29
2.10 PIKA overall architecture. 32
2.11 Zabbix overall architecture. 34
2.12 Prometheus overall architecture. 37
2.13 Nagios overall architecture. 40

4.1 Abstract system architecture of the proposed Smart Monitoring Framework, organized into three functional layers. Solid arrows indicate the data flow path from collection through processing to storage and presentation. Dashed arrows indicate the control flow path through which the Coordinator distributes runtime configuration updates to agents and middleware components. 64

5.1 Smart Monitoring Tool Detailed Architecture with Specific Technologies Stack. 69

6.1 End-to-end flow of the Visualization Chatbot. A single natural-language question is enriched with live schema information from both backends, classified into one of two pipelines, and answered through the path appropriate to its temporal scope. The historical pipeline produces an inline SVG chart; the realtime pipeline produces an embeddable Grafana panel URL. Both pipelines return through a single JSON response shape. . . . 93
6.2 Classifier decision flow. Every error edge—realtime tier unreachable, classifier call failure, malformed JSON, unknown pipeline value—routes to the historical pipeline. The realtime pipeline is reached only when every check on the success path passes and the classifier's verdict is explicitly `realtime`. 104
6.3 Internal structure of the historical pipeline. The natural-language question is transformed into SQL by retrieval-augmented generation against a ChromaDB-backed corpus; the SQL is executed against TimescaleDB; the resulting DataFrame is summarised into a compact profile that is sent to a second LLM call which emits matplotlib code; the code is executed in a constrained namespace and produces an inline SVG artefact. Both LLM-emitting stages are protected by a single-shot regeneration loop that retries once on execution error. 106
6.4 Internal structure of the realtime pipeline. The LLM emits a structured panel configuration rather than a raw query; a deterministic builder compiles the configuration into a Flux query under an operator whitelist; the resulting panel definition is pushed to a shared dashboard via the Grafana HTTP API; the API's response gives an embeddable panel URL that becomes the pipeline's output. 114

8.1 Scenario 1 - Idle CN-Agent footprint under partial collection (CPU and disk) and full collection. 151
8.2 Scenario 2 - Host-process-density sweep on the CN host with full collection and full preprocessing. 153
8.3 Scenario 3 - Collector scaling from one to four attached CN-Agents with full collection and preprocessing. 155

8.4 Scenario 4 - Preprocessing-depth impact with four CN-Agents attached. 157

Chapter 1

Introduction

In Chapter 1, we introduce the project by discussing the foundational context, defining the core problem, and outlining our research goals. The overall context of High-Performance Computing (HPC) monitoring is detailed in ***Section 1.1****, which leads to the specific problem statement addressed by this research in* ***Section 1.2****. Additionally, the primary objectives are discussed in* ***Section 1.3****, followed by the defined scope and boundaries of the project in* ***Section 1.4****. Finally, the structure of the remaining thesis is presented in* ***Section 1.5****.*

1.1 Context

High-Performance Computing (HPC) systems have become a critical part in modern research and academic institutions, supporting a wide range of computational workloads such as scientific simulations, large-scale data analysis and machine learning model training. Basically, these systems consist of interconnected compute nodes equipped with high-performance CPUs, specialized GPUs for parallel processing and high-speed interconnects, their nature enables computation at a scale that far exceeds what conventional machines can offer. In academic and research institutions, HPC systems are typically operated as shared resources, where multiple users and research groups submit jobs concurrently, coordinated by a central scheduling system allocating compute time and resources.

Given the scale and shared nature of these environments, ensuring that the system operates efficiently and that resources are utilized appropriately is a complex challenge. Sys-

tem monitoring has long been recognized as one of the most fundamental approaches to maintaining operational efficiency in HPC clusters. By continuously collecting telemetry data from across the system, monitoring tools provide administrators with real-time visibility into hardware utilization, job behavior and overall system health. Unlike static configurations or scheduler-only approaches, which offer limited feedback, a well-designed monitoring solution delivers continuous, data-driven insights that form the basis for informed resource management, anomaly detection and performance optimization.

1.2 Problem Statement

Although such systems have a significant computing power, they are often underutilized. Making efficient use of these resources has been a critical challenge. The current problem is that some users over-demand the number of cores, memory, or the number of GPUs required for their applications because they may not have a knowledge of the exact amount of resources they will actually need, while others underestimate their requirements. This also results in resource imbalances because some nodes will be idle, while others will have a heavy burden on top of them, which is also the current reason for the contention or performance degradation experienced by the HPC systems recently. Furthermore, such inefficiency not only reduces the overall throughput of the system but also has negative impacts on the fairness and efficiency in job scheduling within the cluster in the long run.

Methods have been considered to solve this problem, such as improving the job schedulers performance, applying predictive resource allocation techniques or using various workload characterization models. However, these solutions require precise and fine-grained monitoring data to operate effectively. Otherwise, even the most advanced scheduling or optimization policies are unable to make informed decisions with the lack of access into how resources are actually consumed by users, jobs, and processes. Unlike other methods, a monitoring solution collect real-time data about system activities, which forms a basis that enabling suitable optimizations and efficient resource management techniques to employ effectively. Compared to a static configuration or scheduler-only solutions, a monitoring tool delivers continuous feedback and data-driven insights, making it a scalable way to handle inefficiencies in multi-user HPC environments.

Certain traditional HPC monitoring tools provide node-level performance metrics, like CPU usage, memory consumption, or network throughput, without correlating those metrics with higher-level of interest to users, jobs, or application activities. Therefore, it would be possible for the cluster administrator to know which nodes are idle or overloaded, but it would still be hard to determine which user or job was causing the problem, or to detect whether a code has scalability problems, is configured or compiled poorly, or is suffering from resource misallocation. On clusters hosting tens of users and hundreds of jobs concurrently, this lack of visibility restricts the ability of the system administrator to optimize cluster efficiency.

However, the higher granularity of monitoring introduces a number of technical challenges in terms of data volume and velocity. Collecting high-frequency metrics from all plugins hosted on each Compute Node (CN) about concurrently executing processes in a distributed cluster generates a huge stream of data. Sending raw data directly to a central server from all sources can easily overwhelm the network bandwidth and cause high processing overhead. Consequently, there is an urgent need for an architecture that will support preprocessing at the compute nodes (edges) to filter and aggregate data at the source before sending to the next modules. Additionally, traditional ”store-then-analyze” approaches (simply aggregating data from the agents then storing it onto a database), are also unacceptable for real-time management due to bottlenecks of the disk I/O latency. To allow the system to provide real-time insights and immediate its health alerts, it requires a streaming analytics ability and preprocessing at both the edges and server side to enhance the storing process as well as to provide immediate system's status with low delay benefits from the ”store-and-analyze” approach.

With the increasing data granularity, another problem arises, which is information overload. Having high-quality metrics at hand may useless when the data is still inaccessible or difficult to analyze due to an enormous amount of information that current HPC computer clusters produce. Current state-of-the-art monitoring systems use static, preconfigured dashboards for data analysis. These dashboards work well for periodic anal-

ysis but lack flexibility in dealing with ad-hoc analyses or complicated problem-solving, where system administrators must correlate unrelated metrics right away. Identifying the root cause of a bottleneck in thousands of jobs, using static graphical views is not only time-consuming but also creates a significant cognitive load for system administrators. Moreover, there is a gap between raw system metrics and understanding on the part of users. Regular users are not system experts and cannot themselves writing complex query to the database or analyze raw system logs, thereby promoting a passive system management technique. Therefore, a definite need for a smarter and more interactive monitoring framework, which goes beyond simple visual representation.

1.3 Objectives

The goal of this thesis is to design and implement a comprehensive real-time monitoring framework for High-Performance Computing (HPC) environments. This solution aims to fill the current gap between raw metric collection and valuable user knowledge by providing detailed resource tracking and an intelligent conversation interface. To achieve this main goal, the study focuses on the following specific objectives:

- **To develop a lightweight agent for monitoring:** Design a data collection mechanism with minimal overhead, which can capture resource utilization (CPU, Memory, GPU and Networking) at various levels of granularity, starting from the node-level down to the user, job and process levels.

- **To build a scalable and real-time data ingestion pipeline:** Implement a system architecture ingesting and storing time-series data from compute nodes at high ingestion rate without impacting the performance of applications running on compute nodes, while enabling immediate health alerts through streaming analytics.

- **To integrate an LLM-based Chatbot:** Incorporate a Large Language Model (LLM) that understands natural language inputs from users and administrators, converting them into database queries that can be processed to obtain related status information about the system without technical database expertise.

- **To enable Dynamic Dashboard Generation:** Create an automated visualization

engine capable of producing interactive and ad-hoc dashboards based on the context of the user queries to overcome the drawbacks that exist with static monitoring views and reduce information overload.

- **To evaluate system performance and usability:** Assess the proposed solution in a real-world HPC environment to verify its low monitoring overhead, data accuracy in reporting and system analysis through user assistance.

1.4 Scope

The scope of this project is limited to the design, implementation and evaluation of the smart monitoring tool within the specific environment of the High-Performance Computing (HPC) laboratory. The primary boundaries and constraints of this research are defined as follows:

- **Infrastructure and Deployment:** The system is implemented and tested on a cluster consisting of seven dedicated machines. Four of these machines are configured as Compute Nodes (CNs) to monitor high-performance workloads, one CN serving as the host for the Collect Agent middleware. The remaining three machines are utilized to deploy the core management and application services, including the etcd coordinator, the storage system (InfluxDB and TimescaleDB), the Grafana visualization platform, Python data processing scripts and the AI Support Agent server.

- **Monitored Resources:** Data collection is focused on the most critical hardware performance metrics: CPU utilization, GPU status (including utilization and memory), system memory consumption, and network throughput.

- **Monitoring Granularity:** The tool provides visibility across primary levels: node-level, user-level, and process-level. While the system captures fine-grained Process ID (PID) data, the correlation and analysis are prioritized for shared resource management among multiple users and jobs.

- **Functional Limitations:** The integrated Chatbot and visualization engine are designed to handle queries and generate dashboards related to the monitored system

resources mentioned. Tasks unrelated to system monitoring or general-purpose administrative actions fall outside the scope of the current AI support agent.

1.5 Thesis Outline

The remaining part of this thesis is organized into seven chapters and an appendix to provide an overall view of the research, design and implementation of the proposed monitoring system:

- **Chapter 2: Related Works** provides a general review of existing academic HPC monitoring tools. It identifies current research gaps and establishes a set of eight evaluation criteria used to benchmark these solutions and validate our proposed architecture.
- **Chapter 3: System Requirements** outlines the functional and non-functional requirements necessary for an effective HPC monitoring system. These requirements are derived from both the literature survey and the specific operational needs of the university's HPC laboratory.
- **Chapter 4: System Architecture** presents the abstract design of the system. It details the high-level modules, including Data Collection, Middleware, Storage and AI Support. Besides it also describes how these components interact to ensure a scalable and real-time data flow.
- **Chapter 5: System Design and Technology Selection** maps the abstract modules from the previous chapter to specific technical choices. It provides the rationale for selecting key technologies such as eBPF, Apache Kafka, InfluxDB, and Large Language Models (LLMs) to fulfill the design goals.
- **Chapter 6: Implementation** describes the technical realization of the system. It covers the details of kernel-level monitoring using eBPF hooks, the implementation of the streaming pipeline, the data aggregation logic between tiered databases and the seven-stage architecture of the visualization chatbot.
- **Chapter 7: Evaluation** presents the experimental results and performance analysis

of the implemented tool. It assesses the system based on efficiency and its overhead to the nodes while handling high-frequency telemetry.

- **Chapter 8: Conclusion and Future Work** summarizes the core achievements of the thesis. It also discusses the system's limitations and identifies potential directions for future enhancements.
- **Appendix: Setup and Installation Guide** provides a practical manual for system administrators, including environmental prerequisites, Docker deployment instructions and configuration procedures for the monitoring cluster.

Chapter 2

Related Works

This chapter provides a comprehensive review of the existing research and systems relevant to HPC monitoring. The content is structured into three main parts: ***Section 2.1*** *surveys some established academic monitoring tools to identify their strengths and limitations. Drawing from this analysis and the specific operational demands of our laboratory,* ***Section 2.2*** *highlights current research gaps and introduces a set of eight evaluation criteria that a modern monitoring system must satisfy. Finally,* ***Section 2.3*** *explores the underlying technologies required for the proposed AI Support Agent, including Natural Language Interfaces to Databases (NLIDB), LLM-based Text-to-SQL systems and automated visualization platforms.*

2.1 Academic Research Tools

This section explores several well-known monitoring frameworks, which are currently utilized in High-Performance Computing (HPC) environments. For each tool, we examine its underlying architecture, operational strengths and existing limitations, particularly concerning fine-grained observability and user interaction.

2.1.1 Data Center Data Base (DCDB)

DCDB [1] is a modular and comprehensive monitoring tool, specifically created to deal with the increasing complexity of modern HPC data centers and scaling computing requirements. It is developed mainly to provide a unified view of large-scale systems by supporting continuous and high-frequency collection of metrics from various sources.

Its design allows for a wide range of metrics, from facility-level infrastructure sensors to fine-grained application performance data.

Architecture: DCDB is built upon a modular, distributed architecture that employs a push-based data collection strategy using the MQTT protocol, allowing for a flexible many-to-many sensor network topology. The overall architecture of the system is illustrated in Figure 2.1. The system is composed of three key components:

- **Pushers:** Lightweight agents are deployed on each compute node to continuously collect hardware and system performance metrics via a plugin-based interface. Each plugin targets a specific data source such as CPU performance counters (Perfevents), system files (ProcFS/SysFS), or out-of-band sensors (IPMI, SNMP).
- **Collect Agents:** These intermediate components act as MQTT brokers, receiving sensor readings from the Pushers and forwarding them to the Storage Backend for persistent storage. Their primary role is data routing rather than preprocessing.
- **Database:** DCDB uses Apache Cassandra, a distributed NoSQL database, as its primary storage backend. Cassandra is selected for its data distribution mechanism, which allows the database to be spread across multiple server nodes for redundancy and scalability, and for its high write throughput when handling large volumes of time-series data.

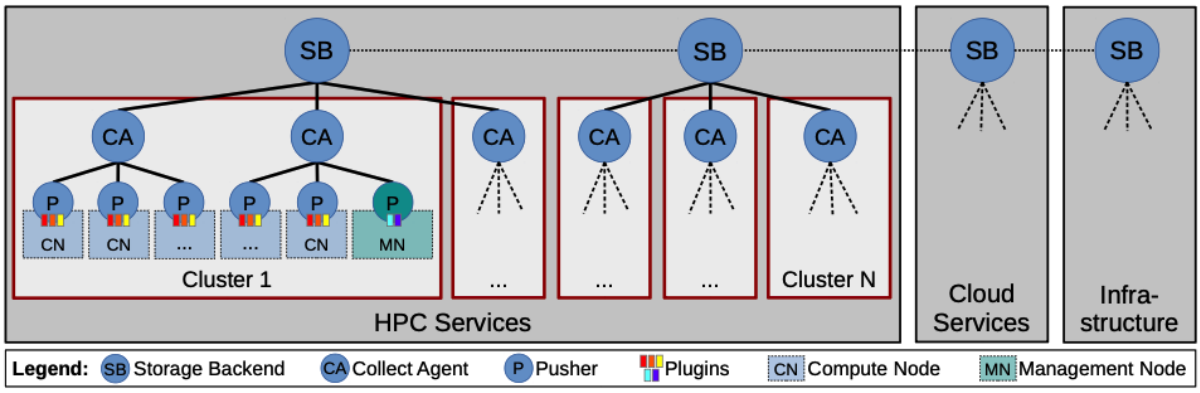


Figure 2.1: DCDB overall architecture.

Strengths: DCDB's push-based and modular architecture ensures fault-tolerant data collection suitable for large-scale HPC installations. One of its strengths lies in its scalabil-

ity, by leveraging MQTT as the communication protocol, the system effectively decouples data generation at the compute nodes from ingestion at the server side, allowing it to handle telemetry streams from thousands of nodes simultaneously without creating significant bottlenecks. The selection of Apache Cassandra as the storage backend provides exceptional write performance, which is critical in supercomputing environments where data volumes grow rapidly. Additionally, the plugin-based Pusher design offers considerable extensibility, allowing administrators to monitor a wide variety of hardware and software sources simply by adding new plugins, without modifying the core framework.

Limitations: However, DCDB has notable limitations relevant to modern HPC monitoring requirements. Although DCDB is capable of collecting fine-grained metrics down to the per-core level, it lacks a native mechanism to associate those metrics with higher-level execution context. Specifically, there is no built-in integration with resource managers such as Slurm or PBS to map sensor readings to JobIDs or UserIDs. As a result, while an administrator can observe that a node is under heavy load, determining which user or job is responsible for the contention remains a manual and time-consuming task. Furthermore, DCDB follows a *store-then-analyze* paradigm, in which data is written directly to the Cassandra backend before any analysis takes place and streaming analytics capabilities are explicitly identified by the authors as future work. This means the system cannot proactively detect anomalies or generate health alerts in real time as data flows through the pipeline. Finally, from a usability standpoint, DCDB relies on static, preconfigured Grafana dashboards for visualization and requires users to interact with the system through raw database queries or command-line tools, creating a significant barrier for non-expert users who need accessible insights into their workloads.

2.1.2 Exascale Monitoring (Examon)

Examon [2] is a scalable and distributed monitoring framework designed to meet the data collection and analytics demands of exascale supercomputing environments. It main focus is on high-frequency, low-overhead collection of hardware and software telemetry, combined with real-time, in-memory data processing for online model learning, distinguishing it from more passive monitoring tools.

Architecture: Examon is organized into four hierarchical layers, as depicted in Figure 2.2. The framework consists of the following core layers:

- **Sensor Collectors:** Lightweight agents deployed on compute nodes gather metrics from a wide range of hardware and software sources, including CPU performance counters (PMU), out-of-band sensors (IPMI), GPUs, batch scheduler interfaces (PBS, Slurm) and low-level hardware interfaces (I2C, PMBus). Each collector implements the MQTT protocol for standardized data delivery to the upper layers.

- **Communication Layer:** Examon uses the MQTT publish-subscribe protocol as its central communication bus, providing low-latency, decoupled data exchange between collectors and consumers.

- **Visualization, Storage and Processing Layer:** Data published to the MQTT broker is consumed by three parallel components: Grafana for real-time visualization, Apache Cassandra for persistent time-series storage and Apache Spark Streaming for real-time, in-memory stream processing and online model learning.

- **User Application Layer:** The top layer accommodates higher-level applications such as infrastructure monitoring dashboards, machine learning pipelines and data analytics tools built on top of the lower layers.

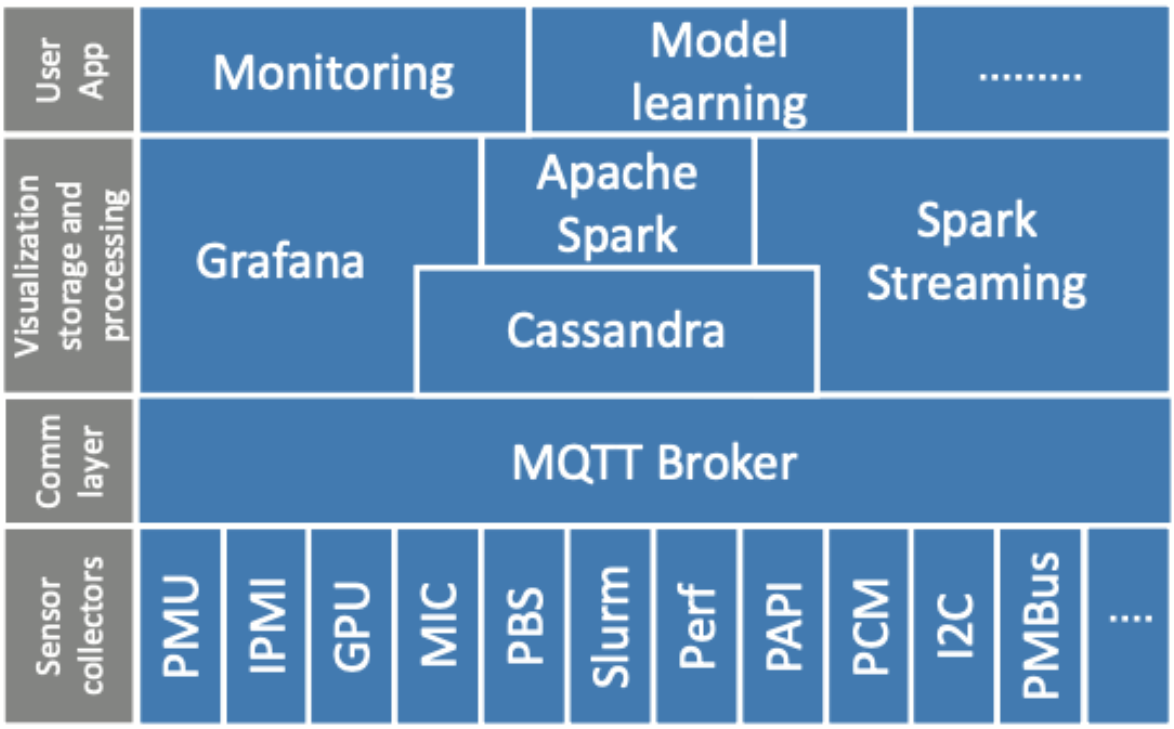


Figure 2.2: Examon overall architecture.

Strengths: Examon's most notable contribution is its integration of Apache Spark Streaming into the monitoring pipeline, which enables real-time, in-memory processing of the data stream as it arrives rather than waiting for it to be written to disk first. This allows the framework to support online model learning, for example, continuously updating CPU power models based on live telemetry, which represents a meaning ful step beyond the *store-then-analyze* paradigm that most contemporary tools follow. The use of MQTT as the communication layer also ensures low overhead and flexible, decouple connectivity between data sources and consumers, making the framework horizontally scalable as the number of monitored nodes grows, ensuring that the monitoring overhead remains low even when thousands of sensors are active simultaneously.

Limitations: Despite its real-time processing capabilities, Examon has several notable limitations in the context of general HPC monitoring. Although the sensor collector layer includes interfaces to batch schedulers such as Slurm and PBS, the MQTT topic structure is organized around hardware entities, including nodes, CPUs and cores rather than execution context. As a result, Examon does not natively associate collected metrics

with specific JobIDs or UserIDs, which limits its ability to support job-centric performance analysis. Furthermore, the framework was primarily developed and evaluated as a specialized solution for a specific production environment, and its architecture lacks the generalized plugin-based extensibility found in tools such as DCDB, making integration of new data sources more involved. From a usability standpoint, Examon relies on static, pre-configured Grafana dashboards for data visualization, which offer limited flexibility for ad-hoc investigation. Users must navigate pre-defined panels and possess sufficient technical knowledge to correlate raw metrics with their application behavior, creating a barrier for non-expert users.

2.1.3 Lightweight Distributed Metric Service (LDMS)

LDMS [3] is a lightweight monitoring framework developed by Sandia National Laboratories and collaborators for large-scale HPC systems. Its primary objective is to provide high-frequency, accurate resource utilization data with negligible impact on the applications running on the compute nodes. By utilizing efficient data transport mechanisms, LDMS is capable of gathering thousands of metrics across thousands of nodes at sub-second intervals on a continuous basis.

Architecture: LDMS is built around a core daemon called `ldmsd`, which can be configured to operate in either sampler or aggregator mode. The high-level architecture is shown in Figure 2.3. The framework consists of the following components:

- **Samplers:** `ldmsd` instances running on each compute node that utilize a modular plugin system to collect metrics from various hardware and software sources, including `/proc` and `/sys` filesystems, CPU performance counters, network interfaces and Lustre filesystem statistics. Each plugin defines a *metric set*, a named collection of related data points sampled together at a user-defined interval.
- **Aggregators:** `ldmsd` instances configured to pull metric sets from samplers or other lower-level aggregators, forming a hierarchical multi-level tree. This pull-based aggregation model distributes the collection and network load efficiently across the cluster. The maximum fan-in per aggregator can reach up to 15,000:1

over Cray's Gemini transport, enabling very large-scale deployments.

- **Transport Layer:** LDMS supports multiple transport backends, including TCP sockets, InfiniBand RDMA and Cray's Gemini RDMA transport. The use of RDMA-capable transports is a key factor in achieving low monitoring overhead, as data is transferred between nodes with minimal CPU involvement on the source node.
- **Storage Plugins:** Once data reaches a top-level aggregator, it can be written to various backends through configurable storage plugins, including CSV flat files, MySQL and the proprietary Scalable Object Store (SOS) format designed for high-throughput time-series writes.

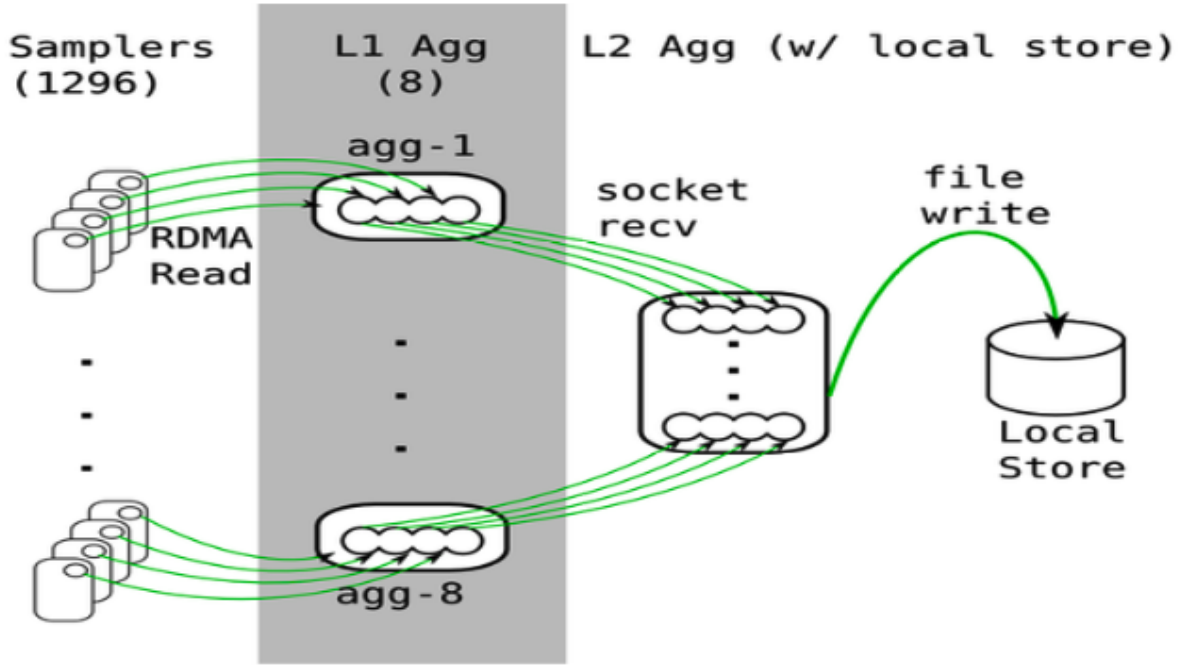


Figure 2.3: LDMS overall architecture.

Strengths: The most distinctive strength of LDMS is its low impact on application performance, even at sub-second sampling intervals. The paper reports CPU utilization of samplers at a few hundredths of a percent of a core in production deployments. The benchmark experiments across multiple applications, including MILC, MiniGhost and CTH shows no significant runtime degradation under LDMS monitoring. The combination of RDMA transport, efficient in-memory metric sets and a hierarchical pull-based architecture makes LDMS one of the most scalable monitoring frameworks available,

capable of sustaining continuous data collection from systems of tens of thousands of nodes. The plugin-based sampler design also provides flexibility for administrators to collect the metrics they need and to develop custom collectors for specialized hardware.

Limitations: Despite its strong collection and transport capabilities, LDMS has several notable limitations. The aggregation schedule is fixed at startup and cannot be modified without restarting the aggregator daemon, which limits operational flexibility compared to the samplers whose intervals can be changed on the fly. While the paper demonstrates that scheduler data can be combined with LDMS metrics to build per-job application profiles, this is described as an ongoing external integration effort rather than a built-in feature of the framework; therefore, job-level and user-level attribution are not natively supported. On the storage side, the available backends at the time of writing are limited to flat files, MySQL and SOS, with no native integration with modern time-series databases optimized for high-frequency monitoring workloads. Furthermore, LDMS provides no built-in visualization or query interface beyond command-line utilities and APIs. Users must rely entirely on external tooling and possess sufficient technical knowledge to correlate raw metrics across nodes and time windows, creating a significant accessibility barrier for non-expert users who need to understand their application's runtime behavior without deep familiarity with the underlying data schema.

2.1.4 Ganglia Monitoring System (Ganglia)

Ganglia [4] is a scalable and hierarchical monitoring system developed for high-performance computing systems such as clusters and Grids. Originally developed at the University of California, Berkeley, it has become one of the most widely adopted monitoring solutions in the HPC community due to its ability to manage federated clusters with low overhead. Its design prioritizes scalability, allowing it to scale to thousands of nodes by leveraging engineered data structures and well-established communication protocols.

Architecture: Ganglia is based on a hierarchical design aimed at federations of clusters. The architecture of Ganglia is illustrated in Figure 2.4. The system operates through several interconnected components:

- **gmond (Ganglia Monitoring Daemon):** A multi-threaded daemon running on every compute node that collects local system metrics including CPU, memory network and load averages then propagates them via a multicast-based listen/announce protocol. As every node both sends and listens on the same multicast channel, each node maintains an approximate view of the entire cluster's state, enabling automatic node discovery and eliminating the need for manual membership configuration.

- **gmetad (Ganglia Meta Daemon):** A federation daemon that polls a configured set of gmond daemons or other gmetad instances via TCP connections, parses the aggregated XML and saves volatile numeric metrics to RRDtool databases for historical visualization. This polling-based federation allows multiple clusters to be aggregated into a unified monitoring tree.

- **RRDtool (Round Robin Database):** The storage and visualization backend used by gmetad to keep time-series data. RRDtool uses fixed-size, constant-overhead databases that automatically consolidate older data at lower resolution, enabling long-term trend visualization at the cost of fine-grained historical precision.

- **gweb:** A PHP-based web frontend that reads from gmetad and renders graphical representations of the stored RRDtool data, providing cluster-wide visualizations across time ranges from minutes to years.

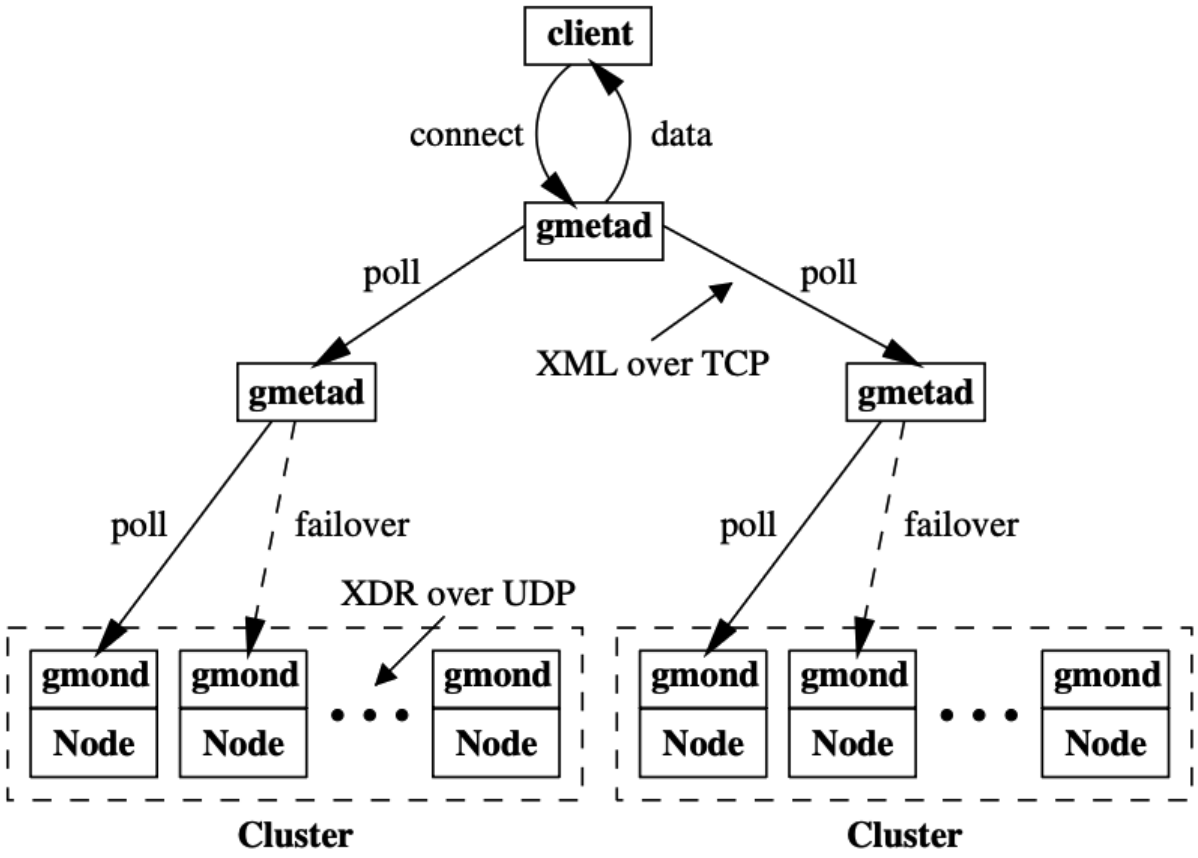


Figure 2.4: Ganglia overall architecture.

Strengths: Ganglia's primary strength lies in its scalable hierarchical architecture, which allows multiple independent clusters to be federated into a unified grid view through a tree of gmetad daemons. The multicast-based listen/announce protocol within each cluster provides automatic node discovery, eliminates manual membership configuration and ensures that cluster state can be reconstructed after node failures. As every node maintains a complete copy of its cluster's monitoring state. Resource overhead is low, the paper reports per-node CPU usage of 0.3–0.4% and a physical memory footprint of under 2 MB per node on production clusters of up to 2,000 nodes. The use of widely adopted standards, XML for data representation and XDR for compact binary transport, making Ganglia compatible with a broad range of external tools and information services.

Limitations: Despite its strengths, Ganglia has several limitations that are relevant to modern HPC monitoring requirements. While the multicast-based protocol is effective for small to medium clusters; however, when the cluster size grows, a scaling concern that the authors themselves identify as a fundamental architectural issue for clusters of thou-

sands of nodes. On the storage side, gmetad's continuous writing to RRDtool databases introduces significant I/O overhead on the aggregation node, which the paper reports can cause performance degradation for interactive workloads on that machine. Furthermore, RRDtool's fixed-size, roll-up consolidation model means that fine-grained, high-frequency data is automatically downsampled over time, making precise post-hoc analysis of transient performance is difficult or impossible. Ganglia's data model is node-centric and operates on a flat metric namespace, it provides no mechanism to associate resource consumption with specific users, jobs, or processes running on a node. As a result, while an administrator can observe aggregate node-level load, identifying which workload is responsible for a particular performance issue requires considerable manual effort. From a usability standpoint, the gweb interface relies on pre-defined PHP templates and static RRDtool graphs, offering no support for ad-hoc queries or interactive investigation. Like the other tools surveyed, Ganglia provides no natural language interface, requiring users to manually correlate information across multiple static panels to diagnose problems in their specific jobs.

2.1.5 Monitoring System for HPC Clusters (MonSTer)

MonSTer [5] is an "out-of-the-box" monitoring framework designed for large-scale HPC systems with an emphasis on ease of deployment and minimal configuration overhead. Rather than requiring custom monitoring agents to be installed on each compute node, MonSTer takes a hybrid approach that leverages existing infrastructure interfaces, specifically out-of-band hardware telemetry via the Redfish API and in-band resource usage data from job schedulers to collect system-wide monitoring data without affecting running applications.

Architecture: It is organized into four functional layers, as illustrated in Figure 2.5:

- **Metrics Collector:** A centralized collecting agent that periodically queries two data sources: (1) the Baseboard Management Controllers (BMCs) on each compute node via the Redfish API over an independent management network, retrieving hardware telemetry such as power usage, CPU and inlet temperatures, fan speeds and system health status; and (2) the resource manager (UGE or Slurm) for job-level

information including JobID, UserID, job start and submit times and the mapping of jobs to nodes. The out-of-band collection approach means no monitoring daemons are deployed on compute nodes, so the system incurs zero intra-node overhead on running applications.

- **Storage (InfluxDB):** Collected metrics are pre-processed and written in batches to InfluxDB, a time-series database, organized into dedicated measurements by data category (Health, Power, Thermal, UGE, JobsInfo, NodeJobs). Pre-processing converts string-based representations to compact binary or integer formats to reduce storage volume and improve query performance.

- **Metrics Builder:** A middleware aggregation layer that abstracts the complexity of querying InfluxDB. It accepts requests from consumers specifying time range, interval and data type, then it generates appropriate InfluxQL query strings, retrieves and aggregates results and returns data in JSON format — optionally compressed to reduce transmission time.

- **HiperJobViz:** A data analysis and visualization tool that consumes the Metrics Builder API to provide interactive visualizations of job scheduling timelines, node health status via radar charts and user-level resource usage comparisons.

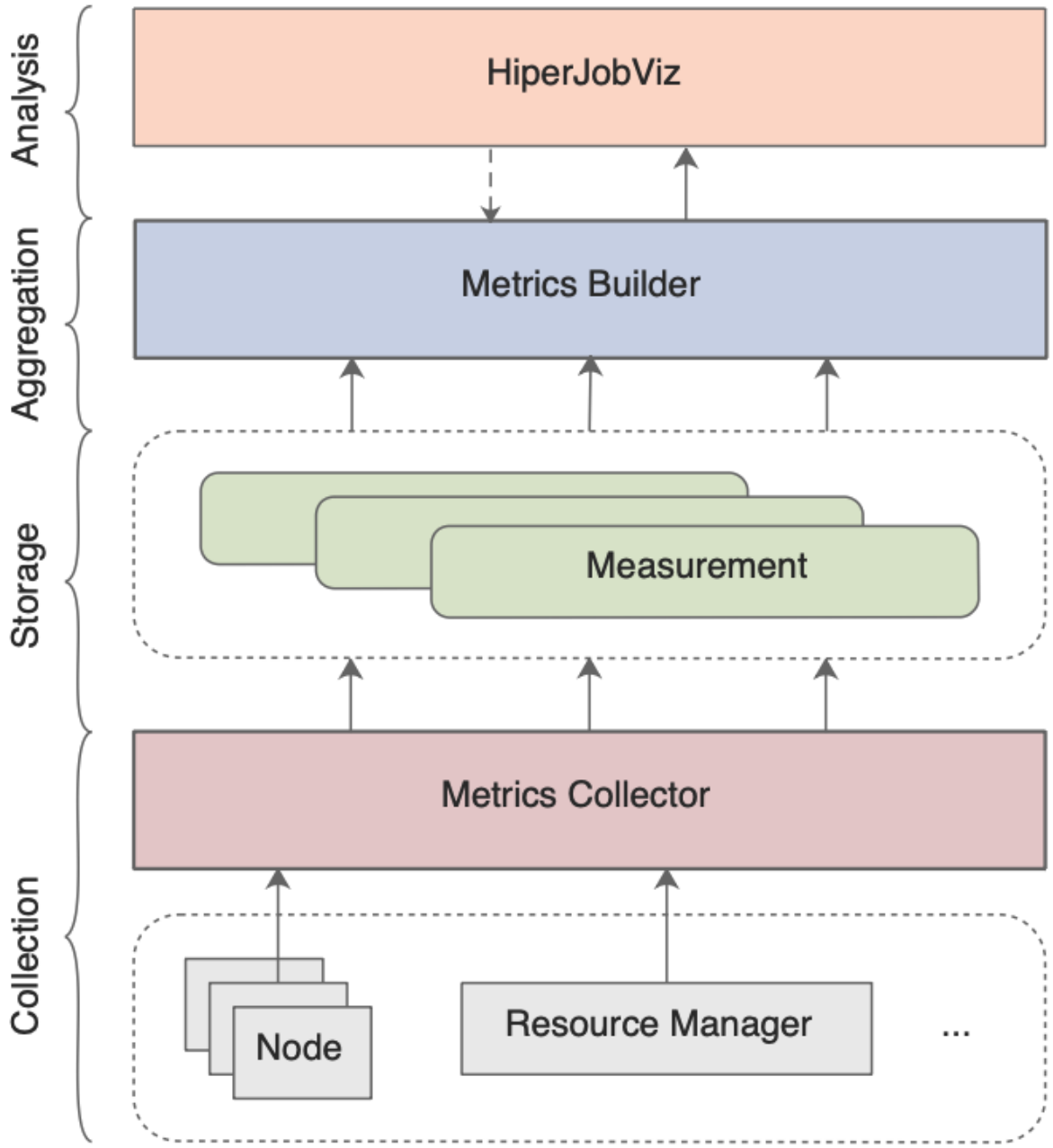


Figure 2.5: MonSTer overall architecture.

Strengths: A primary strength is its agentless, out-of-band collection approach, eliminating the intra-node overhead that affects most other monitoring tools. By leveraging the Redfish API and the resource manager's existing interfaces, MonSTer can collect both hardware telemetry and job-level context without deploying any monitoring daemons on compute nodes. Moreover, MonSTer natively correlates hardware metrics with JobIDs and UserIDs by cross-referencing BMC data with resource manager outputs, enabling administrators to identify which user or job is responsible for abnormal resource consumption. The Metrics Builder middleware further decouples data analysis from data

collection, providing a clean API that simplifies access to historical data without requiring consumers to understand the underlying database schema.

Limitations: Despite its strengths, it has several limitations. The collection interval is constrained by the response time of the BMC ($\sim$ 55 seconds per full node sweep) and the resource manager state update frequency ($\sim$ 40 seconds), resulting in a minimum collection interval of 60 seconds. This eliminates sub-second or even per-second monitoring and means anomalies can only be detected after the data has been written to storage. MonSTer follows a *store-then-analyze* paradigm with no streaming analytics capability. The Metrics Builder generates InfluxDB-specific query strings (InfluxQL), creating a tight dependency on the storage backend that would require significant refactoring to swap for a different database technology. The authors also acknowledge in the paper that MonSTer currently lacks file system and network monitoring capabilities. Finally, while HiperJobViz provides richer visualizations than most tools in this survey, including radar charts and job timeline views but it still relies on pre-defined visual components and does not support natural language querying or conversational interaction, meaning non-expert users must still navigate fixed dashboard views to investigate system behavior.

2.1.6 Cluster Administration Using Relational Databases (CARD)

CARD [6] is an early monitoring system developed at U.C. Berkeley designed to provide flexible, online gathering and visualization of statistics from large clusters of computers. It departs from the monolithic design of contemporary tools by using a relational database as the central data store, decoupling data producers from consumers and enabling ad-hoc SQL queries over collected metrics. CARD gathers both quantitative node-level statistics such as CPU, disk, network and I/O usage, also textual information such as currently executing processes.

Architecture: CARD is organized around a hierarchy of relational databases connected by data-movement processes, as illustrated in Figure 2.6. The main components are:

- **Gather Processes:** Scripts running on each compute node that periodically collect metrics from hardware and OS interfaces (CPU, I/O, network, process information)

and insert them into a local node-level MiniSQL database.

- **Database Hierarchy:** A three-level hierarchy of MiniSQL relational databases, including node-level, mid-level and top-level. Lower-level databases hold high-frequency data from individual or small groups of nodes; upper-level databases aggregate data at reduced frequencies to maintain a complete but slower-changing cluster-wide view.

- **Forwarder and Joinpush Processes:** The forwarder process, associated with each database, accepts SQL requests from consumers, executes them periodically and forwards results upstream. The joinpush process merges updates from multiple lower-level forwarders into the next-level database. Together these processes implement a hybrid push/pull protocol, the consumers send an SQL query along with a count and interval, and the forwarder delivers updates at the specified rate without requiring repeated polling requests.

- **Javaserver and Visualization Applet:** A javaserver acts as a network proxy for a Java-based visualization applet, which displays cluster-wide metrics as strip charts with statistical aggregation, showing average values as bar heights and standard deviation as color shading, allowing hundreds of nodes to be displayed in a single view.

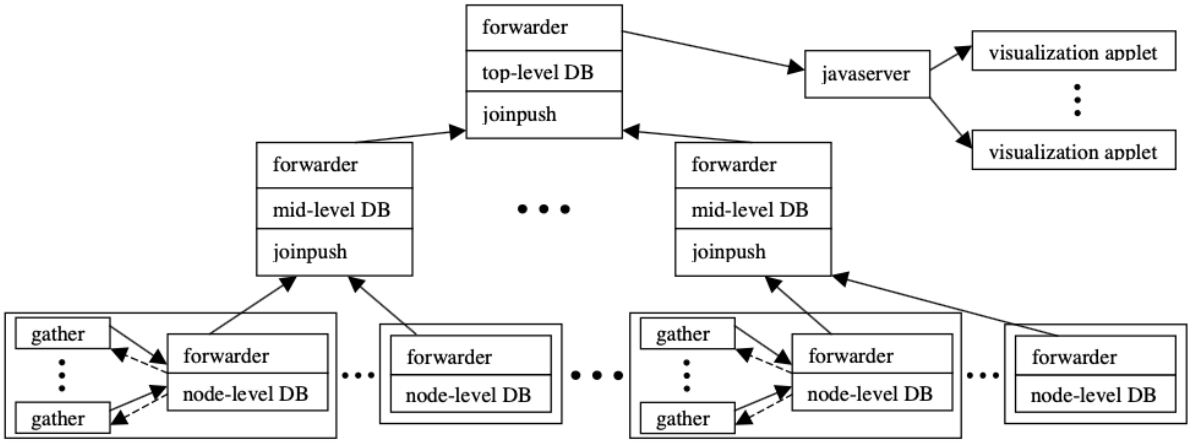


Figure 2.6: CARD overall architecture.

Strengths: CARD's main strength is its use of a relational database as the core storage, decoupling producers from consumers. Any process can insert data into the system by

writing to a table and any consumer can extract data using SQL without requiring modification of existing components. New metrics can be added by introducing new tables or columns without breaking older applications. This design makes CARD extensible as the cluster grows. The hybrid push/pull protocol also improves data transferring, consumers receive data at their requested rate without generating wasted polling packets, and can use SQL expressiveness to select only the data they need. The timestamp-based failure detection and automatic data expiration further add resilience, allowing the system to recover from node and network failures without manual intervention.

Limitations: Despite its design contributions, CARD has several limitations relevant to modern HPC monitoring. The system collects node-level and process-level metrics but lacks integration with job schedulers or resource managers, meaning it cannot associate resource consumption with specific JobIDs or UserIDs. As a result, while an administrator can observe what processes are running on a node, attributing that activity to a particular user's job requires manual cross-referencing. The storage backend, MiniSQL, is lightweight by design and does not support triggers or complex SQL operations, which limits the expressiveness of queries and the ability to implement reactive logic close to the data. The visualization is provided through a Java applet with pre-defined strip chart layouts; while the statistical aggregation approach is practical for cluster-wide overview, it does not support ad-hoc exploration or interactive investigation of specific metrics. Like all tools surveyed, CARD provides no natural language interface, requiring users to possess knowledge of SQL and the database schema to extract specific insights.

2.1.7 A High-Speed Cluster Monitoring System (Supermon)

Supermon [7] is a high-speed, low-impact cluster monitoring system developed at Los Alamos National Laboratory for terascale computing environments. It was built to overcome the performance limitations of the then-dominant `rstatd`-based monitoring tools, which could achieve only around 300Hz sampling rates and had measurable impact on monitored nodes. Supermon's key architectural decision is the uniform use of symbolic expressions (S-expressions) as the data format at every level of the system, from kernel module to cluster concentrator, enabling composable, self-describing data representation

across heterogeneous hardware.

Architecture: Supermon is structured as a three-component hierarchy, as illustrated in Figure 2.7:

- **Kernel Module:** A loadable Linux kernel module that inserts an entry into the `/proc/sys/supermon` filesystem. It exposes hardware and OS metrics, including CPU utilization, network interface statistics, load averages, paging activity and process switch counts in S-expression format. The kernel module achieves sampling rates of 3,400–6,000Hz on production hardware, compared to approximately 300Hz for the `rstatd` interface it replaces.

- **Mon (Single-Node Server):** A user-space daemon running on each compute node that reads data from the `/proc` kernel module entry, adds minimal metadata and serves it to TCP clients on demand. For each connected client, `mon` maintains a bitmask specifying which data fields that client wants, allowing it to filter out unnecessary data and reduce network traffic.

- **Supermon (Data Concentrator):** A data aggregation server that connects to multiple `mon` instances and presents the combined output as a single unified data sample. Multiple Supermon servers can be chained hierarchically, with one Supermon connecting to others that each monitor subsets of nodes. Because all components share the same S-expression protocol, clients interact with the kernel module, `mon`, or Supermon.

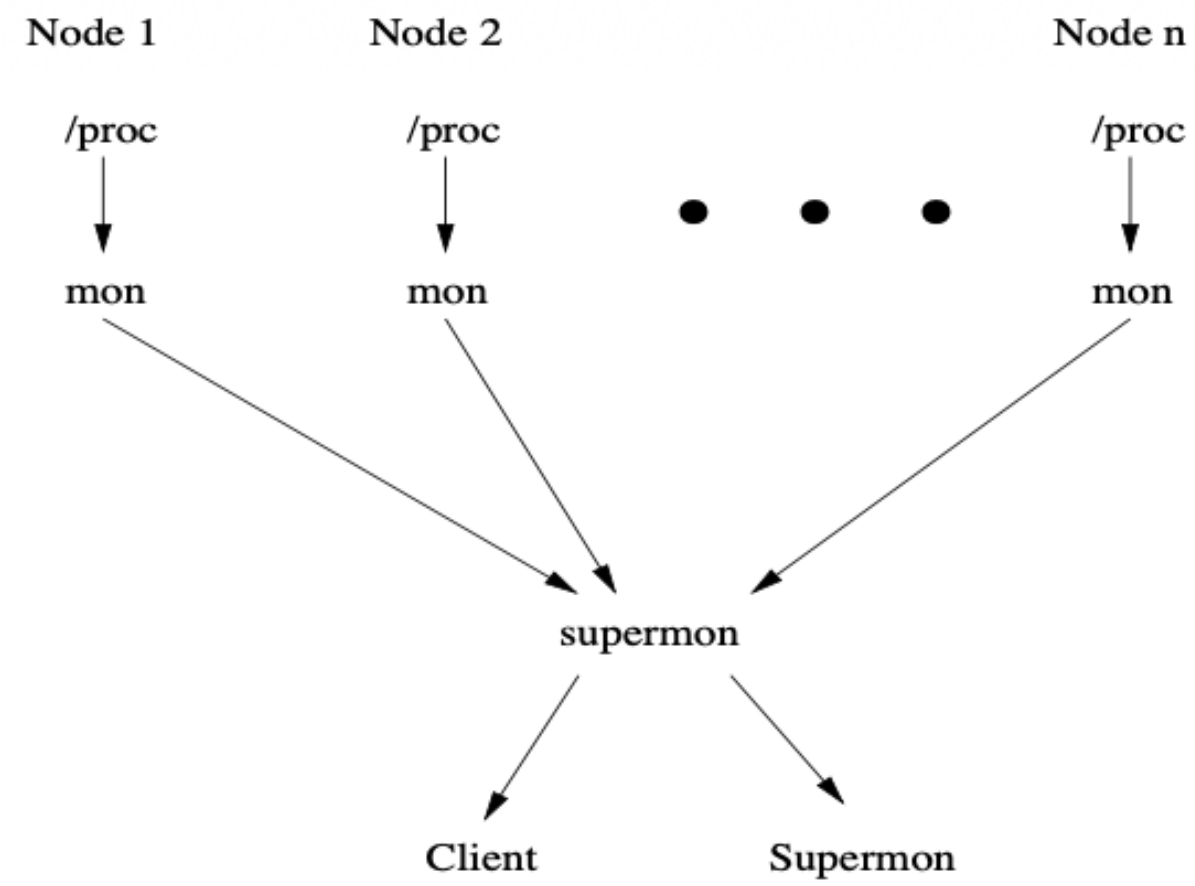


Figure 2.7: Supermon overall architecture.

Strengths: Supermon's most notable strength is its exceptional sampling speed combined with low monitoring overhead. By reading directly from a custom `/proc` kernel module entry rather than relying on RPC-based interfaces, it achieves kernel-level sampling rates over an order of magnitude faster than `rstatd`, reaching up to 6,000 samples per second on a Pentium III-class machine. The uniform use of S-expressions across all system levels is also a genuine strength: the format is self-describing, architecture-independent and naturally handles dynamic hardware configurations such as network interfaces appearing or disappearing at runtime. The protocol design means that any Supermon component can act as both server and client, enabling flexible hierarchical topologies without requiring changes to the data format or parsing logic.

Limitations: Despite its high-speed collection capabilities, Supermon has several significant limitations. The paper's benchmarks reveal a finding: adding hierarchy does not improve scalability. In a 100-node test, a flat single-level Supermon achieved 66Hz,

while hierarchical configurations with fanouts of 10 and 50 achieved only 57Hz and 35Hz respectively, because the additional network traffic between Supermon layers reduced overall throughput. Supermon also provides no persistent storage mechanism; therefore, data is delivered over TCP in real time with no built-in database backend, making historical analysis and long-term trend monitoring impossible without external tooling. The metrics collected are limited to hardware and OS counters from `/proc` (CPU, network, paging, load), with no integration with job schedulers or resource managers. Scheduler integration is explicitly identified in the paper as future work, meaning job-level and user-level attribution of resource consumption is not supported. Finally, like all tools in this survey, Supermon provides no interactive or natural language interface, so clients must connect directly via TCP and parse raw S-expression streams, requiring technical expertise that is inaccessible to non-expert users.

2.1.8 Network Performance Monitoring Tool (NWPerf)

NWPerf [8] is a system-wide performance monitoring tool designed for large-scale Linux clusters. Its goal is to collect high-granularity performance metrics while maintaining a very low impact on user applications. It aims to fill the gap between system-level monitoring and individual application analysis.

Architecture: The architecture of NWPerf is built on a modular and distributed model that prioritizes stability and minimal interference with computational workloads. The system's design is illustrated in Figure 2.8. The framework consists of three main functional tiers:

- **Modular Client:** A daemon running on each compute node that loads modules to collect specific metrics. It transmits data over the network via UDP multicast to minimize connection overhead.
- **Collection Server:** This tier is divided into two components: the Packet-Handler, which receives network packets and places them into a shared memory queue, and the Queue-Drainer, which decodes and inserts the data into the storage.
- **Storage Backend:** A relational database (PostgreSQL) that stores raw metrics and

job-related metadata.

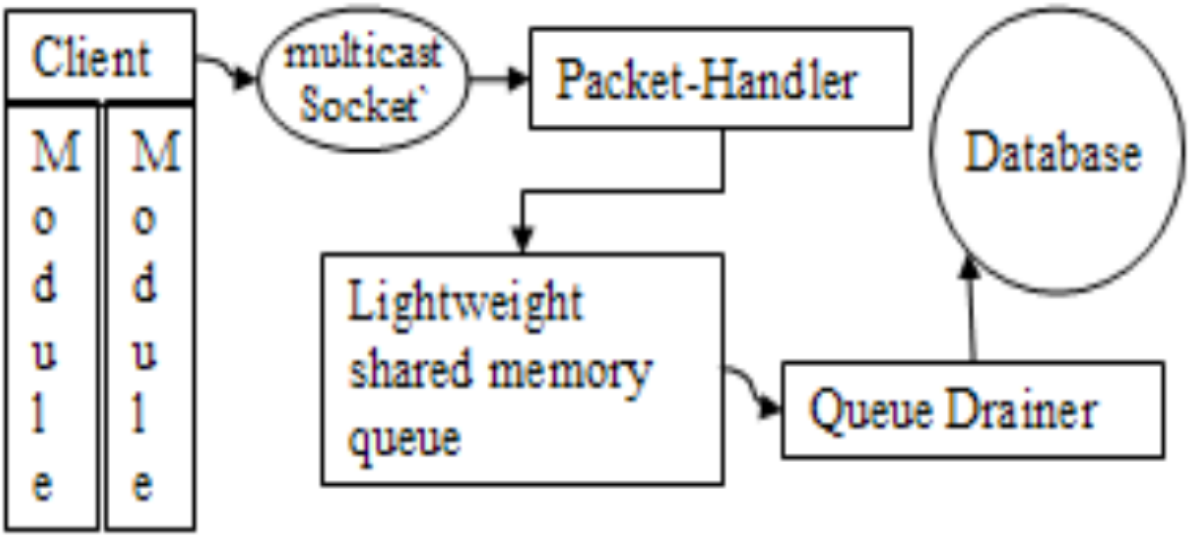


Figure 2.8: NWPerf overall architecture.

Strengths: A standout strength of NWPerf is its ability to facilitate significant performance tuning in production environments. In real-world deployments at PNNL, the tool has been effective in detecting and eliminating critical bottlenecks, with one documented case yielding an application performance improvement of up to 40,000 percent by diagnosing excessive floating-point assists caused by a compiler optimization bug. Its modular architecture ensures that the system is both stable and scalable, capable of supporting large clusters at normal collection rates without measurable overhead to user applications. Furthermore, NWPerf provides a dual-view capability that supports both system-wide aggregate analysis. The first one is tracking metrics such as percent of peak flops and memory bandwidth trends across all jobs and also detailed per-job inspection, enabling administrators to correlate metric interactions and identify anomalous behavior within specific application runs.

Limitations: Despite its historical effectiveness, NWPerf has several limitations that motivate further research. First, its fixed 1-minute sampling interval constrains the temporal granularity of observable events, potentially missing short-lived transient anomalies in modern workloads. Second, the relational database backend presents a storage scalability bottleneck, the paper reports queue drain times approaching 10 seconds when

collecting 12 metrics across 977 nodes under concurrent query load, a figure that will get worse as cluster size and metric count grow. Third, NWPerf's analysis workflow is entirely manual and query-driven, relying on SQL-based post-processing and statistical exploration with no automated anomaly detection and the authors themselves identify this as future work. Finally, NWPerf provides no natural language or conversational interface. Users must formulate SQL queries and interpret raw statistical outputs directly, creating a significant barrier for non-expert researchers who cannot independently diagnose why their specific jobs are underperforming, thus maintaining a reliance on expert system administrators for performance diagnosis.

2.1.9 End-to-End I/O Monitoring And Diagnosis (Beacon)

Beacon [9] is a lightweight, end-to-end monitoring and diagnosis system, designed for large-scale supercomputers, such as the 40,960-node Sunway TaihuLight. It focuses on overcoming the complexities of I/O performance monitoring by simultaneously collecting and correlating performance data from multiple layers, including compute nodes, forwarding nodes, storage nodes and metadata servers.

Architecture: The architecture of Beacon is characterized by its multi-layered data collection strategy, which allows it to correlate events across the entire I/O stack. The system's architecture is illustrated in Figure 2.9. The framework consists of three key functional components:

- **Data Collection Layer:** A collection daemon runs on each compute node using `collectd`, gathering standard system-level metrics (CPU usage, memory, network, I/O) from node-local sources. Fine-grained hardware performance counters (IPC, FLOPS, memory bandwidth, power consumption) are captured via the `LIKWID` library. Crucially, data collection is job-agnostic, the job ID is not known at collection time, which reduces runtime overhead and avoids redundant data when multiple jobs share the same node.

- **Data Storage:** Time-series metrics are stored in a short-term and a long-term time-series database (InfluxDB is used with MetricQ and TimescaleDB noted as alter-

natives). Job metadata and per-job performance footprints are stored separately in a relational database (MariaDB, MySQL, or PostgreSQL).

- **Job Metadata Integration:** Job-specific metadata is collected from the batch system (e.g., SLURM) via prolog/epilog hooks, environment variables and optional queries to SLURM's internal database. Job-specific time-series data are then deduced post-hoc by querying the time-series database using the job's node list, allocated core list, and start/end times.

- **Analysis and Visualization:** An automatic analysis step computes per-job performance footprints and assigns tags (e.g., memory-bound, compute-bound, I/O-heavy) based on threshold comparisons against measured peak values. A custom web frontend (Angular + PHP backend) provides interactive table views, footprint histograms and scatter plots, and per-job timeline charts for both administrators and end-users.

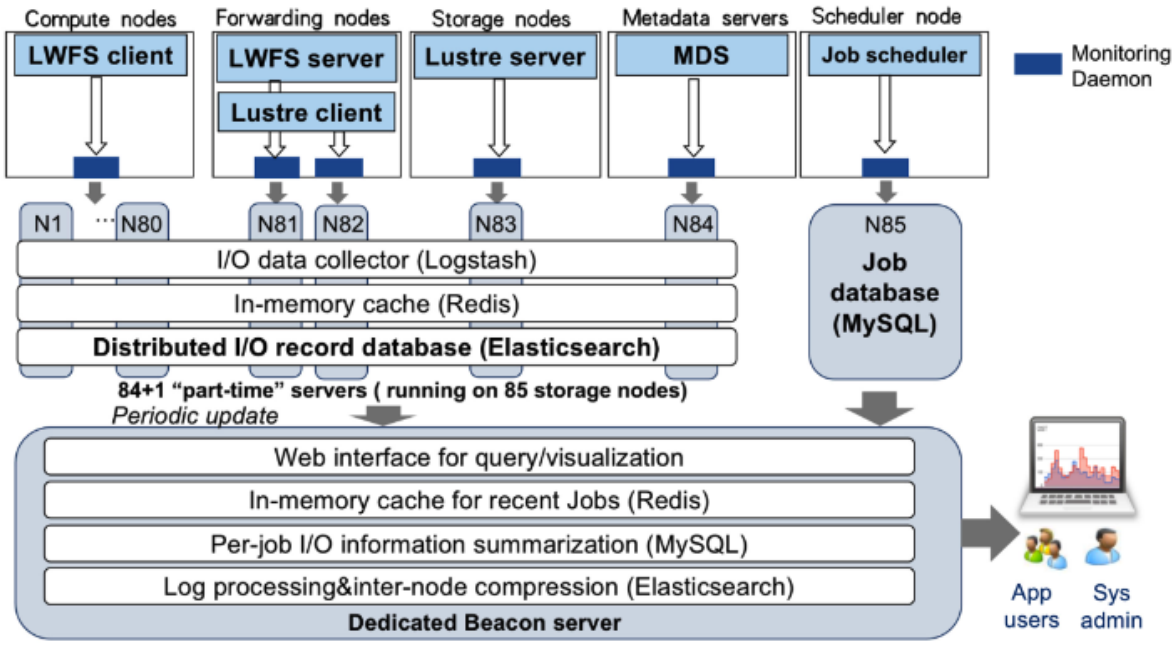


Figure 2.9: Beacon overall architecture.

Strengths: One of the strengths of Beacon is its scalability in production environments, having managed over 40,960 nodes on the Sunway TaihuLight supercomputer for more than three years. Its end-to-end correlation capability is a notable point, by linking compute-node-side traces with forwarding node and OST-level measurements, it enables

administrators to determine whether an I/O slowdown originates from a specific application's behavior, inter-job interference on shared forwarding nodes, or a hardware degradation such as a faulty OST. Furthermore, Beacon's use of distributed caching and two-pass compression ensures that monitoring overhead remains under 1% across all tested production workloads, including jobs using up to 160,000 processes, confirming that the diagnostic process does not degrade the performance it seeks to optimize. Beacon also includes automatic anomaly detection, using clustering algorithms to identify both job-level I/O performance anomalies and node-level stragglers, which has enabled Taihu-Light administrators to fix previously invisible design flaws and configuration problems.

Limitations: Despite its effectiveness, Beacon has several limitations that motivate further research. First, Beacon's monitoring scope is restricted to the I/O subsystem. It does not collect CPU micro-architectural metrics, memory performance counters or application-level computation profiling data. This means that performance problems outside the I/O path such as load-imbalanced computation or memory bottlenecks, remain invisible to Beacon's diagnostic tools. Second, while Beacon provides a web-based GUI for query and visualization, the paper acknowledges that correlation across layers requires users to navigate multi-layer bandwidth timelines and statistics tables and that anomaly diagnosis still requires domain expertise to interpret the results. There is no automated guidance or explanatory feedback for non-expert application users beyond the raw performance charts. Third, Beacon's anomaly detection requires job history accumulation before it can take effect, it is states that applications need to accumulate at least several executions for the clustering-based detection to function, making the system ineffective for new or infrequently run applications. Finally, like the other tools surveyed, Beacon provides no natural language or conversational interface. Users must formulate explicit queries using job IDs and interpret technical visualization outputs directly, maintaining a reliance on expert administrators for root cause diagnosis.

2.1.10 Center-Wide and Job-Aware Cluster Monitoring (PIKA)

PIKA [10] is a job-aware monitoring infrastructure designed to characterize HPC jobs and identify underperforming tasks for optimization. Developed at TU Dresden, it fo-

cuses on continuous monitoring and analysis without intrusive operations like code instrumentation. PIKA fills the gap between raw hardware metrics and job-specific metadata, providing both live and post-mortem insights into system efficiency.

Architecture: The architecture of PIKA is built upon a modular and highly scalable pipeline that integrates metadata from the resource manager with high-frequency time-series data. The high-level design is illustrated in Figure 2.10. The framework consists of the following core stages:

- **Data Collection Layer:** PIKA utilizes a dual collection strategy. It employs `collectd` for gathering standard system-level metrics (e.g., CPU load, memory usage) and integrates the `LIKWID` library to capture fine-grained, job-specific hardware performance counters without introducing significant overhead.
- **Data Ingestion and Storage:** Telemetry data is pushed to a centralized storage backend, utilizing InfluxDB, optimized for high-write-throughput time-series data.
- **Job Metadata Integration:** A critical component of PIKA is its ability to poll the workload manager (e.g., Slurm) to retrieve job-specific metadata, such as Job IDs, user names, and allocated nodes. This metadata is then used to ”tag” the performance metrics in real-time.
- **Analysis and Visualization:** PIKA provides both live and post-mortem visualization through customized Grafana dashboards, offering dedicated views for both system administrators and end-users.

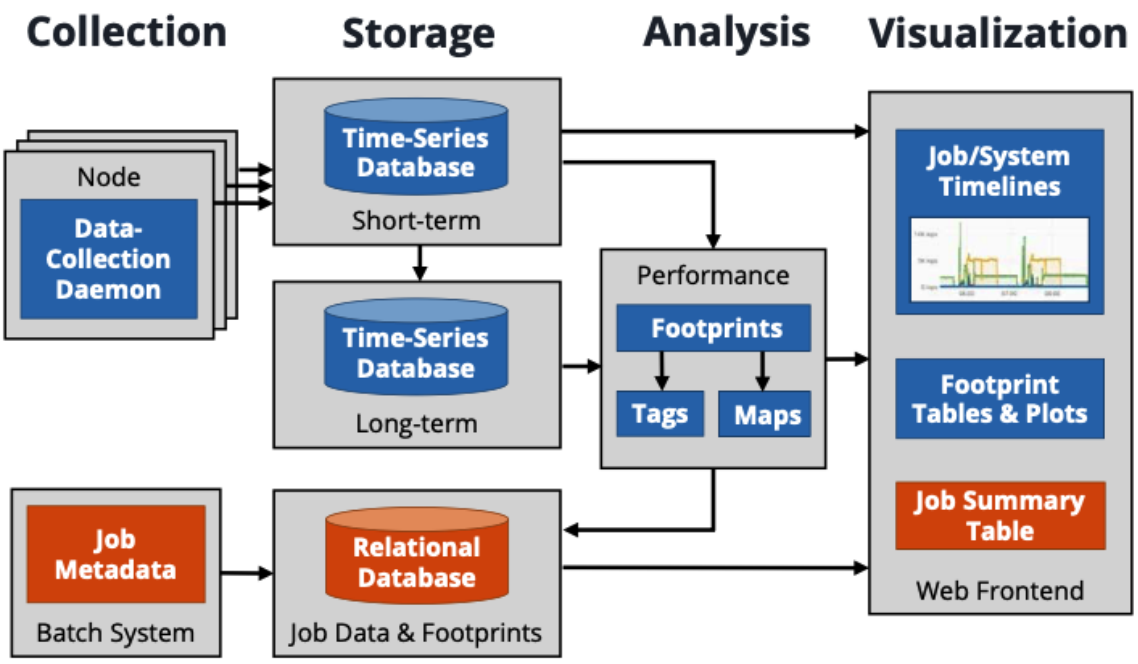


Figure 2.10: PIKA overall architecture.

Strengths: One of PIKA's strengths is its job-awareness. Unlike traditional node-centric tools, PIKA automatically correlates system metrics with specific jobs, allowing administrators to identify which jobs are wasting resources or underperforming. Its design with `collectd` plugins makes it adaptable to evolving hardware and it has been demonstrated to scale to over 10,000 nodes. The monitoring overhead is negligible, even at a one-second sampling interval, the maximum observed runtime increase was 1.16% and at the standard 60-second interval it was around 0.3% or lower. The automated performance footprint and tagging system provides actionable characterization of jobs without requiring any application instrumentation, empowering both operators and researchers to identify optimization opportunities at the facility level.

Limitations: Despite its advanced job-awareness features, PIKA has several limitations that motivate further research. First, the performance tagging system relies entirely on static, pre-defined thresholds, the paper acknowledges that the proposed thresholds must be adjusted incrementally to each specific environment and that approximately 91% of all completed jobs are tagged as "unrestrained", this means the system cannot characterize them more specifically. This suggests the current rule-based categorization is insufficiently discriminating for the majority of real-world jobs. Second, it is stated that the

unrestrained category as needing further refinement through additional lower-bound tags as future work, indicating that the analysis layer remains incomplete. Third, while PIKA provides an interactive web frontend, navigating from a performance anomaly to its root cause still requires manual, expert-driven exploration of timeline charts, histograms and scatter plots. Even the system provides no automated guidance to help users interpret what the metrics mean for their specific job. Finally, like all tools surveyed, PIKA provides no natural language or conversational interface. Users must interact with structured query forms and interpret raw metric visualizations directly, maintaining a reliance on expert knowledge for performance diagnosis.

2.2 Industrial Tools

This section surveys three adopted industrial monitoring frameworks that are commonly encountered in production HPC and enterprise IT environments. Unlike the academic tools reviewed in Section 2.1, these tools are actively maintained commercial or community-driven products with large user bases and broad deployment histories. For each tool, we examine its underlying architecture, operational strengths and existing limitations, particularly concerning fine-grained observability, job-level context and user interaction.

2.2.1 Zabbix

Zabbix [11] is a mature, open-source monitoring platform originally developed by Alexei Vladishev in 2001 and maintained since then by Zabbix LLC. It is one of the most widely deployed monitoring solutions in production IT environments, with reported adoption across more than 300,000 installations including numerous Fortune 500 companies. Unlike the academic tools surveyed in the previous section, Zabbix is designed as a general-purpose, enterprise-grade infrastructure monitoring platform focusing on networks, servers, virtual machines, applications and cloud services. Its inclusion here is motivated by the fact that it represents the kind of mature, production-ready tool that HPC administrators often reach for when building a monitoring infrastructure, and understanding its capabilities and limitations in the HPC context is necessary for positioning the proposed framework.

Architecture: Zabbix is built around a three-tier architecture (Figure 2.11)consisting of a central server, distributed agents and a web-based frontend. The Zabbix server is the core component: it collects data from agents and other sources, evaluates trigger conditions, sends alerts and stores data in a relational database. Supported backends include PostgreSQL, MySQL/MariaDB, Oracle and TimescaleDB as a PostgreSQL extension for time-series optimization. For distributed environments, Zabbix proxies collect data at remote sites and forward it to the central server, reducing bandwidth and providing local buffering if the connection is interrupted. Two agent variants are available: the original agent written in C, which collects standard system metrics such as CPU, memory, disk and network statistics, and the newer Agent2 written in Go, which adds a plugin system for extending data collection to databases, message queues and cloud APIs. Beyond agent-based collection, Zabbix supports agentless monitoring through SNMP, IPMI, JMX and HTTP-based protocols, allowing it to monitor systems where installing an agent is not feasible.

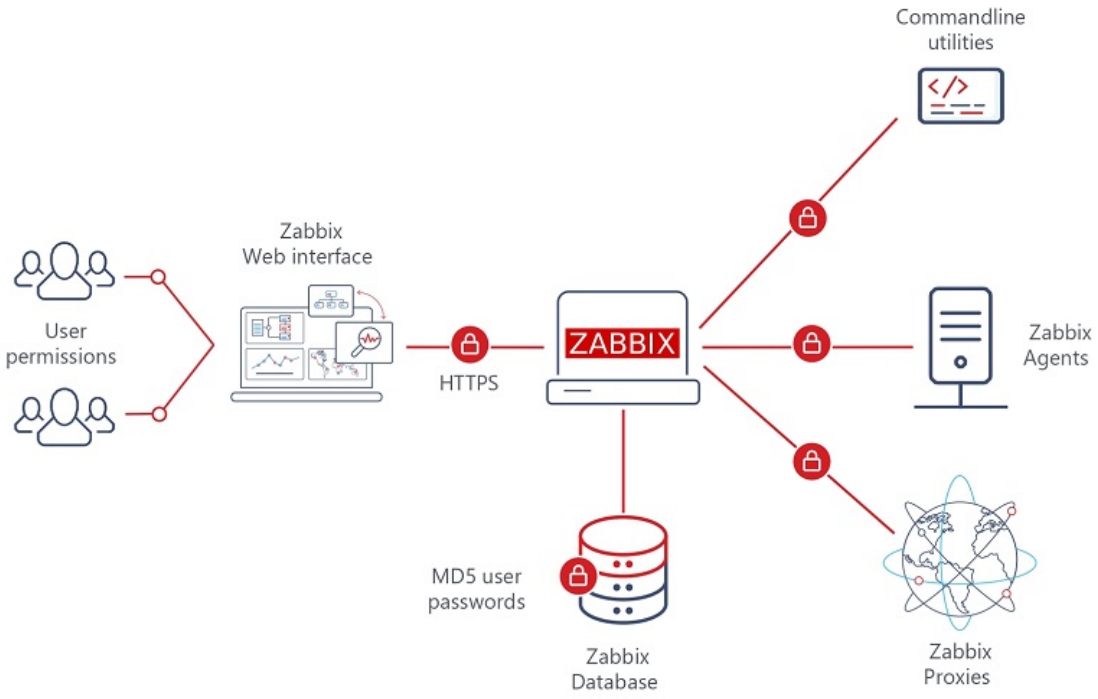


Figure 2.11: Zabbix overall architecture.

Strengths: Zabbix's primary strengths lie in its breadth of coverage, its operational maturity and its alert management capabilities. Low-level discovery automatically detects

and monitors new resources such as network interfaces, file systems or database instances without manual configuration and templates define reusable sets of items, triggers, graphs and discovery rules that can be applied across hosts. The alert system is comprehensive: administrators can define complex trigger conditions, configure multi-channel notification delivery including email and messaging services, and benefit from event correlation that strengthens related alerts into single events to reduce noise. Its open-source nature enables it to be extended with custom scripts, plugins and modules, and it supports both agent-based and agentless monitoring through various protocols, allowing it to monitor systems where installing an agent might not be permitted. Zabbix has also seen real-world deployment in HPC environments: a documented case study at a chemistry faculty cluster of 584 dual-socket compute nodes with 25,652 cores demonstrates that Zabbix can be adapted to monitor HPC infrastructure alongside standard IT equipment.

Limitations: Despite its maturity and breadth, Zabbix presents several significant limitations when evaluated against the requirements of fine-grained HPC monitoring. Its data collection model is mainly node-centric and infrastructure-oriented, it collects aggregate system metrics per host but has no native mechanism for correlating those metrics with HPC-specific execution context such as JobIDs, UserIDs or running processes to specific users. In the HPC context, there are no native GPU utilization metrics in Zabbix, although it is possible to create new metrics using the UserParameter feature — a workaround that requires custom scripting for each metric type rather than built-in support. The storage backend, while supporting relational databases including TimescaleDB, follows a *store-then-analyze* mechanism with no streaming analytics capability because data is written to the database and analyzed after the fact, which is insufficient for real-time anomaly detection on a continuously evolving cluster workload. From a usability perspective, Zabbix has a learning curve and its installation and configuration can be complicated, requiring considerable technical expertise, these characteristics limit its accessibility for non-expert users in a shared HPC environment. Its web frontend provides dashboards and reports, but these are pre-configured static views with no support for natural language querying or interactive ad-hoc investigation.

2.2.2 Prometheus

Prometheus [12] is an open-source systems monitoring and alerting toolkit originally built at SoundCloud. Since its establishment in 2012, many companies and organizations have adopted it and the project joined the Cloud Native Computing Foundation in 2016 as the second hosted project after Kubernetes. It has since become the standard for metric-based monitoring in cloud-native and microservices environments, and its exporter ecosystem has made it increasingly important in HPC contexts as well. Unlike Zabbix, which is oriented toward infrastructure health and alerting, Prometheus is fundamentally a *metrics collection and query platform*, its design is optimized for storing and querying dimensional time-series data rather than for broad infrastructure management.

Architecture: Prometheus collects and stores its metrics as time-series data, metrics information is stored with the timestamp at which it was recorded, alongside optional key-value pairs called labels. It stores all scraped samples locally and runs rules over this data to either aggregate and record new time-series from existing data or generate alerts. The system is organized around several core components. The Prometheus server is the central component (shown in Figure 2.12), responsible for scraping and storing metrics using a local time-series database (TSDB). Exporters are external components that expose metrics in a format Prometheus can scrape, for example, the Node Exporter provides Linux system metrics, while purpose-built exporters exist for databases, message queues and cloud APIs. The Alertmanager is a separate component that handles notifications: when an alerting rule is met, Prometheus fires the alert to the Alertmanager, which then deduplicates, groups, silences and routes notifications to the appropriate destination such as email, Slack or PagerDuty. A defining architectural characteristic that distinguishes Prometheus from most tools in this survey is its **pull-based collection model**: Prometheus operates by scraping metrics from predefined data sources at regular intervals, rather than receiving data pushed from monitored targets. Targets simply expose a `/metrics` HTTP endpoint and Prometheus polls them at the configured scrape interval.

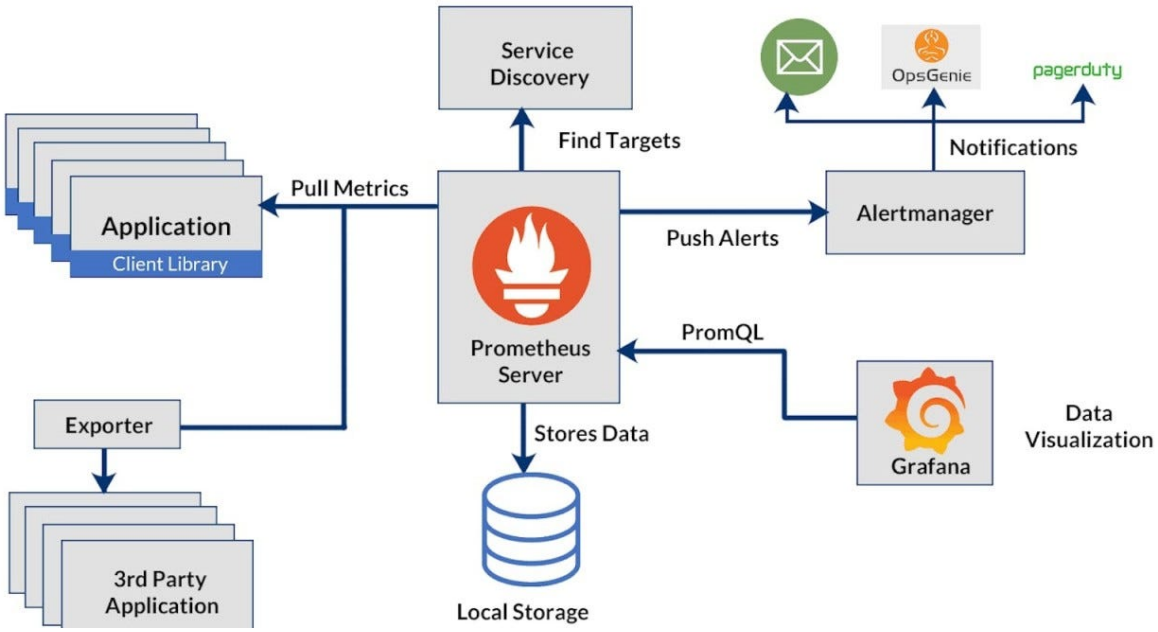


Figure 2.12: Prometheus overall architecture.

Strengths: Prometheus's most significant strength is its **multi-dimensional data model** and the expressive power of its query language, PromQL. Time-series data are identified by a metric name and a set of key-value pairs and the PromQL query language allows users to query, correlate and transform time-series data in powerful ways for visualizations, alerts,... Alerting rules are based on PromQL and make full use of the dimensional data model. This label-based dimensional model is crucial in HPC contexts because it allows metrics to be tagged with arbitrary metadata including node ID, job ID, queue name or user name and then queried and aggregated across those dimensions. When combined with a Slurm exporter or a custom job scheduler exporter, Prometheus can associate node-level metrics with job and user context through label enrichment, providing a degree of application-level visibility that most of the academic tools surveyed earlier do not support natively. Prometheus servers operate independently and only rely on local storage, making deployment simple, where the linked Go binaries are easy to deploy across various environments. Its integration with Grafana for visualization is also well-established, making it easy to build rich dashboards on top of collected metrics without additional development effort.

Limitations: Despite its strengths, Prometheus has several architectural limitations that are relevant to large-scale HPC monitoring. The first is its **local storage constraint**. A

native Prometheus server is not a great long-term storage solution: when trying to store large-scale data over multiple years, there is a risk of crashing the reliability limits of a single machine or its maximum disk size. Furthermore, Prometheus has no concept of multi-tenancy or per-user overload controls. Prometheus also lacks built-in down-sampling capabilities, meaning storage costs grow linearly with retention, and queries requesting data from extensive timeframes can cause Prometheus instances to run out of memory. Addressing these limitations requires integrating external solutions such as Thanos or Cortex, which add significant operational complexity. The **pull-based model**, also presents a challenge in HPC environments where compute nodes run short-lived jobs: these tasks pose an obvious challenge for Prometheus's pull model, since the server cannot scrape something that is not running. The Pushgateway workaround introduces a risk of old data, if a job succeeds on one day but fails to run on the next, its old success metrics will still be served, making monitoring potentially misleading. Finally, while Prometheus's label model can support job and user attribution through careful exporter design, this is not a built-in capability because it requires manual configuration of exporters and label enrichment, and the system has no integration with HPC resource managers. There is also no message broker layer for decoupled ingestion, no streaming analytics capability and no natural language interface since the query interface requires proficiency in PromQL, which is non-trivial for non-expert users.

2.2.3 Nagios

Nagios [13] is an open-source network and infrastructure monitoring system written by Ethan Galstad in 1999 under the name NetSaint and renamed Nagios in 2002. It monitors hosts, services and network devices, sending alerts when components fail and again when they recover. What started as a solution to monitor network services has evolved into one of the most widely adopted open-source monitoring platforms, trusted by over one million users worldwide. Nagios is relevant to this survey because it is one of the oldest and most commonly encountered monitoring tools in institutional IT environments, and it has seen use in HPC data centers as an infrastructure health and alert management layer. Its design is different from both Zabbix and Prometheus: whereas those tools emphasize metric collection and visualization, Nagios is primarily an *alert-oriented* framework,

which its core purpose is to detect when something has gone wrong and notify the right people, rather than to provide rich analytical insight into system behavior.

Architecture: Nagios uses a plugin-based architecture (illustrated in Figure 2.13): the core scheduler runs check plugins at configured intervals, processes the results and triggers notifications. Hundreds of community-developed plugins exist for monitoring specific services and hardware. The system supports two complementary check models. Active checks are initiated by the Nagios Core process at regularly scheduled intervals, when a check is due, Nagios executes the corresponding plugin, which tests the operational state of the host or service and reports the results back to the Nagios daemon, which then takes appropriate action such as sending notifications or running event handlers. By contrast, passive checks are initiated and performed by external applications, with the results submitted to Nagios Core for processing, this is useful for monitoring services that are asynchronous in nature and cannot be monitored effectively by polling or for hosts located behind a firewall that cannot be checked actively from the monitoring server. For remote monitoring, the Nagios Remote Plugin Executor (NRPE) runs plugins on remote Linux and Unix systems and returns results to the Nagios server. An optional web interface is provided for viewing current network status, notification history and problem logs, though it is minimal by design.

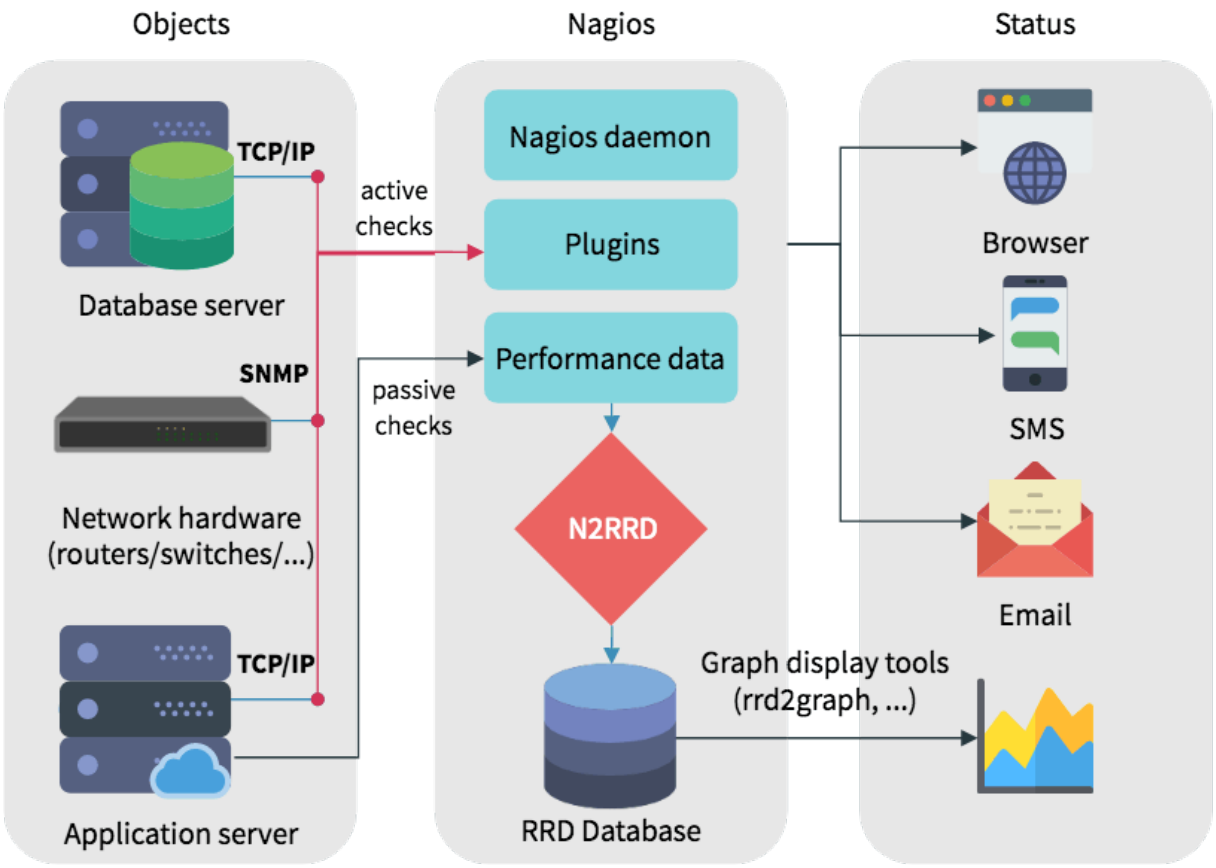


Figure 2.13: Nagios overall architecture.

Strengths: Nagios's primary strength is its alert management capability, which remains among the most mature and flexible in the open-source monitoring landscape. It supports multiple notification methods including email, pager, SMS and user-defined scripts, with rising paths that route alerts to different teams based on severity, time-based alerting that schedules notifications based on time of day. This rich alerting model is well-suited to HPC environments where administrators need to be immediately notified of node failures, service outages or threshold violations across a large cluster. The plugin-based architecture has also proven highly impactful as its design inspired several derivative projects including Icinga, Naemon and Checkmk, and the Nagios Exchange hosts thousands of community-contributed plugins covering an enormous range of services, hardware types and operating systems. The simplicity and lightweight nature of Nagios Core is also valued in practice: the system can run on modest hardware and has been known to operate reliably for years without significant maintenance overhead.

Limitations: Nagios's design decisions that made it effective as an alert engine are what

limit its usefulness for fine-grained HPC performance monitoring. Its most significant limitation is the absence of a time-series storage backend. Nagios does not gather performance data from the past, it only shows performance statistics as of the current moment, making it challenging to perform trend analysis or evaluate network health over time. Performance data produced by plugins can be exported to external tools such as Grafana or InfluxDB, but this is not a built-in capability, it requires additional configuration and third-party integration. The check-based polling model is also coarse-grained as it checks execute at intervals of minutes rather than seconds, which is insufficient for detecting temporary performance events in a busy HPC cluster. Scalability is a significant concern as the number of monitored hosts grows when users must deploy multiple Nagios servers or make use of third-party solutions to increase capacity. Like Zabbix and all other tools in this survey, Nagios has no native integration with HPC job schedulers and therefore cannot associate node-level metrics with specific JobIDs or UserIDs. There is no message broker layer, no streaming analytics capability, no natural language interface and the web interface. While it is functional for alert management, which is a CGI-based static display that is not suitable for ad-hoc metric investigation or interactive querying.

2.3 Research Gaps and Evaluation Criteria

This section shows the findings from the comprehensive review of existing monitoring tools to identify the critical research gaps that persist in the current HPC landscape. Moreover, it defines a set of eight evaluation criteria, structured across four key dimensions: Scope and Context, System Architecture, Data Processing Strategy and Usability. These criteria serve as an objective framework for benchmarking existing solutions and guiding the design of the proposed smart monitoring architecture.

2.3.1 Research Gap Analysis

The previous review of ten academic HPC monitoring tools in **Section 2.1** reveals a consistent pattern of strengths and limitations that define the requirements for a next-generation monitoring framework. While each tool makes valuable contributions in specific areas, none addresses the operational demands of a modern, multi-user HPC envi-

ronment. Before establishing our evaluation criteria, we synthesize the recurring gaps observed across the surveyed tools into four dimensions.

Lack of Application-Level Context. The most general limitation across the surveyed tools is the absence of job-level and user-level visibility. Tools such as DCDB, Examon, LDMS, Ganglia, CARD and Supermon all operate at the node level, reporting aggregate hardware metrics without correlating them to the users or jobs responsible for resource consumption. Even tools that collect from scheduler interfaces such as Examon with its Slurm and PBS collectors organize their data around hardware entities rather than execution context, leaving job-level attribution as an external, manual concern. Only MonSTer and NWPerf natively associate metrics with JobIDs and UserIDs, and PIKA does so through its integration with SLURM. This gap is critical, in a shared HPC environment where tens of users submit hundreds of concurrent jobs, an administrator who can only observe that a node is overloaded but not which user or application caused it as having incomplete information for effective resource management.

Architectural Rigidity and Scalability Constraints. Several surveyed tools suffer from tightly coupled architectures that limit their long-term maintainability and adaptability. Ganglia's gmond, gmetad and RRDtool components are interdependent, making it difficult to replace the storage backend without refactoring the entire visualization pipeline. MonSTer's Metrics Builder generates InfluxDB-specific query strings, creating a hard dependency on InfluxQL that would require significant engineering effort to migrate. Supermon has no persistent storage at all. More broadly, most tools in this survey write data directly to a database or filesystem without any intermediate buffering layer, meaning they are vulnerable to bottlenecks and data loss under high ingestion loads. Only a handful of tools, DCDB and Examon employ a message broker for decoupled, fault-tolerant data transport and even these do not leverage a high-throughput broker like Apache Kafka. As HPC clusters scale to thousands of nodes generating millions of metrics per second, the lack of a scalable, decoupled data infrastructure becomes a fundamental barrier.

Insufficient Data Processing Capabilities. Most of surveyed tools follow a *store-then-*

analyze paradigm, where raw metrics are first persisted to a database and then retrieved for analysis afterward. DCDB, Ganglia, NWPerf, Beacon, MonSTer, CARD and Supermon all exhibit this pattern to varying degrees. This introduces latency that is incompatible with real-time management, by the time an anomaly is visible in the stored data, the window for timely intervention may have already closed. Only Examon, through its integration of Apache Spark Streaming, demonstrates genuine in-memory, real-time stream processing. The next problem that also need to be mentioned is the preprocessing stage. Transmitting raw, high-frequency metrics from every node to a central server imposes substantial network overhead. While some tools perform basic filtering or aggregation at the node (Examon, Ganglia, PIKA) or at the server (LDMS, CARD, PIKA), few do both as part of a designed pipeline. The result is architectures that either overwhelm the network with raw data or sacrifice metric granularity through aggressive reduction.

Poor Accessibility for Non-Expert Users. Across all tools surveyed, a consistent gap is the complete absence of intelligent, interactive user interfaces. Every tool reviewed, from the relatively modern PIKA and Beacon to the early CARD and Supermon relies on static, pre-configured dashboards, command-line utilities or raw query interfaces as the primary means of data access. Users must possess deep technical knowledge of the underlying database schema, metric naming conventions and visualization tooling to extract meaningful insights. This creates a significant barrier in multi-user academic HPC environments, where the majority of users are researchers and lecturers rather than systems experts. When a user needs to understand why their job is running slowly or consuming unexpected resources, they currently have no choice but to navigate dozens of static panels manually. This is both time-consuming and cognitively demanding. The absence of natural language interfaces or automated query assistance across the entire surveyed landscape represents perhaps the clearest opportunity for meaningful innovation.

2.3.2 Evaluation Criteria

Drawing from the research gaps identified in the previous analysis, we establish eight evaluation criteria organized into four dimensions. These criteria serve as a framework for assessing the capabilities of existing monitoring tools and will guide the design vali-

dation of our proposed architecture in subsequent chapters.

The first two criteria address **Scope and Context**.

Criterion 1 - Application-level Monitoring determines whether a tool can map physical resource usage, such as CPU load or memory consumption, to abstract high-level entities like JobID, UserID or specific applications. A tool satisfies this criterion if its architecture integrates with a resource manager such as Slurm or PBS or if it supports tagging metrics with job and user metadata. In a multi-user HPC environment, monitoring hardware metrics in isolation is insufficient; administrators must be able to manage resource consumption to specific workloads in order to resolve contention and make informed scheduling adjustments.

Criterion 2 - Target Scope classifies whether a tool is general-purpose or application-specific. A general-purpose tool monitors the overall health of the cluster across hardware, operating system and network dimensions, whereas an application-specific tool is designed for a domain such as power anomaly detection or I/O profiling. Given the requirements of the laboratory environment, a general-purpose approach is preferred as it provides the visibility needed to manage a shared, multi-workload cluster effectively.

The next two criteria address **System Architecture**.

Criterion 3 - Design Philosophy assesses the architectural structure of the framework in terms of how its components are organized and coupled. A *modular* design consists of independent, interchangeable components, such as a data collector, aggregator and storage backend that communicate via well-defined standard protocols. In such a design, any individual component can be replaced or upgraded in isolation without requiring changes to the rest of the system; for example, swapping the storage backend from Cassandra to InfluxDB should not affect the collection or aggregation layers. A *monolithic* design, by contrast, embeds tight interdependencies between components such that the system functions as a single indivisible unit; therefore, modifying or replacing one part inevitably requires refactoring others, increasing both maintenance effort and the risk of vendor lock-in. A modular architecture is therefore a prerequisite for the long-term maintainability and adaptability that a production HPC monitoring framework demands.

Criterion 4 - Data Infrastructure examines the mechanism used for data transport and ingestion. It distinguishes between systems that write data directly to a database and those that route data through a dedicated message broker such as Apache Kafka or RabbitMQ before persistence. A *direct-to-database* approach creates ingestion bottlenecks under high load and couples producers tightly to consumers, whereas a broker-based infrastructure provides buffering, fault tolerance, and the ability to scale ingestion independently from processing and storage.

The following three criteria address **Data Processing Strategy**.

Criterion 5 - Preprocessing at the Computing Node evaluates whether the monitoring system performs local data transformation, such as filtering, sampling, aggregation or compression at the compute node before transmitting data upstream. Transmitting raw, high-frequency metrics from every node to a central server places substantial pressure on network bandwidth, making edge preprocessing an essential capability for any framework targeting fine-grained, continuous monitoring at scale.

Criterion 6 - Preprocessing at the Server checks whether the central server or middleware layer performs data transformation, such as downsampling, summarization or derived metric calculation before writing data to a storage backend. Storing continuous streams of raw telemetry is both costly and inefficient for downstream queries; server-side preprocessing ensures that what reaches persistent storage is already structured and meaningful, reducing storage overhead and improving query response times.

Criterion 7 - Streaming Analytics evaluates whether the system can process and analyze data in real-time as it arrives, rather than relying on batch processing after data has been stored. A tool satisfies this criterion if it employs a stream processing engine such as Apache Spark Streaming or Apache Flink to analyze data on-the-fly. Real-time stream processing is essential for detecting anomalies and assessing system health with low latency. A *store-then-analyze* approach introduces delays that are unacceptable in production HPC environments where fast intervention is required.

The final criterion addresses **Usability**.

Criterion 8 - Query Support assesses how the system enables users to retrieve and

interact with monitoring data. It distinguishes between tools that require users to possess deep technical knowledge, such as writing raw SQL against an unfamiliar database schema and those that provide intelligent assistance through natural language interfaces, high-level APIs or automated query generation. In a shared HPC environment serving researchers and lecturers with varying levels of technical expertise, lowering the barrier to data access is not merely a convenience, it is a prerequisite for the system to be useful to its intended users.

Table 2.1 summarizes the assessment of all ten surveyed tools against the eight evaluation criteria. As the results show, no existing tool satisfies all criteria, which reinforces the need for the proposed monitoring framework described in the following chapters.

Tools	App-level (C1)	Target Scope (C2)	Design (C3)	Data Infra. (C4)	Preproc. at Node (C5)	Preproc. at Server (C6)	Stream. Analyt. (C7)	Query Support (C8)
Academic Research Tools								
DCDB	✗	General	Modular	DB+MB	✗	✗	✗	✓
Examon	✗	General	Modular	DB+MB	✓	✗	✓	✓
LDMS	✗	General	Modular	DB	✗	✓	✗	✓
Ganglia	✗	General	Monolithic	DB	✓	✓	✗	✗
MonSTer	✓	General	Monolithic	DB	✗	✓	✗	✓
CARD	✗	General	Modular	DB	✓	✓	✗	✗
Supermon	✗	General	Monolithic	✗	✓	✗	✗	✗
NWPerf	✓	General	Modular	DB	✓	✗	✗	✗
Beacon	✓	General	Monolithic	DB	✓	✓	✗	✓
PIKA	✓	General	Modular	DB	✓	✓	✗	✓
Industrial Tools								
Zabbix	✗	General	Monolithic	DB	✗	✗	✗	✗
Prometheus	✗	General	Modular	DB	✗	✗	✗	✗
Nagios	✗	General	Monolithic	✗	✗	✗	✗	✗

Table 2.1: Comparative Summary of HPC Monitoring Tools based on Evaluation Criteria

2.4 Visualization Chatbot

This section surveys the prior research that motivates the design of the Visualization Chatbot in Chapter 6. The literature falls along three axes: natural-language interfaces to databases (NLIDB), natural-language interfaces to data visualization (NL→Viz), and—since the Visualization Chatbot couples the two on top of an HPC monitoring substrate—

the integration gap between them. We trace each axis from rule-based origins through machine-learning approaches to the current LLM-driven generation, and close the section by stating explicitly which gaps remain open and which our co-design addresses.

2.4.1 Natural-Language Interfaces to Databases

Natural-language interfaces to databases (NLIDB) have been a long-standing research goal in database systems, aiming to democratize data access for users who cannot or do not wish to write SQL. The most recent comprehensive survey by Liu and Xu [14] divides NLIDB systems into three evolutionary stages, mirroring broader trends in natural-language processing.

Rule-based and template-based systems (1970s–2010s). Early NLIDB systems relied on hand-crafted grammars, semantic lexicons, and pattern matching to translate questions into SQL. Representative systems include LUNAR, MASQUE, and PRECISE [15], which used syntactic parsing and domain-specific dictionaries to bridge English and structured queries. These approaches achieved high precision within narrow domains but generalised poorly: every new schema required new rules, and even small linguistic variations could derail the parser. The fundamental limitation was that manual rule engineering does not scale to the diversity of real-world databases.

Machine-learning-based systems (2010–2020). The rise of sequence-to-sequence learning [16] and attention mechanisms [17] enabled data-driven approaches to text-to-SQL. The Spider benchmark [18] established the cross-domain setting that has dominated the field since: train on a set of databases, evaluate on a disjoint set, and reward systems that generalise across schemas. Early architectures specialised the decoder to the SQL grammar: SQLNet [19] replaced left-to-right generation with a sketch-based slot filler, TypeSQL [18] introduced type-aware decoding, and IRNet [20] introduced an intermediate representation as a stepping stone between English and SQL. These systems made measurable progress on Spider but required tens of thousands of labelled examples and continued to struggle with complex joins, nested queries, and out-of-distribution schemas.

LLM-based systems (2020–present). Large language models such as Codex [21], GPT-3, and GPT-4 demonstrated few-shot learning capabilities that displaced much of the specialised text-to-SQL machinery built in the previous era. Recent surveys [22], [23] classify LLM-based text-to-SQL approaches into four broad categories: zero-shot prompting (direct SQL generation from a schema and a question), few-shot prompting (3–5 in-context exemplars), task-specific fine-tuning of smaller open models, and agent-based approaches that incorporate execution feedback. Hong et al. [22] report that GPT-4 reaches an execution accuracy of 85.3 % on Spider in the zero-shot setting, exceeding the best supervised model from the prior era while requiring no domain-specific training.

2.4.2 LLM-Based Text-to-SQL Techniques

Within the LLM era, three threads of work shape the design choices that the Visualization Chatbot inherits. Each thread addresses a different failure mode of vanilla LLM SQL generation, and each finds its analogue in a specific component of the agent.

Prompt engineering strategies. The accuracy of LLM-generated SQL depends critically on prompt construction. AWS best practices for Meta Llama 3 [24] identify three patterns that have become canonical: schema-in-context (`CREATE TABLE` statements interpolated directly into the prompt), few-shot exemplars (3–5 question/SQL pairs from the target domain), and chain-of-thought prompting (the model is asked to reason step-by-step before emitting SQL). The principal limitation of pure prompt engineering is the model's context window: enterprise schemas with hundreds of tables exceed any reasonable prompt budget, forcing prior systems to either truncate the schema or accept degraded accuracy on unrelated tables.

Retrieval-augmented generation. To address the context-window constraint, several systems combine an LLM generator with a vector store of schema fragments and exemplars, retrieving only the elements relevant to the user's question at query time. Vanna.AI [25] embeds DDL statements, written documentation, and curated question/SQL pairs into a ChromaDB vector store and retrieves the nearest neighbours to assemble a tight prompt. DAIL-SQL [26] demonstrates that retrieving similar questions from a training corpus

also improves few-shot prompting. Nascimento and de Oliveira [27] show that hybrid keyword-plus-embedding retrieval outperforms either approach in isolation. The Visualization Chatbot's historical pipeline (Section 5.2 of Chapter 6) builds on Vanna's RAG architecture directly, with the corpus re-trained against the live schema at server startup so that retrieval reflects the current state of the database.

Self-correction and execution feedback. A recurring observation is that the database itself is the cheapest critic of an LLM-generated query: a syntax error, a missing column, or a type mismatch surfaces as an explicit error message that the LLM can read. DIN-SQL [28] systematises this insight by decomposing text-to-SQL into sub-tasks and incorporating execution feedback into a self-correction loop. The cost of execution feedback is latency—each retry is an additional LLM call—which makes the technique attractive for accuracy-bound deployments and unattractive for latency-bound ones. The historical pipeline of Chapter 6 adopts a deliberately constrained variant of this idea: SQL execution failures trigger a single regeneration attempt rather than an unbounded loop, bounding worst-case latency while still recovering the most common errors.

Multi-turn dialogue and clarification. A separate thread of work studies natural-language interfaces that hold a conversation rather than answering a single question. CoSQL [29] and SParC [30] establish multi-turn benchmarks on which a system must follow up on its own previous answers and resolve user clarifications. Multi-turn dialogue is orthogonal to the present work: the Visualization Chatbot is intentionally single-turn, on the grounds that an operator who asks a vague question should receive a useful chart rather than a clarification request. The single-turn design defers the multi-turn case to future work.

Question rewriting, decomposition, and step-back prompting. A complementary strand of research aims to improve the quality of the question itself before it reaches the SQL generator. Zheng et al. [31] introduce step-back prompting, in which the LLM first generates a broader contextual question ("what factors affect salary distribution across job roles?") before tackling the specific query ("what is the average salary of engineers in San

Francisco?"); the broader framing improves multi-hop reasoning. Mouravieff et al. [32] show that decomposing a complex table-QA question into independent sub-questions improves accuracy by 8–14 % on relational reasoning benchmarks. Practical frameworks expose query rewriting and decomposition as composable building blocks. We survey these techniques here because they are candidate extensions for handling vague or compound user questions; the current Visualization Chatbot prototype (Chapter 6) forwards the natural-language question directly to the SQL generator without an NL-to-NL preprocessing step, and integrating rewriting, decomposition, and step-back prompting at a pre-classifier stage is identified as future work.

2.4.3 Natural-Language Interfaces to Data Visualization

A parallel thread of research has addressed natural-language interfaces to *data visualization*, in which the system's output is a chart specification rather than a query result. The same three-era arc—rule, ML, LLM—applies, with a fourth more recent generation that targets end-to-end natural-language-to-chart pipelines.

Rule-based recommendation. The Show Me feature of Tableau [33] pioneered automated chart recommendation, ranking visualization types by an effectiveness score derived from data characteristics (categorical vs. continuous, cardinality, ordering) and from perceptual studies of how humans read charts [34]. Rule-based recommenders are highly precise within their rule set, predictable, and interpretable. Their limitation is the inverse of their strength: they cannot generalise to data patterns or analytical intents that the rule author did not anticipate.

Machine-learning-based recommendation. VizML [35] represents the data-driven turn in chart recommendation: a gradient-boosted classifier trained on roughly one million dataset/visualization pairs harvested from the Plotly platform. VizML predicts chart types from features of the data (column types, distributions, correlations) and reports 78 % accuracy on a held-out test set, compared with 64 % for rule-based baselines. Draco [36] formulates visualization design as a constraint-satisfaction problem and uses answer-set programming to find Pareto-optimal designs that balance multiple effectiveness criteria.

DeepEye [37] scores visualization candidates with a deep neural network trained on user-interaction data.

LLM-based chart generation. Recent work uses LLMs to generate chart specifications or chart code directly. ChartGPT [38] maps natural-language descriptions to Vega-Lite specifications. Data Formulator [39] combines natural-language input with GUI interactions for iterative chart refinement. LLM4Vis [40] uses GPT-4 to generate Python visualization code (matplotlib, seaborn, Plotly) from natural-language requests. These systems demonstrate that an LLM with appropriate prompting can serve as a general-purpose visualization recommender, but they all assume that the data is already in hand and that the user will copy the resulting chart specification or code into their own environment.

End-to-end natural-language-to-chart systems. A more recent generation closes the loop and accepts a question over a database directly. NL4DV [41] maps analytic questions to Vega-Lite encodings, casting the task as an interpretation problem. Luo et al. [42] (ncNet) frame natural-language-to-visualization as neural machine translation and emit a structured visualization-query language. LLM-based systems such as Chat2VIS [43] use few-shot prompting over GPT-3, Codex, and ChatGPT to generate chart code from English questions over relational tables, while Chat2Data [44] composes a vector store and an LLM to support interactive data analysis. The benchmark VisEval [45] consolidates this line of work into a public, cross-domain testbed; the chart-generation core of Chapter 6 is evaluated against four of the systems above on VisEval, and the comparison is reported in Chapter 8.

2.4.4 The Gap and Our Position

The two research communities surveyed above have produced increasingly capable building blocks, but they meet nowhere on the operational HPC-monitoring problem that motivates this thesis. Three gaps stand out, and the Visualization Chatbot in Chapter 6 is positioned to address all three.

No prior NL→Viz system targets operational HPC telemetry. NL4DV, ncNet, Chat2VIS, Chat2Data, ChartGPT, LLM4Vis, and Data Formulator all target generic relational data: Spider-style cross-domain databases, public CSV files, or curated demonstration datasets. None target HPC time-series telemetry with per-process identifiers, hypertable partitioning, and the deeply nested join structure (node × job × user × process) that operational monitoring requires. The closest HPC-adjacent work is chatHPC [46], which combines retrieval and fine-tuning over HPC documentation and support tickets to answer user-facing HPC questions; chatHPC, however, is explicitly a documentation Q&A system and does not consult telemetry. No prior system, to our knowledge, has targeted natural-language access to live HPC telemetry.

No prior NL→Viz system returns a live, continuously refreshing artefact. Every surveyed natural-language-to-visualization system returns a static artefact: a Vega-Lite specification, a matplotlib figure, a chart image, or a chart code fragment. The user can re-issue the question to obtain a fresh view, but the artefact itself does not refresh. For HPC monitoring, the most operationally valuable questions concern present-moment behaviour—"what is the current memory utilisation on node 12," "which jobs are running right now on the GPU partition"—and a snapshot of the present is obsolete the instant it is rendered. None of NL4DV, ncNet, Chat2VIS, Chat2Data, ChartGPT, LLM4Vis, or Data Formulator targets a continuously refreshing panel embedded in a visualization service. The dual-pipeline design of Chapter 6 is the first, to our knowledge, to unify natural-language access to both static analytical charts and live dashboard panels behind a single API.

No prior system co-designs the natural-language frontend with the storage substrate. Existing NL→Viz systems are designed as frontends bolted onto whatever database the user happens to have. The substrate is treated as a black box that returns a result set, and the frontend is responsible for everything else. The present work takes the opposite stance: the Visualization Chatbot is co-designed with the dual-tier substrate (Chapters 4 and 5) so that each pipeline maps onto exactly one storage tier through a classifier-routed API, and the schema of both tiers is introspected on every request rather than embedded

statically in a prompt. The substrate's separation of analytical and realtime tiers is what makes the agent's two pipelines tractable; the agent's two pipelines are what make the substrate's separation visible to the user.

Positioning. Table 2.2 summarises the four most consequential differences between prior natural-language visualization work and the Visualization Chatbot of Chapter 6. The contribution claims listed in the right-hand column are properties of the agent's design, not of its evaluation; empirical evidence is reserved for Chapter 8.

Table 2.2: Comparison of prior natural-language visualization work and the Visualization Chatbot.

Axis	**Prior work**	**Visualization Chatbot (Chapter 6)**
Domain	Generic cross-domain relational schemas (Spider-style)	HPC operational time-series with per-process identifiers and job/user metadata
Pipeline shape	Single NL→query→chart pipeline, static artefact	Dual pipeline (historical static SVG + realtime live Grafana panel) behind a single classifier-routed API
Schema handling	Schema embedded statically in the prompt; cached on a fixed cadence	Per-request introspection of both storage tiers; structurally eliminates schema drift
Output artefact	Static chart, chart specification, or chart code	Inline SVG (historical) and embeddable `/d-solo/` Grafana panel URL (realtime); dual-tier safety via structured-config indirection on the realtime path

The remainder of the thesis develops the agent in three further chapters. Chapter 6 presents the design and architecture of the Visualization Chatbot in detail. Chapter 7

describes the implementation that realises this design: the prompt templates, the construction of the retrieval corpus, the deterministic Flux builder, and the deployment topology. Chapter 8 reports the empirical evaluation, including the VisEval [45] comparison against four reference NL→Viz systems and the per-stage latency and auto-fix recovery rates measured in a full benchmark run. Section 2.4.4's gap statements identify the criteria along which Chapter 8 should be read.

Chapter 3

System Requirements

In this chapter, both functional and non-functional requirements are presented in ***Section 3.1*** *and* ***Section 3.2*** *respectively, such that our proposed Smart Monitoring Tool must satisfy to address the operational needs of the HPC laboratory and overcome the limitations of existing tools identified in Chapter 2.*

3.1 Functional Requirements

Functional requirements define the core behaviors and capabilities that the system must support to enable effective monitoring, data analysis and user interaction. Each requirement below is motivated by a specific operational gap identified in the HPC laboratory environment described in Chapter 1 and motivated by the limitations of the surveyed tools in Chapter 2.

Multi-granularity Resource Monitoring. The system must collect resource usage data, including CPU, memory, disk I/O, network and GPU at multiple levels of granularity. And it must be capable of correlating that data with abstract execution entities such as users, jobs, applications and individual processes.
The need for this requirement arises directly from the resource misallocation problem described in Chapter 1. In a shared HPC environment where multiple users submit jobs concurrently, a node-level view of resource consumption is insufficient for effective management. Consider a scenario where a compute node reports 95% CPU utilization, without process-level and user-level acknowledgement, an administrator cannot determine

whether this is caused by a single misconfigured job consuming all available cores, several users competing for the same resources or a legal high-priority workload running as expected. The distinction matters enormously because the correct response is entirely different in each case. Multi-granularity monitoring bridges this gap by capturing data at the node level for overall health assessment, at the user and job level for resource utilization and accounting, and at the process level for diagnosing specific application behavior. This layered view gives administrators the contextual information necessary to identify which workload is responsible for contention and to make informed decisions about rebalancing or reallocating resources.

Real-time Streaming Analytics. Unlike traditional batch-processing frameworks, the system must support streaming analytics. Incoming data streams are processed and analyzed as they arrive before being written to storage rather than being stored first and analyzed afterward in a batch processing model.
The motivation for this requirement is the time-sensitivity of anomaly detection in production HPC systems. Consider a scenario where a compute node begins suffering a memory leak in a user's application: memory consumption rises gradually, approaching the node's physical limit. In a *store-then-analyze* system, this condition would only become visible after the data has been written to the database and a scheduled query or dashboard refresh has retrieved it, which could take potentially minutes after the problem began. By that time, the out-of-memory condition may have already caused the job to crash, other jobs sharing the node to be affected or the node itself to become unstable. Real-time streaming analytics eliminates this delay by evaluating incoming metrics against threshold conditions and detecting irregular patterns as the data flows through the pipeline, enabling the system to generate an alert within seconds of the first sign of a problem. This capability is also essential for the system's health assessment since administrators require a current and accurate picture of cluster state at all times, not a view that lags behind reality by the latency of the storage and query cycle.

Intelligent Natural Language Interface. To make the system accessible to a broad range of users, the solution must include a conversational interface powered by a Large

Language Model (LLM) that allows users to retrieve system metrics through natural language prompts without requiring knowledge of the underlying database schema, query language or metric naming conventions.

This requirement addresses the gap that is consistent across all tools surveyed in Chapter 2. The users of an HPC laboratory are not systems administrators, they include lecturers, researchers and graduate students who have needs to understand their own resource consumption but lack the technical background to construct database queries or interpret raw monitoring dashboards. Consider a scenario where a researcher suspects that their simulation job is running slower than expected and wants to investigate whether resource constraints are the cause. Without a natural language interface, this requires the user to know which database tables contain the relevant data, what the column names mean, how to join process-level metrics with job-level metadata and how to construct a query that filters by their username and the correct time window. With a natural language interface, the same investigation begins with a question such as ”How much CPU and memory has my job been using in the last hour on Node 03?” and the system handles all of the translation from query to result automatically. This shift from requiring technical expertise to enabling conversational access is what transforms the monitoring system from a tool for administrators into a platform accessible to the entire user community it serves.

Dynamic Dashboard Generation. The system must be capable of automatically generating ad-hoc, context-specific visualizations based on the content of user queries rather then requiring users to navigate pre-defined, static dashboard layouts.

The static dashboards used widely by most tool in the survey, are designed in advance by an administrator who anticipates what information users will need. This works well for routine periodic checks through verifying that all nodes are healthy, reviewing the previous day’s utilization summary. However, it fails for investigative and diagnostic scenarios where the relevant metrics are not known in advance. Consider a scenario where an administrator receives a complaint that a particular user’s jobs have been running unusually slowly over the past week. Diagnosing this requires correlating that user’s CPU time, memory usage, disk I/O patterns and network utilization across multiple nodes and over a specific time window, such a combination of metrics and filters that almost certainly

does not correspond to any pre-built dashboard panel. Dynamic dashboard generation solves this by treating each user query as a specification for a new visualization: the system determines which metrics are relevant to the question, constructs the appropriate queries, selects suitable chart types and assembles a dashboard adapted to that specific request. The result is that every query, whether routine or investigative produces exactly the visual representation the user needs, without navigating dozens of unrelated panels to find the relevant information.

3.2 Non-Functional Requirements

Non-functional requirements specify the quality attributes, performance constraints and architectural characteristics necessary to ensure the system is scalable, efficient, and maintainable in a production HPC environment. Unlike functional requirements, which define what the system does, non-functional requirements define how well it does it and in an HPC context, these constraints are often as critical as the functional ones.

Low Overhead and Edge Preprocessing. The monitoring agent deployed on each compute node must maintain a minimal resource footprint and the system must perform preprocessing at the edge, including local data filtering, sampling and basic aggregation before transmitting metrics upstream.

This requirement exists because compute nodes in an HPC cluster are not idle servers waiting to be monitored, they are actively running the computational workloads that justify the cluster's existence. Every CPU cycle and memory byte consumed by the monitoring agent is a resource taken directly away from a user's job. A monitoring agent that consumes even a few percent of a node's CPU capacity at peak load would introduce measurable slowdowns in tightly coupled parallel applications, where even small timing occupancy degrades overall job performance. Beyond the overhead on individual nodes, the volume of raw metric data generated by continuous process-level monitoring across dozens of nodes presents a network challenge: if every node streams unfiltered, full-resolution metrics to a central server at high frequency, the aggregate bandwidth consumption and central processing load quickly become unsustainable. Edge preprocessing addresses both problems simultaneously, by filtering out irrelevant records, aggregating

short-lived process entries and compressing payloads before transmission, the agent reduces its own processing footprint and the volume of data placed on the network, without sacrificing the metric granularity that the system needs.

Scalability and High Throughput. The system architecture must be capable of handling the high volume and velocity of metric data generated continuously across all compute nodes. It must use a message broker to decouple data collection from data processing, ensuring that metrics are buffered and delivered reliably even under peak load to ensure that the ingestion capacity can scale horizontally as the number of monitored nodes grows. The need for this requirement becomes clear when considering the data rates. A cluster of even modest size, about thirty compute nodes each running dozens of active processes and reporting metrics every thirty seconds would generate thousands of data points per minute continuously. If the data collection pipeline writes directly to the storage backend without any intermediate buffering, a temporary slowdown in the database which is caused by a heavy query, a backup operation or a network issue would cause the collection pipeline to crash, dropping data during the disruption. Furthermore, a tightly coupled pipeline where the collector must wait for the storage backend to acknowledge each write before proceeding limits throughput to the speed of the slowest component. A message broker decouples these concerns: the collection side publishes data at whatever rate the nodes produce it, the broker buffers messages and the storage side consumes at whatever rate it can sustain. Spikes in ingestion rate are absorbed by the broker rather than dropped, and a storage backend restart causes a catchup rather than a gap in the monitoring record. This architecture also enables horizontal scaling. When the cluster grows, additional consumer instances can be added to increase processing throughput without any change to the collection pipeline.

Server-side Data Efficiency. The system must perform preprocessing on the server side before data is written to long-term storage. This includes data normalization, downsampling and advanced aggregation to ensure that only meaningful, structured data is persisted, reducing storage overhead and improving query response times for downstream analytical workloads and the AI Support Agent.

To understand why this is necessary, consider what happens if full-resolution monitoring data is kept indefinitely. A cluster collecting per-process metrics every thirty seconds across thirty nodes generates millions of data points per day. Over weeks and months of operation, the storage backend gathers a dataset that grows linearly with time and that becomes more expensive to query as its size increases. More importantly, the analytical questions that administrators and users ask about historical data rarely require full second-by-second granularity, a question such as ”How has this user’s average memory consumption trended over the past month?” is best answered with hourly or daily aggregates, not with thirty-second snapshots from thirty days ago. Storing full-resolution historical data therefore imposes a storage cost without providing a proportional analytical benefit. Server-side preprocessing addresses this by aggregating raw data into compact, meaningful summaries at regular intervals before writing to the long-term store, ensuring that what is persisted is both useful and efficiently queryable. This is also directly relevant to the AI Support Agent’s performance: an LLM-driven query pipeline that must retrieve and process millions of raw data points to answer a simple historical question would be both slow and expensive, whereas the same question answered from pre-aggregated summaries is fast and cheap.

Modularity. The system must follow a modular architectural approach in which key components, the data collection agent, middleware aggregator, streaming engine and storage backend are designed as independent modules that communicate via well-defined interfaces. Any individual component must be replaceable or upgradeable without requiring changes to the rest of the system.

This requirement is motivated by the practical situations of long-term system maintenance in a research environment. Technology choices that are appropriate today may not remain appropriate as the system evolves, for instances, a time-series database that performs well at current data volumes may need to be replaced as the cluster grows, a streaming framework may be replaced by a better alternative or new hardware on the compute nodes may require a different metric collection approach. In a monolithic system where components are tightly coupled, as observed in tools like Ganglia and Nagios from the survey in Chapter 2, any such change requires modifications to multiple components

simultaneously, increasing the risk of regression and the effort required for testing and deployment. A modular architecture prevents this by enforcing component boundaries through well-defined interfaces: the collection layer only needs to know how to publish to the message broker, the storage layer only needs to know how to consume from it and neither needs to know anything about the other's internal implementation. This also enables independent scaling, if the storage backend becomes a bottleneck, it can be scaled or replaced without touching the collection or processing layers.

Extensibility. The data collection mechanism must be extended through a plugin-based architecture that allows new monitoring targets, such as a new GPU model, a specialized accelerator or a custom application metric to be added by introducing a new plugin without modifying the core agent code.

This requirement reflects the reality that HPC hardware environments are not static. The laboratory's cluster today consists of nodes with a specific set of CPUs and GPUs, but hardware procurements, equipment upgrades and research projects involving specialized accelerators or novel computing architectures will change what needs to be monitored over time. If adding support for a new hardware component requires modifying the core monitoring agent, then recompiling it, testing all existing functionality to verify nothing broke and redeploying it to every compute node, the operational cost of keeping the monitoring system current with the hardware it monitors is high and discourages administrators from extending coverage. A plugin-based architecture reduces this cost to writing a new plugin that implements a defined interface, configuring the agent to load it and deploying only the new plugin file rather than the entire agent binary. Combined with the dynamic reconfiguration capability provided by the Coordinator, this means new sensor types can be enabled across the entire cluster without any service interruption. This property is essential for a monitoring system that is itself expected to operate continuously without downtime.

Chapter 4

System Architecture

The monitoring framework proposed in this thesis is designed around a single central principle: *modularity*. Rather than building a monolithic system where data collection, processing, storage and user interaction are interconnected into one inseparable unit, the architecture decomposes the system into a set of independent, well-defined modules, each responsible for a specific function and communicating with its neighbors through standard interfaces. This design idea is not simply a unintended choice, it is a direct response to the limitations observed in the surveyed tools in Chapter 2, where tightly coupled architectures made it difficult to replace components, adapt to new hardware or scale individual parts of the system independently. In our framework, any module can be upgraded, replaced or scaled without requiring changes to the rest of the system.

Before describing the architecture in detail, it is worth stating the four guiding principles that shaped its design. First, the system must provide *context-aware monitoring*, physical resource metrics such as CPU and GPU utilization must be correlated with logical metadata such as application name, user ID and node ID to produce operationally meaningful information rather than raw numbers. Second, the architecture must be *scalable and adaptable*, built as a modular pipeline, agents, broker and services that supports horizontal scaling to hundreds of nodes without requiring a fundamental redesign. Third, the system must be *efficient and real-time*: edge preprocessing must minimize bandwidth and central load, while broker-based streaming must enable low-latency delivery for near real-time analytics and anomaly detection. Fourth, the system must support *dynamic reconfiguration*, a Coordinator acts as a control plane that propagates runtime updates

to sampling rates, enabled sensors, preprocessing rules and alert thresholds across all agents, enabling changes to take effect without redeploying any component. These four principles are reflected in every architectural decision described below.

At the highest level, the system is organized into three distinct layers: the Compute Node Layer, the Middleware Layer and the Application and Data Services Layer. Each layer has a clearly defined responsibility and the boundaries between them are enforced through explicit communication interfaces rather than direct dependencies. Data flows in one direction, from the compute nodes upward through the middleware and into the application layer; while configuration and control signals flow in the opposite direction, originating from a central coordination point and propagating downward to the agents. This separation of the data path from the control path is an intended design decision that allows operational changes, such as adjusting sampling rates or enabling new sensors to be applied at runtime without interrupting the monitoring pipeline. The architecture is illustrated in Figure 4.1.

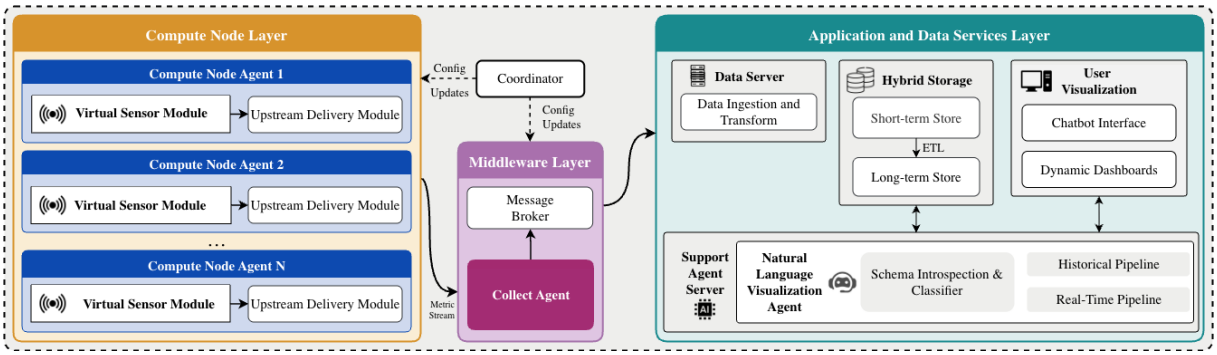


Figure 4.1: Abstract system architecture of the proposed Smart Monitoring Framework, organized into three functional layers. Solid arrows indicate the data flow path from collection through processing to storage and presentation. Dashed arrows indicate the control flow path through which the Coordinator distributes runtime configuration updates to agents and middleware components.

The **Compute Node Layer** is where monitoring begins. Each physical compute node in the HPC cluster runs a dedicated Compute Node Agent, which is the system's primary data acquisition unit. The reason this agent exists at the node level rather than relying on a centralized collector to pull data from each node is a fundamental architectural de-

cision motivated by scale and efficiency. In a cluster of dozens or hundreds of nodes, a centralized pull-based approach would place enormous polling overhead on the central server and introduce collection latency proportional to the number of nodes. By placing an agent on each node, the system distributes the collection workload across the cluster and allows data to be gathered continuously and independently on each machine.
Each Compute Node Agent consists of two sub-modules. The Virtual Sensor Module is responsible for the actual data acquisition, it observes resource utilization at multiple levels of granularity, from node-wide summaries down to individual process-level metrics and packages the collected measurements into a structured, normalized payload. At this layer, basic preprocessing is also performed, including sampling, filtering and tagging metrics with node-level identifiers such as node ID and hostname, which reduces the volume of raw data transmitted upstream and ensures that all subsequent components receivve consistently structured inputs. The Upstream Delivery Module then handles reliable transmission of the packaged payload to the middleware layer. This internal separation within the agent reflects the same modular principle applied at the system level, the concern of *what to collect* is kept entirely separate from the concern of *how to transmit it*, so that either aspect can be modified independently. The agent also supports a plugin-based configuration model, meaning new types of sensors or metrics can be introduced by adding a new plugin rather than modifying the agent's core logic, this is an extensibility property that is essential as the hardware environment evolves over time.

The **Middleware Layer** serves as the system's central coordination and streaming hub. It is composed of three modules: the Collect Agent, the Message Broker and the Coordinator. The Collect Agent is the first point of contact for data arriving from the compute nodes. Its role is not merely to receive and forward data, it is an active processing stage that applies a structured preprocessing pipeline to every incoming metric package. This pipeline performs schema validation to reject abnormal data, filtering to remove irrelevant or redundant readings, enrichment to attach system-level metadata such as the Collect Agent ID and pipeline version and problem detection to identify threshold violations that need immediate alerts. The reason this preprocessing happens at the middleware layer rather than in the application layer, is that it allows all downstream storage and ana-

lytics components to operate on clean, structured andmalready-filtered data, this reduces their processing burden and improving overall system efficiency.

After preprocessing, the Collect Agent publishes data to the Message Broker, which acts as the architectural backbone separating data producers from data consumers. This decoupling is one of the most consequential design decisions in the entire architecture. Without a message broker, the Collect Agent would need to write data directly to the storage backend, creating a tight dependency between the two components and making the system vulnerable to backpressure whenever the storage layer is slow or temporarily unavailable. With the broker in place, the Collect Agent can publish at its natural rate regardless of what downstream consumers are doing and new consumers can be added to the system without any modification to the producer. This is the kind of loose, modular connectivity that enables the system to scale horizontally as the cluster grows and allows the downstream processing pipeline to evolve independently of data collection.

The third component of the middleware layer is the Coordinator, which plays a distinctly different role from the other two. Rather than participating in the data flow, it manages the system's control plane. It is the single authoritative source for runtime configuration across the entire framework, which is responsible for distributing updates such as changes to sampling intervals, enabling or disabling specific sensors, modifying preprocessing rules or adjusting alert thresholds. When an administrator wishes to change the behavior of any part of the system, they issue control actions through the Application Server in the layer above, which forwards those actions to the Coordinator.It then propagates the updated configuration to the relevant modules, including the Compute Node Agents and the Collect Agent without requiring any component to be restarted or redeployed. This capability, known as dynamic reconfiguration, transforms the system from a static monitoring deployment into a living, adaptable framework that can respond to the changing operational conditions of a production HPC environment in real-time.

The **Application and Data Services Layer** represents the system's endpoint, where data is stored, analyzed and made accessible to users. It contains three major components: the Data Server, the Hybrid Storage and the Support Agent Server, alongside the Application Server and user-facing visualization interfaces. The Data Server consumes preprocessed

metrics from the Message Broker and takes responsibility for all persistent storage operations. It computes derived metrics, performs additional aggregation where necessary and routes data to the appropriate storage tier based on its characteristics. This routing logic reflects a key insight about monitoring data: not all data has the same access pattern. Recent data is queried frequently at high granularity with low latency requirements, while historical data is queried less frequently at coarser granularity and is more suited to complex analytical queries. A single storage technology cannot optimally serve both patterns simultaneously, which motivates the Hybrid Storage design consisting of two complementary tiers. The short-term store retains high-resolution recent data optimized for fast time-series queries, while the long-term store holds aggregated historical data optimized for complex analytical workloads. A periodic aggregation process automatically migrates and summarizes data from the short-term store into the long-term store on a regular schedule.

The Application Server provides the APIs and user interfaces through which administrators interact with the system for monitoring, visualization and configuration management. Importantly, user actions that modify system behavior such as updating sampling policies or toggling sensors are validated by the Application Server and forwarded to the Coordinator, which then propagates those changes across the relevant agents. This feedback loop closes the control plane: user intent expressed through the interface is translated into configuration updates that propagate all the way down to the compute nodes, completing the cycle from observation to action.

Finally, the **Support Agent Server** hosts the AI-powered chatbot that allows users to interact with the monitoring data through natural language. Rather than requiring users to understand the database schema or construct queries manually, the chatbot translates user prompts into appropriate data retrieval operations and presents the results as dynamically generated dashboards. This capability directly addresses the usability gap identified in the related works survey, where every reviewed tool required significant technical expertise to extract meaningful insights. The details of the chatbot's internal pipeline are discussed separately in Chapter 6.

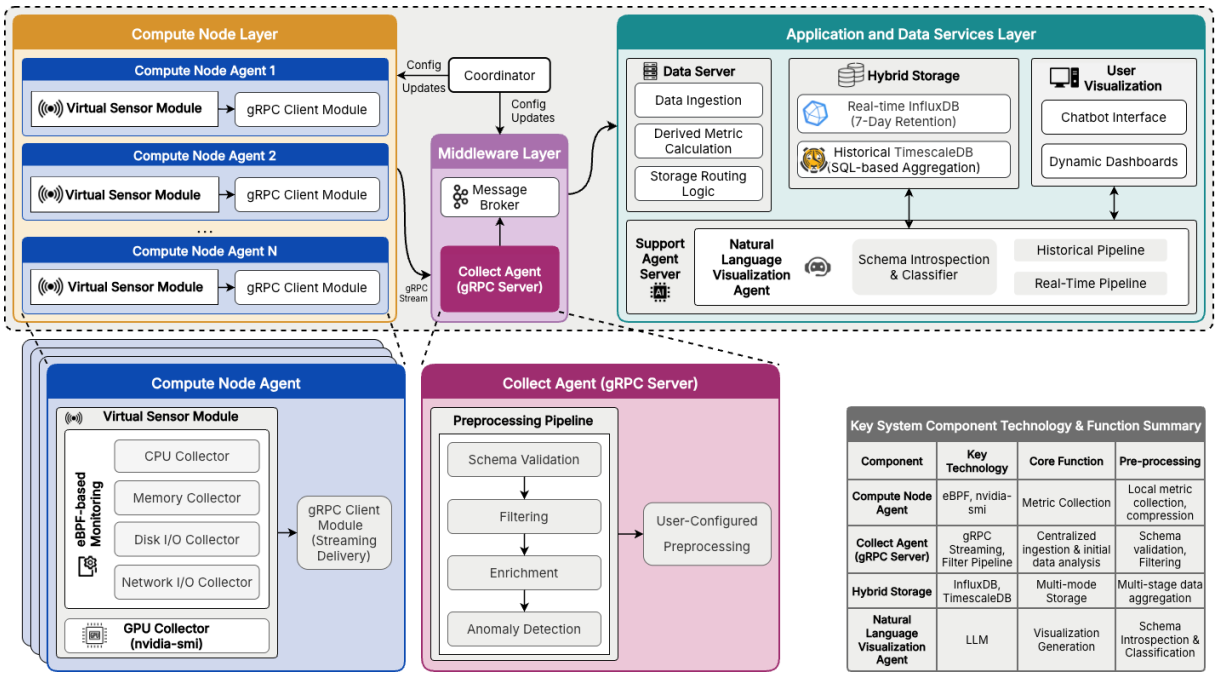
Together, these three layers and their components form a complete, end-to-end monitor-

ing framework that is modular at every level, from the plugin-based sensors on individual nodes to the interchangeable storage backends and the extensible agent preprocessing pipeline in the middleware, making it not only functional today but adaptable to the demands of tomorrow's HPC environments.

Chapter 5

System Design

Building upon the abstract architecture presented in Chapter 4, this chapter describes the specific technology decisions that turn each module into reality. For every component introduced previously, we examine the available implementation approaches, discuss the trade-offs between them and explain why a particular technology was chosen over the alternatives. The goal is to make clear not just what the system is built with, but why it is built that way. The complete technology-specific architecture is illustrated in Figure 5.1.



Key System Component Technology & Function Summary

Component	Key Technology	Core Function	Pre-processing
Compute Node Agent	eBPF, nvidia-smi	Metric Collection	Local metric collection, compression
Collect Agent (gRPC Server)	gRPC Streaming, Filter Pipeline	Centralized ingestion & initial data analysis	Schema validation, Filtering
Hybrid Storage	InfluxDB, TimescaleDB	Multi-mode Storage	Multi-stage data aggregation
Natural Language Visualization Agent	LLM	Visualization Generation	Schema Introspection & Classification

Figure 5.1: Smart Monitoring Tool Detailed Architecture with Specific Technologies Stack.

Compute Node Layer

The Compute Node Layer is where all monitoring activity begins. As described in Chapter 4, each compute node runs a dedicated Compute Node Agent composed of two sub-modules: the Virtual Sensor Module, which handles data acquisition and the Upstream Delivery Module, which handles transmission. The design decisions for each of these sub-modules are driven by the same core constraint: the monitoring agent must be as invisible as possible to the workloads running on the node. Any CPU cycles, memory or network bandwidth consumed by the agent is taken directly away from the user's application, so every technology choice at this layer is evaluated first and foremost by its overhead cost.

Collecting CPU, Memory, Disk I/O and Network Metrics. The Virtual Sensor Module needs to observe resource utilization at the process level — meaning it must know not just how much CPU the node is using overall, but which specific processes are responsible for that usage. There are three commonly used approaches for this on Linux systems. The simplest is to periodically read from the `/proc` and `/sys` virtual filesystems, which expose process-level statistics in plain text files. This approach is easy to implement and requires no special privileges, but it has a fundamental weakness: because it only observes the system at discrete polling moments, any event that starts and finishes between two polls — a short-lived process spike, a burst of network activity — is entirely missed. The second approach is to use hardware performance counters via the `perf_event` interface, which gives precise CPU-level measurements but is architecturally limited to CPU metrics and does not extend naturally to network or disk I/O attribution at the process level. The third approach, and the one we adopt, is **eBPF** (Extended Berkeley Packet Filter).

eBPF is a Linux kernel technology that allows user-defined programs to execute safely inside the kernel in response to specific events — for example, whenever a process is scheduled, a system call is made, a block I/O operation completes, or a network packet is transmitted. Rather than polling the system at fixed intervals from user space, eBPF programs are triggered by the events themselves and update per-process counters stored

in kernel-resident data structures called BPF maps. A user-space collection loop then reads these maps periodically to retrieve accumulated measurements. The kernel's built-in verifier inspects every eBPF program before it runs, checking for memory safety, bounded execution, and type correctness, which guarantees that no monitoring code can crash or destabilize the kernel. Cassagnes et al. demonstrated that eBPF enables non-intrusive performance monitoring by attaching programs to kernel functions including system calls and the network stack, providing detailed telemetry without interfering with the monitored workloads. More recently, Dai et al. showed that an eBPF-based per-PID monitoring system achieves submillisecond reaction time and exposes short-lived process bursts with approximately $14\times$ finer temporal resolution than traditional user-space polling tools such as `top`, while imposing very low overhead.

Table 5.1 summarizes the key differences between the three approaches across the dimensions most relevant to continuous HPC process-level monitoring.

Criterion	`/proc` polling	`perf_event`	eBPF
Collection model	Periodic pull	Periodic sampling	Event-driven
Short-lived process capture	✗ Missed between polls	✗ May be missed	✓ Captured at event
Per-PID resource attribution	✓ via `/proc/PID`	Limited (CPU only)	✓ via BPF maps
Metric coverage	CPU, memory, I/O	CPU counters only	CPU, memory, I/O, network
Overhead model	Proportional to poll rate	Sampling interval dependent	Proportional to event rate
Kernel safety guarantee	N/A	None (module risk)	✓ Verifier + JIT
Dynamic attach at runtime	✗	✗	✓

Table 5.1: Comparison of Linux process-level metric collection approaches. eBPF's event-driven model, per-PID BPF map attribution, and kernel safety guarantee make it the most suitable choice for continuous, low-overhead HPC monitoring **Cassagnes2020**, **Dai2025eHashPipe**.

These properties — event-driven precision, broad metric coverage, per-PID attribution, and the safety guarantee enforced by the verifier — make eBPF the most suitable choice for collecting CPU, memory, disk I/O, and network metrics in an environment where minimizing perturbation to running workloads is a first-order requirement. The detailed implementation of the eBPF collection pipeline, including the BPF map design, the scheduler tracepoint attachment, and the handling of short-lived process collisions, is described in Chapter **??**.

Collecting GPU Metrics. For GPU telemetry, eBPF is not applicable because GPU state is managed entirely by the NVIDIA driver stack and is not exposed through standard

kernel interfaces that eBPF can hook into. The two available options here are NVIDIA's NVML library, a programmatic C API that provides low-level access to driver internals and **nvidia-smi**, the command-line management utility that ships with every NVIDIA driver installation. NVML offers finer-grained control, but it requires combining with NVIDIA's proprietary library, adding a compile-time dependency that reduces portability across different driver versions and system configurations. In contrast, nvidia-smi, is available on every NVIDIA-equipped node by default, exposes the device-level and per-process GPU metrics we need, from utilization, memory usage to process-to-GPU mapping. It also produces structured, easily parseable output. For the monitoring granularity this system requires, nvidia-smi provides everything necessary without introducing additional dependencies. It is therefore selected as the GPU telemetry source, invoked at each collection cycle and parsed to produce a per-PID GPU metric mapping that aligns with the same PID namespace used by the eBPF collectors, allowing metrics from both sources to be joined cleanly in the next step.

Packaging the Collected Data. Once the eBPF collectors and nvidia-smi have each produced their per-process measurements for a given collection cycle, the Virtual Sensor Module combines them into a single, unified metric package. This is done by joining all collector outputs on PID, so that each active process is represented by one entry containing its CPU, memory, disk I/O, network and GPU measurements together. Rate-based metrics such as bytes per second are derived from raw counters using the sampling window duration and the completed package is stamped with global metadata, including node ID, IP address, hostname and collection timestamp. The output of this step is the standard payload unit that the Upstream Delivery Module will transmit upstream. Keeping packaging as a distinct step inside the agent rather than sending raw per-source outputs separately, this means the Collect Agent always receives a single, self-contained, consistently structured message per node per cycle, which simplifies every subsequent processing stage.

Transmitting Data to the Middleware. With a normalized metric package ready, the Upstream Delivery Module must transmit it reliably and efficiently to the Collect Agent. The

most straightforward option would be a REST API over HTTP, it is widely supported and easy to debug, but it carries significant per-request overhead from HTTP headers, JSON serialization and connection setup. Given that the agent transmits a new package every few seconds continuously for the entire lifetime of the cluster, that overhead accumulates. A more suitable alternative is **gRPC**, a modern Remote Procedure Call framework built on HTTP/2 and using Protocol Buffers (Protobuf) for binary serialization. Compared to REST, gRPC produces more compact messages, encodes and decodes them faster and supports a *client-streaming* communication pattern in which the client opens a single long-lived connection and pushes a continuous stream of messages to the server without needing to re-establish the connection for each one. This client-streaming model maps naturally onto the agent's continuous telemetry delivery behaviour, eliminating the repeated handshake cost of per-cycle connections. The service contract between the agent and the Collect Agent is defined in a `.proto` file, which enforces a shared, strongly typed message schema and ensures that both sides remain compatible as the system evolves. For these reasons, efficient binary encoding, persistent connection, and clear contract definition, gRPC with client-streaming is selected as the transmission mechanism for the Upstream Delivery Module.

Middleware Layer

Once metric packages leave the compute nodes, they enter the Middleware Layer, the part of the system responsible for receiving, validating, preprocessing and routing data before it reaches the storage and analytics components above. As introduced in Chapter 4, this layer consists of three modules: the Collect Agent, the Message Broker and the Coordinator. Each plays a different role and the technology choices for each are driven by correspondingly different requirements. Together, they form the backbone that keeps the data flowing reliably and the system behaving consistently, regardless of what is happening at the nodes below or the application layer above.

Receiving and Preprocessing Data. The Collect Agent is the first component in the system that sees data arriving from the compute nodes. Its job is to act as a gRPC server, accepting the client-streaming connections opened by each Compute Node Agent and to

process every incoming metric package through a structured preprocessing pipeline before publishing it downstream. The preprocessing pipeline consists of a fixed sequence of stages that always execute in order: schema validation, which rejects any package whose structure does not follow to the agreed Protobuf contract; filtering and cleaning, which removes bootstrap metrics from nodes that have just started up and discards any invalid records; enrichment, which attaches system-level metadata such as the Collect Agent ID and pipeline version to support traceability; and problem detection, which checks each package against predefined resource thresholds and triggers an immediate alert if a violation is found. After these fixed stages, a user-configurable plugin chain is applied, allowing administrators to attach custom preprocessing logic such as domain-specific filtering rules or derived metric calculations without modifying the Collect Agent's core code.

We intended to place this preprocessing pipeline in the middleware rather than pushing it downstream to the Data Server as every stage in the pipeline reduces the volume and improves the quality of data before it moves further through the system. If validation and filtering were gave to the Data Server, abnormal or irrelevant records would travel all the way through the message broker and consume ingestion bandwidth unnecessarily. By cleaning the data as close to the source as possible, the Collect Agent ensures that everything published to the Message Broker is already structured, enriched and meaningful, this reduces the burden on every component that comes after it.

An important ability of the Collect Agent is its support for *dynamic reconfiguration* of the plugin chain. In practice, preprocessing rules always change, new sensors are added, filtering thresholds are adjusted, aggregation windows change. If all of this logic were hardcoded into the Collect Agent, every change would require a service restart, which is unacceptable in a production monitoring system that must operate continuously. The plugin architecture solves this by separating the preprocessing logic from the agent's core execution loop. Each plugin implements a shared interface and is executed as part of a configurable chain. When the Coordinator distributes an updated configuration, enabling a new plugin, disabling an existing one or reordering the chain, the Collect Agent applies the change to its in-memory pipeline without any restart. This hot-reload capability is what makes the middleware layer adaptive rather than merely functional.

Beyond the standard data path, the Collect Agent also maintains a separate *direct alert channel* to the Application Server for time-sensitive events. When the problem detection stage identifies a threshold violation, for example, a node's CPU utilization exceeding a critical bound. Instead of waiting for the alert to travel through the message broker, be consumed by the Data Server, stored in the database and then queried by the Application Server would introduce unacceptable latency. The Collect Agent sends the alert directly to the Application Server via a unary gRPC call, entirely skipping the standard data path. This dual-path design, one path for continuous telemetry, another for urgent alerts ensures that the system can provide both high-throughput data delivery and low-latency incident notification without bothering the other.

Decoupling Producers from Consumers. After the Collect Agent finishes preprocessing a metric package, it must forward the data to the downstream consumers in the Application and Data Services Layer. The most direct approach would be to write the data straight to the database, but this creates a tight coupling between the middleware and the storage layer: if the database is slow to respond or temporarily unavailable, the Collect Agent would delay and data would be lost. A better approach is to introduce a message broker between producers and consumers, allowing them to operate at their own pace independently of each other. The broker buffers incoming messages, ensures reliable delivery and allows multiple consumers to subscribe to the same data stream without any producer needing to be aware of them.

There are several widely used message broker technologies available. RabbitMQ is a mature, general-purpose broker that implements the AMQP protocol and excels at task queue workloads where individual message routing and acknowledgement semantics matter. Redis Streams is a lightweight option built into the Redis in-memory store, suitable for low-latency pub/sub scenarios but limited in durability and replay capability. **Apache Kafka** is a distributed event streaming platform designed specifically for high-throughput, fault-tolerant, persistent message delivery at scale. For a monitoring system that continuously ingests metric packages from hundreds of compute nodes, Kafka's design aligns with the requirements: it partitions topics across multiple brokers to distribute ingestion load, replicates messages for fault tolerance, retains messages on disk so that a

downstream consumer can replay or catch up after a temporary failure and supports consumer groups that allow multiple independent services to consume the same stream in parallel. Kafka also preserves per-partition ordering, which is important for time-series consistency through partitioning on node ID, the system ensures that metric packages from each individual node always arrive at the consumer in the order they were produced.

For these reasons, Apache Kafka is selected as the Message Broker. The system organizes the Kafka pipeline into two topics. The first, `rawData`, receives preprocessed metric packages published by the Collect Agent and is consumed by the Data Server. This topic serves as a durable buffer between the middleware and the storage layer, ensuring that ingestion spikes or temporary slowdowns in the Data Server do not cause data loss. The second topic, `processedData`, is produced by the Data Server after it has performed further aggregation and derivation and can be consumed by any other component in the system that requires instant data. This fan-out capability is a direct benefit of the broker-based design: adding a new consumer, for example, a real-time anomaly detection service requires no changes to the producer or to any existing consumer.

Managing Runtime Configuration. The third component of the middleware layer serves a purpose that is different from the other two. The Coordinator does not touch the data path at all, its role is to act as the system's control plane, maintaining a consistent, authoritative view of the runtime configuration across all distributed components. Whenever an administrator changes a system parameter by adjusting a sampling rate, enabling a new sensor plugin, modifying a preprocessing rule or updating an alert threshold, that change must be propagated reliably to every affected component, which may include dozens of Compute Node Agents distributed across the cluster. Ensure reliability in a distributed system requires a coordination service that provides strong consistency guarantees, that means every component always reads the most recently committed configuration rather than an old version.

The technology choices for distributed coordination fall into two broad categories. Centralized configuration stores such as Consul or ZooKeeper provide key-value storage with distributed consensus, but ZooKeeper's operational complexity and Java-based runtime

make it heavyweight for this purpose. A lighter and more modern alternative is **etcd**, a distributed key-value store built on the Raft consensus algorithm that is designed specifically for storing configuration data and coordinating state across distributed systems. etcd exposes a gRPC-based API, supports continuous reads that guarantee the most recently committed value is always returned and provides a *watch* mechanism through which clients can subscribe to specific keys or key prefixes and receive real-time notifications whenever those values change. This mechanism is particularly valuable for the Coordinator's use case: rather than requiring each Compute Node Agent to periodically poll for configuration updates, agents can register a watch on their relevant configuration keys and be notified immediately when a change is committed. etcd also supports leases with time-to-live semantics, which can be used to detect when an agent has gone offline and compare-and-swap operations that allow safe concurrent updates to shared configuration state. For those reasons, such as strong consistency, efficient change notification, operational simplicity and native gRPC compatibility with the rest of the system, etcd is selected as the foundation for the Coordinator.

In practice, the Coordinator exposes a structured configuration namespace in etcd organized by key prefixes; for example, separating keys for sampling policies, enabled sensor plugins, preprocessing rules and alert thresholds into distinct namespaces. The Application Server acts as the entry point for administrative control actions: when an administrator updates a configuration parameter through the user interface, the Application Server validates the request and writes the new value to the appropriate etcd key. The affected agents and the Collect Agent, which are subscribed to their respective configuration prefixes via etcd watches, receive the change event and apply the updated configuration to their in-memory state immediately without requiring a restart. This design centralizes configuration authority in one place while distributing configuration delivery efficiently across the entire cluster.

Application and Data Services Layer

Data arriving through the Middleware Layer has been validated, filtered, enriched and buffered in Kafka but it has not yet been stored anywhere persistent or made accessible to users. The Application and Data Services Layer is where that changes. It is responsible

for consuming the data stream, storing it in a form that supports both real-time queries and long-term historical analysis and ultimately presenting it to users through an intelligent interface. This layer contains three major components: the Data Server, the Hybrid Storage and the Support Agent Server alongside the Application Server and user-facing visualization interfaces. Each of these components addresses a distinct concern and together they transform a continuous stream of raw data into actionable insight.

Consuming Data Stream and Data Structuring. The first component to receive data in this layer is the Data Server, whose responsibility is to consume metric packages from the Kafka `rawData` topic and write them into persistent storage in a structured, queryable form. The simplest approach would be a script that reads each Kafka message and writes it directly to a database as a single record. However, this approach has a fundamental problem: the metric package produced by the Compute Node Agent is a unified payload that packs node-level, GPU-level and process-level information into one flat structure. Storing it as a single record would make queries inefficient, since every query, whether asking about a node's overall CPU load or a specific user's memory consumption would need to scan through the same monolithic records and extract only the fields it needs. A better approach is to decompose each incoming package into separate measurements at ingestion time, so that each measurement contains exactly the fields relevant to one analytical concern and can be indexed and queried independently.

There are two possible choices for implementing this consumer. The first is to use a dedicated stream processing framework such as Apache Spark Structured Streaming or Apache Flink, which provide built-in Kafka integration, fault-tolerant exactly-once processing semantics and powerful transformation APIs. These are appropriate when the transformation logic is complex, involves operations such as windowed joins across multiple streams or needs to scale across a cluster of processing nodes. The second approach is a lightweight Python consumer using the `kafka-python` library, which is simpler to deploy and maintain when the transformation logic is straightforward and the ingestion rate does not require distributed processing. In our case, the transformation performed at ingestion, which decomposes a single message into three measurements and applying grouping and normalization to process-level data. The overhead of deploying and

managing a full Spark or Flink cluster is not justified by the complexity of the task. We therefore implement the Data Server as a **Python consumer script** that subscribes to the Kafka topic using a consumer group and processes each message as it arrives, decomposing it into the three target measurements before writing them to the storage backend. For each incoming message, the script produces three distinct measurements. The first, `node_status`, which captures the overall state of each compute node, including node-level CPU and memory usage percentages, total and used memory in bytes and collection metadata such as the collection window duration and Collect Agent ID. The second, `gpu_status`, records per-GPU data for each GPU on the node, which are utilization, temperature, power draw, power limit and memory consumption, they are tagged by both node ID and GPU index to handle multi-GPU nodes correctly. The third, `process_status`, captures resource consumption at the process level, it consists of CPU on-time, disk and network I/O bytes, GPU memory usage and resident set size, tagged by node ID and user ID.

One additional design decision concerns process-level data volume. A busy compute node may have hundreds of active processes at any given moment and writing a separate record for every individual PID at every collection cycle would generate a large number of data points. To address this, the Data Server groups processes by the combination of user ID and command name, summing their resource metrics within each group before writing. User IDs are normalized to distinguish root, system processes and regular users, and command names are also normalized by collapsing known kernel and system thread prefixes into canonical categories. This grouping reduces the cardinality of the `process_status` measurement while preserving the user-level and application-level attribution that the system needs. All three measurements are written to the storage backend in a single batched write per incoming Kafka message to minimize request overhead.

Choosing Right Storage for Each Access Pattern - The Hybrid Storage. Once the Data Server has structured the incoming data, it must decide where to write it. The choice of storage technology here is not easy, because monitoring data is accessed in two fundamentally different ways depending on how recent it is. For recent data, the last few hours or days is queried frequently at full granularity and with low latency requirements.

An administrator investigating an ongoing performance issue wants second-by-second CPU usage for the last thirty minutes, returned in milliseconds. For historical data, on the other hand, is queried less frequently, at less granularity hourly or daily averages, is typically used for trend analysis, capacity planning or investigation where response times of a few seconds are acceptable.

The question is whether a single storage technology can serve both patterns well. A general-purpose relational database such as PostgreSQL can store time-series data, but its row-oriented storage engine is not optimized for the high write throughput and time-range scan patterns that continuous monitoring generates. A pure time-series database such as InfluxDB is purpose-built for high-frequency writes and fast time-range queries, but is not designed as a long-term analytical store as keeping months of high-resolution data in it would consume storage at an unsustainable rate and degrade query performance as the dataset grows. Furthermore, InfluxDB's proprietary Flux query language is not directly compatible with SQL-based analytical tools or AI-driven text-to-SQL pipelines, which creates a usability barrier for the Support Agent Server that will query the data later.

Rather than forcing a single technology to serve both access patterns poorly, the system adopts a two-tier **Hybrid Storage** architecture that assigns each tier to the technology best suited to it. **InfluxDB** serves as the short-term, high-resolution store with a retention period of seven days, holding full-granularity data for immediate operational queries, this is where the Data Server writes its three measurements in real time. **TimescaleDB**, a PostgreSQL extension built specifically for time-series workloads, serves as the long-term analytical store, holding hourly aggregated summaries. TimescaleDB stores data in a standard SQL schema, supports complex joins and aggregations across multiple tables, partitions data automatically by time for efficient range scans, and exposes a standard SQL interface that is directly usable by the Support Agent's text-to-SQL pipeline. The combination of these two databases ensures that both real-time operational queries and long-term analytical queries are served by the storage engine best suited to each of them.

Bridging Two Tiers - The Hourly ETL Pipeline. The transition of data from InfluxDB to TimescaleDB is handled by a dedicated hourly ETL (Extract, Transform, Load) pipeline,

which is a scheduled Python service that runs once per hour, queries InfluxDB for the data accumulated in the previous hour, computes aggregated summaries and writes the results to TimescaleDB. The alternative to this approach would be to write aggregated data to TimescaleDB directly at ingestion time, in parallel with the full-resolution write to InfluxDB. However, this would couple the Data Server's ingestion path to two databases simultaneously, increasing write latency and introducing a partial-write failure scenario where one database receives data and the other does not. A scheduled ETL pipeline that reads from InfluxDB after the data has already been safely stored is more efficient: if the pipeline fails for any reason, it can be rerun against the same InfluxDB data without any data loss and the ingestion path remains unaffected.

The pipeline produces two output tables in TimescaleDB. The first, `node_status_hourly`, stores one record per node per hour, containing the average and maximum CPU and memory usage, average GPU utilization and temperature, total GPU power consumption and total disk and network I/O bytes accumulated during that hour. The second table, `user_app_hourly`, stores one record per unique combination of node, user ID and application name per hour, containing total CPU time consumed in seconds, average and maximum memory usage, total I/O and network bytes and the peak process count observed during that hour. This second table is particularly significant because it is the primary data source for answering user-level resource questions; for example, how much CPU time a specific user's application consumed across the cluster over the past week, which is one of the core analytical use cases the system was designed to support.

The pipeline leverages InfluxDB's Flux query language to perform time-windowed aggregations directly within the database before results are transferred to Python, which minimizes the volume of data that must be processed in-memory. Average and maximum values for node-level metrics are computed with one-hour windows.

Providing the User Interface. With data flowing into the storage tier, the system needs a user interface layer through which administrators and users can interact with the monitoring platform. The Application Server is responsible for providing this interface, it is the component that connects all other parts of the system together from the user's perspective, exposing monitoring dashboards, historical analytics, cluster configuration, alert notifications and the chatbot interface in a single application.

For the frontend, the two main choices in modern web development are single-page application frameworks such as React and Vue and full-stack frameworks that combine server-side rendering with client-side interactivity. Single-page frameworks offer maximum flexibility but require a separate backend service to handle API routing, authentication and server-side logic. Full-stack frameworks such as **Next.js** combine the frontend and backend into a single unit, supporting both server-side rendered pages for fast initial load and client-side navigation for a responsive user experience, while providing built-in API route handling that eliminates the need for a separate backend server. Given that the web application needs to integrate tightly with multiple backend services, InfluxDB via Grafana embeds, TimescaleDB for historical queries, the Coordinator for configuration management and the Support Agent for chatbot functionality. Therefore, the unified deployment model and built-in API routing of Next.js make it the most practical choice. It is then selected as the technology stack for the Application Server.

The web application provides five main functional areas. The *real-time monitoring dashboard* displays live cluster metrics by embedding Grafana panels that connect directly to InfluxDB as their data source, giving administrators an real-time view of node utilization, GPU status and active processes without requiring the application server to poll the database itself. The *historical analytics* interface queries TimescaleDB for aggregated usage data, presenting trend charts and user-level resource summaries that help administrators understand long-term utilization patterns. The *AI Support interface* provides a page for interacting with the LLM-powered chatbot, through which users submit natural language queries and receive dynamically generated visualizations in return in either an inline static chart for historical questions or a live embedded Grafana panel for real-time questions. The *cluster configuration page* allows administrators to modify system parameters such as sampling rates and enabled sensor plugins; changes entered on this page are sent via API call to the Application Server's backend, which connects to the Coordinator and applies the update directly, propagating it to the relevant agents without any service restart. Finally, *alert notifications* shows events detected by the Collect Agent's problem detection stage, informing the administrator in real time when a threshold violation has been identified. In the current implementation, access to the application is restricted to administrator accounts.

Enabling Natural Language Interaction. The final component of this layer is the Support Agent Server, which hosts the AI-powered chatbot that allows users to query the monitoring data through natural language. This is the most innovative component in the entire system, none of the ten tools surveyed in Chapter 2 offered anything comparable and its design involves several technology decisions that are worth explaining at this level before the full pipeline architecture is described in Chapter 6.

The Support Agent is deployed as a **separate microservice**, independent of the Application Server. This separation is a on purpose architectural choice: the chatbot's workload is different from that of the web application. It involves multiple sequential LLM API calls, database queries against two different backends, code generation and execution, and external API calls to Grafana. This processing pattern is latency-sensitive and resource-intensive; therefore, keeping it as a separate microservice means it can be scaled, restarted or updated independently of the web application, and any failure in the chatbot pipeline does not affect the rest of the monitoring interface.

The backbone of the Support Agent is **GPT-4o-mini**, selected as the LLM for all generation tasks within the pipeline. The choice between a locally hosted open-source model and a cloud-hosted model involves a trade-off between cost, latency and capability. Locally hosted models such as LLaMA or Mistral offer full data privacy and no per-call cost, but require significant GPU resources to serve at acceptable latency and typically not as good as GPT-class models on complex multi-step tasks such as SQL generation. GPT-4o-mini offers strong performance on these tasks at a lower cost than GPT-4o, making it a practical balance between capability and operational cost for a research laboratory deployment.

A key architectural feature of the Support Agent is that it queries **both storage tiers** depending on the nature of the user's question. Specifically, InfluxDB using Flux for real-time questions and TimescaleDB using SQL for historical questions. Rather than requiring users to specify which database their question targets, the agent uses an LLM-based classifier to automatically route each incoming question to the appropriate pipeline. This dual-pipeline design, one anchored in InfluxDB and Grafana for live visualization, the other in TimescaleDB and matplotlib for static analytical charts is the central archi-

tectural decision of the Support Agent and is described in full detail in Chapter 6.

With all the components and technology selection described, the rationale of all major components in the monitoring framework is complete. Table 5.2 summarizes the technology stack used across the entire system for quick reference.

Component	**Technology**	**Role**
Compute Node Agent	eBPF (BCC) + nvidia-smi	Kernel-level and GPU metric collection
Upstream Delivery	gRPC (client-streaming)	Continuous metric transmission to Collect Agent
Collect Agent	Python asyncio + gRPC server	Stream ingestion, preprocessing pipeline, Kafka publishing
Coordinator	etcd	Distributed runtime configuration management
Message Broker	Apache Kafka	Decoupled, durable metric streaming
Data Server	Python + kafka-python	Kafka consumption, InfluxDB decomposed writes
Short-term Storage	InfluxDB	High-resolution 7-day time-series store
ETL Pipeline	Python + schedule	Hourly aggregation from InfluxDB to TimescaleDB
Long-term Storage	TimescaleDB	SQL-queryable aggregated historical store
Application Server	Next.js	Web interface, configuration management, alert display
Support Agent Server	Python + GPT-4o-mini + ChromaDB + Grafana	LLM-powered natural language query and dashboard generation

Table 5.2: Summary of technology stack across all system components.

Chapter 6

Visualization Chatbot

6.1 Introduction and Motivation

The substrate described in the preceding chapters collects per-process telemetry continuously from every compute node and stores it in two complementary backends: an InfluxDB tier optimised for high-frequency time-series ingestion and a TimescaleDB tier optimised for long-retention analytical queries. Collecting this data, however, is only half of the monitoring problem. The other half is letting the people who depend on it—cluster operators, system administrators, and end users—ask questions of it without first having to learn a query language. This chapter presents the Visualization Chatbot, an LLM-driven agent that turns one English sentence into either a static analytical chart or a live dashboard panel, and that closes the loop between the monitoring substrate and the human who needs to understand what it is observing.

The interface gap in HPC monitoring. Existing monitoring stacks expose two interaction surfaces. The first is a query language: SQL for analytical stores, Flux or PromQL for time-series stores. Operating these surfaces requires the user to translate a half-formed analytical intuition ("why did node 7 throttle this morning?") into a precise query that joins the right tables, filters on the right tags, and aggregates over the right window. The translation is laborious for engineers and impossible for the non-experts—application owners, junior administrators, occasional users—who nevertheless have legitimate questions to ask. The second surface is a static dashboard: a panel arrangement configured in advance by an administrator, showing whatever metrics that administrator anticipated

would be useful. Static dashboards answer the questions that were predicted; they cannot answer the question that was not.

The result of this two-surface design is that gigabytes of telemetry flow into the storage tier every day, while the operational decisions that depend on it—scaling, debugging, capacity planning, anomaly investigation—remain bottlenecked on a small number of engineers fluent in the query languages. The data is collected, but it is not consulted. A natural-language interface promises to invert this asymmetry: every member of the organisation able to articulate a question in English becomes able to consult the monitoring substrate directly.

The artefact question. A natural-language interface that returns rows of numbers does not solve the problem; it only relocates it. An operator who asks "which jobs blew their I/O budget last week?" does not want a list of `job_id, sum_io_bytes` pairs—they want a chart that shows the distribution at a glance, identifies the outliers, and is ready to embed in an incident report. Likewise, an operator who asks "show me current GPU utilisation on rack 3" does not want a single timestamped scalar—they want a panel that updates as the workload progresses, comparable across machines, and reachable from a shared URL. The right input is English; the right output is a chart, and the right *kind* of chart depends on whether the question is about the past or about the present. Designing the agent therefore means designing for both kinds of question and for both kinds of artefact.

Two questions, two pipelines. Operator questions in HPC monitoring split cleanly along a temporal axis. *Historical* questions ask about past behaviour over a defined window: "which user consumed the most GPU hours last quarter," "which node has the highest disk-error rate over the last seven days," "did the new scheduler reduce average job wait time compared to the previous version." These questions reward an analytical store that supports joins, group-bys, and time bucketing, and the most useful artefact is a static chart that summarises the answer and can be archived. *Realtime* questions ask about the present: "what is the current memory utilisation on node 12," "which jobs are running

right now on the GPU partition," "is the network saturated this minute." These questions reward a time-series store optimised for the freshest samples, and the most useful artefact is a live panel that continues to refresh as long as the operator looks at it. The agent therefore comprises two architecturally distinct pipelines—one anchored in TimescaleDB and matplotlib, the other in InfluxDB and Grafana—unified only at the outermost API surface, where a single classifier decides which path a given question travels.

Chapter roadmap. Section 6.2 presents the central design decision of the agent: two pipelines, one API. Sections 6.3 and 6.4 describe the two shared mechanisms that operate before either pipeline is selected—live schema introspection of both backends, and an LLM classifier that routes the question with a safe default. Sections 6.5 and 6.6 describe the two pipelines themselves: the historical path that produces a static SVG chart, and the realtime path that produces an embeddable Grafana panel. Section 6.7 discusses the agent's caching and freshness strategy, which is shaped by the dual-pipeline design. Section 6.8 treats security as a first-class design property rather than an afterthought, accounting for the three threats that LLM-mediated query and code generation introduce. Section 6.9 closes the chapter and bridges to the implementation material in Chapter 7 and the empirical evaluation in Chapter 8.

6.2 Two Pipelines, One API

The agent exposes a single boundary to its callers: a question goes in, and either a static chart artefact or a live dashboard panel comes back. The caller does not specify which pipeline to use. Internally, however, the agent is two distinct systems behind a shared front door, and the choice of which to engage is made by a classifier rather than by the caller. This section explains why a single pipeline is not enough, why a single API surface is nevertheless desirable, and how the dual structure is unified.

6.2.1 One Sentence In, One Artefact Out

The single-input/single-output principle is the foundation of the agent's user-facing contract. Regardless of how rich the underlying machinery becomes, the operator's experi-

ence must remain elementary: type a question, receive an artefact. Every additional decision pushed onto the caller—"do you want historical or realtime data?", "which backend should I query?", "what visualisation type?"—erodes the value of the natural-language interface, because each question relocates the cognitive load that natural language was meant to remove.

The principle has three operational consequences. First, the agent must be capable of handling *both* historical and realtime questions transparently; refusing to answer one class would force the operator to pre-classify, defeating the unified interface. Second, the agent must *itself* make every routing and configuration decision that an underlying tool would normally expect from a user, which is why the classifier (Section 6.4) is not optional. Third, the response shape must be uniform enough that a downstream client (a chat UI, a notebook, a dashboard widget) can render the result without inspecting which pipeline produced it; the agent therefore returns a single JSON envelope whose payload is either an inline SVG string or a Grafana embed URL, distinguished by a single discriminator field.

6.2.2 Why Two Pipelines Instead of One

Operator questions in HPC monitoring decompose along three orthogonal axes, and the product of these three axes is what makes the dual pipeline irreducible.

Data tier. Historical analysis demands joins between metric tables, job records, and user metadata; group-by aggregation across hours, days, or weeks; and the kind of relational reasoning that columnar or relational time-series engines such as TimescaleDB are built for. Realtime monitoring demands the freshest sample of a small number of measurement series, indexed by tag, returned in milliseconds—the access pattern that a tag-indexed time-series store such as InfluxDB is built for. Forcing one engine to serve both patterns penalises one of them: a relational engine cannot match a tag-indexed engine on freshest-sample latency, and a tag-indexed engine cannot match a relational engine on multi-table aggregation. The substrate maintains both tiers (Chapter 4) precisely so that each access pattern is paid for in the engine that handles it cheapest, and the agent inherits the same

separation.

Freshness semantics. A historical answer is, by construction, a snapshot. The query window is closed: "the last seven days" is a fixed interval at the moment the query runs, and re-running it minutes later would return essentially the same result. A realtime answer, by contrast, is a stream. "Current GPU utilisation" is meaningful only as a continuously refreshing observation; a single snapshot of it would be obsolete by the time the operator finished reading it. These two freshness regimes cannot share a single response shape: the snapshot wants to be a finished, archivable artefact, and the stream wants to be a live, regenerating view.

Output artefact. A static chart and a live dashboard panel are not two formats of the same thing. A static chart is a self-contained visual that can be embedded in a report, attached to a ticket, compared against a chart from last month, or printed. It is finished. A live dashboard panel is a window onto a service: it has no value detached from the visualisation backend that updates it, and its utility is a function of how long the operator keeps watching it. Confusing the two—returning a stale snapshot when the operator wanted a live view, or returning a live URL when the operator wanted to archive the chart—is more harmful than slightly degraded chart quality, because the operator is forced to discard the artefact and re-ask the question.

A single pipeline could be forced to accommodate the cross-product of these three axes only by compromise. It would either freeze the realtime case into a snapshot (sacrificing freshness), pretend the historical case is a stream (sacrificing the finished-artefact property), or unify the two access patterns behind a lowest-common-denominator query interface (sacrificing analytical depth). The agent therefore keeps the pipelines architecturally distinct and accepts the implementation cost of two parallel mechanisms, in exchange for letting each case be handled in its native idiom.

6.2.3 Why a Unified API Nonetheless

If the two pipelines are irreducibly different, why not expose them as two endpoints? The answer is the same as the answer to "why a natural-language interface at all": asking the caller to choose forces them to know the distinction between historical and realtime, which in turn forces them to know which backend their question reaches. The natural-language premise is that the operator should be free not to know. The classifier therefore takes responsibility for the distinction, the API surface remains a single `POST`, and the dual pipeline structure is an implementation detail that does not appear in the contract between agent and caller.

The unified surface also has a secondary benefit: it permits the agent to fall back. If the realtime path is unreachable—because the time-series tier is down, the dashboard service is unavailable, or the classifier itself errors—the agent can transparently route to the historical path and still produce *some* answer. A two-endpoint design would either fail the request or require the caller to retry against the alternate endpoint, neither of which preserves the natural-language abstraction.

6.2.4 Pipeline Overview

Figure 6.1 shows the end-to-end flow. A natural-language request first passes through schema introspection on both backends; the introspected schemas, together with the question, are fed to the LLM classifier. The classifier emits a routing decision—historical or realtime—which directs the request to one of the two pipelines. The historical pipeline performs retrieval-augmented SQL generation, executes the query against TimescaleDB, profiles the resulting DataFrame, and synthesises matplotlib code that is executed server-side to produce an SVG artefact. The realtime pipeline instead generates a structured panel configuration, compiles it into a Flux query through a deterministic builder, and pushes a panel definition to a remote Grafana instance. Both pipelines converge on a unified JSON response carrying either an inline SVG artefact (historical) or an embeddable panel URL (realtime).

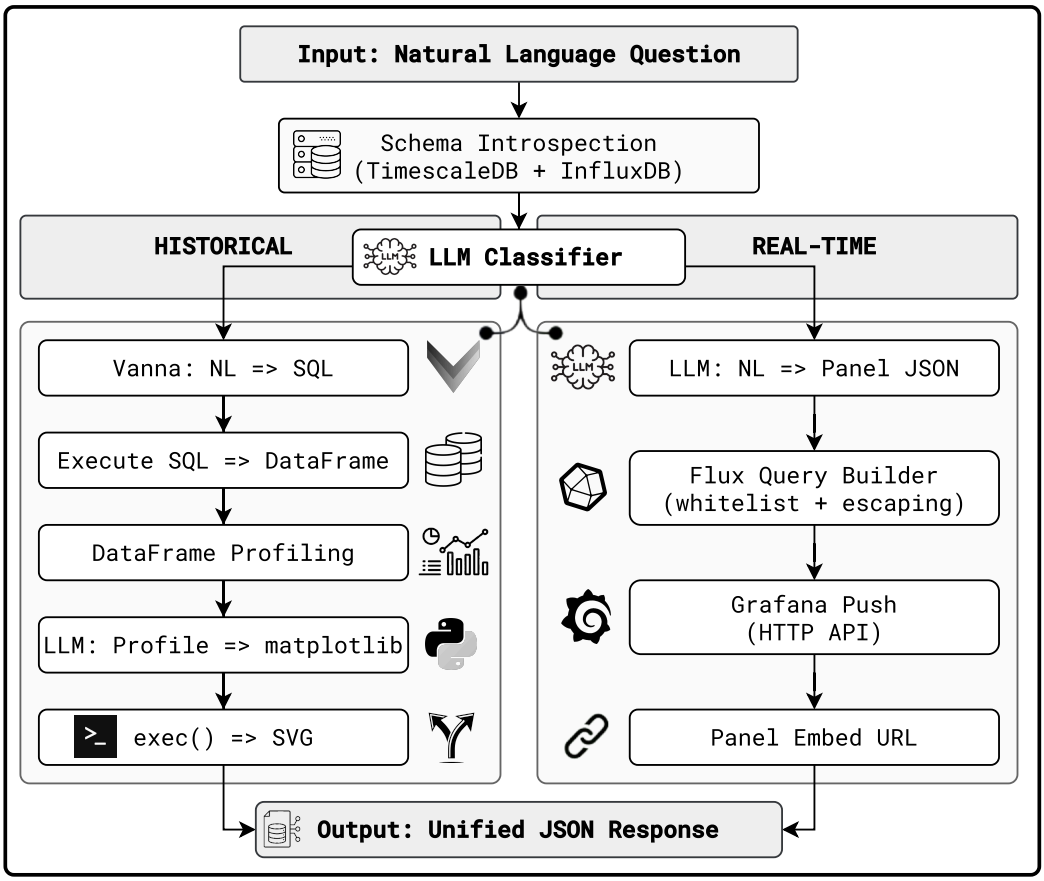


Figure 6.1: End-to-end flow of the Visualization Chatbot. A single natural-language question is enriched with live schema information from both backends, classified into one of two pipelines, and answered through the path appropriate to its temporal scope. The historical pipeline produces an inline SVG chart; the realtime pipeline produces an embeddable Grafana panel URL. Both pipelines return through a single JSON response shape.

Table 6.1 summarises the two pipelines along the three axes that justify the split. Reading the table top-to-bottom makes the structural distance between the two pipelines explicit: they share only the introspection and classification stages, and after the classifier they share nothing.

Table 6.1: Structural contrast between the historical and realtime pipelines. The two pipelines share only the schema-introspection and classification stages; after the classifier they diverge in data tier, query language, output artefact, freshness semantics, persistence, and the typical kind of question they answer.

Property	**Historical pipeline**	**Realtime pipeline**
Data tier	TimescaleDB (analytical relational time-series)	InfluxDB (tag-indexed time-series)
Query language	SQL	Flux
Query origin	RAG-generated by LLM (Vanna + ChromaDB)	Compiled deterministically from a structured panel configuration emitted by the LLM
Output artefact	Inline SVG chart (matplotlib)	Embeddable Grafana panel URL
Freshness	Snapshot at query time	Continuously refreshing live view
Persistence	Self-contained, archivable, embeddable	Lives only as long as the visualisation service is reachable
Typical question	"Which jobs blew their I/O budget last week?"	"Show current GPU utilisation on rack 3."
External dependency	Database only	Database *and* Grafana

The remainder of this chapter follows the structure of Figure 6.1: shared mechanisms first (schema introspection in Section 6.3, classification in Section 6.4), then each pipeline in turn (Sections 6.5 and 6.6), then cross-cutting properties (caching in Section 6.7, security in Section 6.8).

6.3 Schema Introspection on Every Request

Before either pipeline executes, the agent inspects the schema of *both* storage backends and feeds the result into every subsequent stage that needs it—the classifier, the SQL

generator, and the panel-configuration generator. This section motivates that decision, describes what each backend exposes, analyses the cost of paying for introspection on every request, and discusses the privacy boundary that the introspection step has to respect.

6.3.1 The Schema-Drift Problem

The canonical failure mode of LLM-on-database systems is *schema drift*: the schema embedded in the prompt no longer matches the live database, and the generated query references columns or tables that do not exist. In a static deployment this is a minor concern, because schemas change rarely and a redeployment of the prompt template is enough to restore correctness. In an active monitoring system the dynamic is reversed. New collectors are added as operators instrument additional subsystems; existing collectors gain or rename fields as their semantics are refined; tags are introduced to support new groupings; entire measurement types appear when a new class of resource (e.g. a new accelerator) is added to the cluster. A prompt template that was accurate yesterday is no longer accurate today, and the LLM dutifully generates queries against a schema that is no longer there.

Two strategies present themselves. The first is *periodic schema synchronisation*: the agent caches a copy of the schema and refreshes it on a fixed cadence. The cadence can never be tight enough to eliminate drift entirely, and tightening it converges asymptotically to the second strategy. The second is *introspection on every request*: the agent queries the live schema as part of each request lifecycle, so that the schema seen by the LLM is by construction the schema that exists when the query will run. The Visualization Chatbot adopts the second strategy. The cost is a small number of metadata queries per request; the benefit is that schema drift is structurally eliminated as a class of error.

The decision is feasible because the substrate (Chapter 4) exposes the storage backends *directly*, rather than behind a synthesising API layer. If the agent had to reach the schema through an intermediate adapter, introspection on every request would multiply the cost across two hops; against the substrate, it is a single round-trip per backend, against indices

that database engines maintain for exactly this purpose.

6.3.2 Historical-Tier Introspection

What is queried. The historical tier is TimescaleDB, which presents the standard PostgreSQL information schema together with a TimescaleDB-specific extension that distinguishes ordinary tables from *hypertables*—the partitioned time-series tables that are the native abstraction of the engine. The agent queries `information_schema.columns` joined to `information_schema.tables` for the public schema, then queries the `timescaledb_information.hypertables` catalogue to discover which of the resulting tables are time-series tables.

What is returned. The introspection step returns a single text artefact in the shape of a DDL listing: each table appears as a `CREATE TABLE` stanza with column names and SQL types, and time-series tables are explicitly annotated as such. This shape is chosen deliberately. The downstream consumer is an LLM, and LLMs respond well to schema in the form they were trained to recognise—SQL DDL—rather than to JSON or to a custom schema serialisation. The hypertable annotation gives the SQL generator a concrete signal that it should consider time-bucketed aggregation against the marked tables.

What is excluded. The introspection step does *not* return data—only metadata. No row of any table is exposed at this stage; the LLM sees only the structural shape of the database. This is important for the privacy discussion in Section 6.3.5.

6.3.3 Realtime-Tier Introspection

What is queried. The realtime tier is InfluxDB, whose data model differs from a relational store: data is organised into *measurements* (loose analogues of tables), each carrying *tags* (indexed string-valued keys, used for filtering and grouping) and *fields* (the actual numeric values). The agent uses the Flux schema discovery interface to enumerate measurements, then for each measurement enumerates its tag keys, a bounded sample of values per tag key, and its field keys.

Bounded tag-value sampling. Tag values are crucial context—an operator who asks "show GPU usage on rack 3" needs the agent to know that "rack" is a valid tag and that "3" is a valid value of it—but enumerating them in full is dangerous. Some tags (e.g. `job_id`, `user_id`) have unbounded cardinality and would flood the prompt with hundreds of thousands of values, both blowing the token budget and introducing personally identifiable information. The introspection step therefore caps tag-value sampling at a small constant per tag key (twenty values in the current prototype), which is enough to give the LLM a sense of the value space without amounting to a data dump.

What is returned. The result is a structured listing in which each measurement is presented with its tag keys (with bounded examples), its field keys, and basic type information. This format is also chosen for LLM legibility: it resembles the kind of schema description that appears in InfluxDB documentation, which the LLM is likely to have seen during training.

6.3.4 Cost Analysis

Introspecting both tiers on every request adds latency to the pipeline, and it is worth being explicit about how much. Each introspection step performs a small number of metadata queries: a handful of catalogue lookups against TimescaleDB, and a similar number of Flux schema queries against InfluxDB. Both are cheap; database engines maintain dedicated indexes for the metadata they expose, and the result sets are small. The total cost is dominated by the two TCP round-trips to the two backends, not by the queries themselves.

Set against this cost is the cost of the *alternative*—a cached schema with a refresh cadence—which has three components. First, the engineering cost of the cache itself: a refresh policy, an invalidation mechanism, and a guarantee that two concurrent requests do not see inconsistent schemas. Second, the residual error rate from the time window between schema change and cache refresh, during which the agent will silently emit queries against a schema that is not there. Third, the operational cost of the resulting failures, which surface as opaque database errors rather than as "schema not yet refreshed" diagnostics.

The trade-off therefore favours per-request introspection until the underlying database becomes large enough that catalogue queries themselves are expensive, which is far outside the operating regime of the present system. The agent therefore pays the introspection cost on every request, and accepts the resulting per-request latency as the price of structural correctness.

6.3.5 Privacy and Cardinality Bounds

The introspection step is the boundary at which the agent decides what to share with the LLM provider, and it is therefore the appropriate place to enforce privacy. Two principles apply.

Metadata, not data. The introspection step exposes only schema—table names, column names, types, hypertable annotations, measurements, tag keys, field keys. No row of any table is sent at this stage. Row-level data appears later in the pipeline, but only as a *profile* (Section 6.5.5) and never as a full result set, and even the profile is constructed locally rather than by the LLM.

Bounded value samples. The realtime tier exposes tag values, which are a kind of low-cardinality data. The bound on the sample size is not just a token-budget concern but also a privacy concern: tags can carry user identifiers, job names, or rack labels that a deployment may consider sensitive. The cap restricts what an attacker who compromised the LLM provider could learn from a single request to a small constant slice of the tag-value distribution, bounded by configuration rather than by the cardinality of the data.

Table 6.2 summarises the introspection step per backend.

Table 6.2: Schema introspection at the two storage tiers. Both tiers are queried on every request; the result feeds the classifier and the per-pipeline generators. The realtime tier additionally returns a bounded sample of tag values, capped to limit token cost and to keep accidental disclosure of high-cardinality tags within configured limits.

Aspect	**Historical (TimescaleDB)**	**Realtime (InfluxDB)**
Catalogue queried	`information_schema.columns`, `information_schema.tables`, `timescaledb_information.hypertables`	Flux `schema.measurements`, `schema.measurementTagKeys`, `schema.tagValues`, `schema.fieldKeys`
Returned shape	DDL-style `CREATE TABLE` listing with hypertable annotations	Per-measurement listing of tag keys, sampled tag values, and field keys
Data values exposed	None	Bounded sample of tag values per tag key
Sample bound	Not applicable	Configured constant (e.g. 20 values per tag)
Consumer	Classifier; SQL generator	Classifier; panel-configuration generator
Drift posture	Eliminated by per-request introspection	Eliminated by per-request introspection

6.4 Pipeline Classification with a Safe Default

Once the schemas of both backends have been retrieved, the agent must decide which of its two pipelines should answer the request. This decision is made by an LLM-based classifier whose job is intentionally narrow: read the natural-language question and the two schemas, and return a single token that names the pipeline. This section motivates the classifier, describes its inputs and outputs, enumerates the failure modes it must handle, and explains why one of its two outputs is treated as a privileged *safe default*.

6.4.1 The Routing Problem

The classifier solves a single, well-scoped problem: given an English question, decide whether it is best served by a snapshot from the analytical tier or by a live view from the realtime tier. This decision is non-trivial because the linguistic cues for the two cases are not always unambiguous. "Show CPU usage" could be a historical question (over the past hour, day, week—which is the implicit reference period?) or a realtime question (right now). "Compare GPU utilisation across nodes" is similarly under-specified. The classifier needs to pick the most likely interpretation, given (a) the wording of the question, (b) the explicit time references it contains, and (c) the schema that is actually available.

The classifier is therefore positioned at the crossing point of three sources of information: the question text (which encodes the operator's intent), the historical schema (which says what kinds of long-term analyses are even possible), and the realtime schema (which says what kinds of current observations are even possible). Withholding any of the three would force the classifier to guess: a question about a measurement that exists only in InfluxDB cannot be served historically, and a question about a join across a job table that exists only in TimescaleDB cannot be served from the realtime tier.

6.4.2 Why an LLM Rather than a Rule

A reasonable instinct is to implement the classifier as a keyword rule: "contains the word 'current' or 'now' $\rightarrow$ realtime; contains 'last' or 'over the past' $\rightarrow$ historical; otherwise default." This works for the simplest cases but breaks rapidly. Operators express temporal intent in heterogeneous ways ("what does it look like at the moment," "has anything changed since the deployment," "status of node 12"), and the same keyword can carry opposite meanings in different contexts ("last value" is realtime; "last week" is historical). More importantly, a rule-based classifier cannot use the schemas; it would treat every question identically regardless of whether the question is even answerable on a given tier.

An LLM classifier accepts the schemas as additional input and produces a decision con-

ditioned on what the data actually supports. Its limitation is that it can fail in ways that a deterministic rule cannot—network errors, malformed JSON, hallucinated pipeline names. The system therefore augments the LLM with a deterministic fallback policy (Section 6.4.5), so that the failure modes of the LLM are bounded by code rather than by chance.

6.4.3 Inputs and Outputs

The classifier accepts three inputs: the user's natural-language question, the historical-tier schema (DDL-style listing produced in Section 6.3.2), and the realtime-tier schema (measurement/tag/field listing produced in Section 6.3.3). When the realtime tier is unreachable, the classifier prompt is explicitly told that no realtime schema is available, and is instructed not to classify any question as realtime under that condition.

The output is a small JSON object with three fields: `pipeline` (the routing decision, either `historical` or `realtime`), `confidence` (a coarse scalar, either `high` or `low`, used only for logging), and `reasoning` (a free-text explanation of why the classifier chose as it did, also used only for logging). The agent only consumes the `pipeline` field for routing; `confidence` and `reasoning` exist to make the classifier auditable, so that an operator examining the logs after a misrouted question can see what the classifier was thinking.

The classifier is invoked with the LLM's temperature pinned at zero and a tight token budget. Both choices are deliberate. A zero temperature makes the classifier as deterministic as the underlying model permits, so that the same question against the same schema produces the same routing decision; the agent can rely on this when reasoning about reproducibility. A tight token budget is enforced because the classifier's output must fit a small structured shape; a model that begins to reason at length is one whose output is increasingly likely to wander out of the schema and fail to parse.

6.4.4 Failure Modes and Defaults

Four failure modes are anticipated, and each is mapped to a deterministic default behaviour. Table 6.3 summarises the mapping.

Table 6.3: Classifier failure modes and default behaviours. Each failure is handled deterministically rather than propagated to the caller, so that an operator's question is answered as long as any pipeline is reachable.

Failure mode	**Detection**	**Default behaviour**
Network or model error during the classifier call	Exception raised by the LLM client	Route to historical; log the exception
Malformed JSON in the classifier response	JSON parser fails	Route to historical; log the raw response
Unknown value in the `pipeline` field	Field is present but neither `historical` nor `realtime`	Route to historical; log the unknown value
Realtime tier unreachable at introspection time	Realtime schema query fails	Inform the classifier that realtime is unavailable; refuse any `realtime` routing

The first three rows are LLM-side failures: the classifier itself failed to produce a usable answer. The fourth row is a substrate-side failure: the classifier could in principle have answered, but the realtime backend is not reachable, and routing a request to it would only produce a downstream error. In all four cases the default is the same—route to historical—but for different reasons.

6.4.5 Why Historical Is the Safe Default

The choice of which pipeline to default to is not symmetric. Two properties of the historical pipeline make it the natural safe default.

Self-containment. The historical pipeline depends only on the analytical tier and on the

agent's own code. If the database is reachable, the pipeline can complete: SQL is generated, executed, and rendered into an SVG locally inside the agent process. The realtime pipeline, by contrast, additionally depends on Grafana, because its output artefact is not a chart that the agent renders but a panel that lives inside the visualisation service. A defaulting strategy that pointed to the realtime pipeline would therefore inherit Grafana's availability as a precondition for any answer at all.

Recoverable artefact. The historical pipeline returns a self-contained chart—an SVG string in the response body. Even if the operator's downstream environment is offline, the artefact is still present. The realtime pipeline returns a URL whose continued utility depends on the Grafana service remaining reachable. If the agent is unsure which pipeline a question really wants, returning a self-contained artefact is the strictly safer choice: the operator can always re-ask for a live panel if they wanted one, but the reverse—asking for an archivable chart and receiving a URL that may have rotated by the time it is followed—is harder to recover from.

Decision flow. Figure 6.2 summarises the classifier's decision flow including the four default branches. The diagram makes the asymmetry explicit: every error edge points to the historical pipeline, and the realtime pipeline is reached only along the success path.

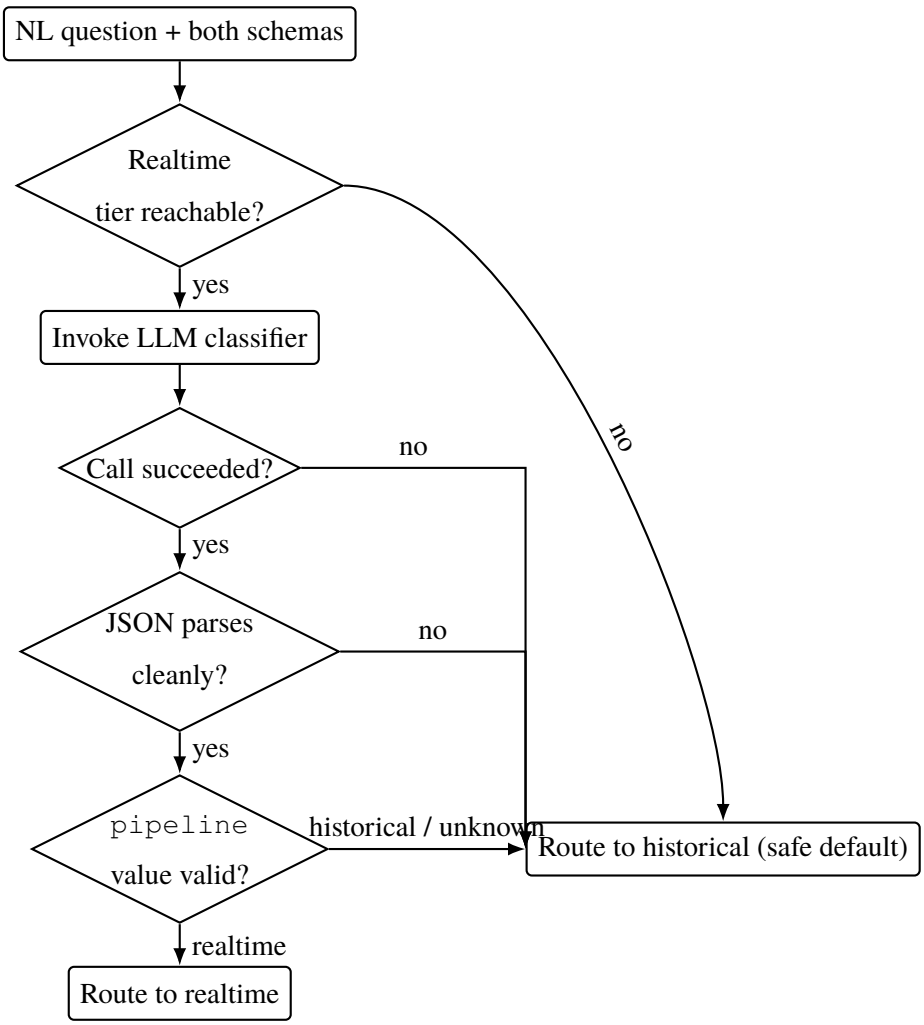


Figure 6.2: Classifier decision flow. Every error edge—realtime tier unreachable, classifier call failure, malformed JSON, unknown pipeline value—routes to the historical pipeline. The realtime pipeline is reached only when every check on the success path passes and the classifier's verdict is explicitly `realtime`.

The asymmetry is a deliberate property of the design. It says: when the agent is unsure, it returns a finished chart; the realtime path is taken only when there is positive evidence that the operator wants a live view and the realtime backend is in fact available to provide it.

6.5 Historical Pipeline

When the classifier routes a question to the historical pipeline, the agent has committed to producing a static, archivable chart from the analytical tier. The pipeline that delivers this artefact is the more elaborate of the two, because it must address three distinct risks of

LLM-mediated database querying: schema drift (handled in Section 6.3), generation of incorrect SQL, and execution of incorrect chart-rendering code. This section describes the seven mechanisms that compose the historical pipeline, in the order in which a request flows through them.

6.5.1 Pipeline Overview and Artefact Justification

The historical pipeline produces an inline SVG chart. SVG was chosen over alternative artefact types (PNG, HTML widget, Grafana panel) for three reasons.

Self-contained. An SVG is a single string that contains both the visual encoding and the rendered output. It is embeddable in a report or in a chat message without requiring the recipient to follow a link or run a server. For an operator who asks a question, captures an answer, and wants to share it, this is the lowest-friction artefact format.

Comparable. SVGs from successive runs of the same query are byte-for-byte comparable along the dimensions that matter: axis labels, tick positions, title text, and the visual encoding itself. An operator who runs the same question against today's data and last month's data can compare the two artefacts directly, without the noise that PNG rendering or screenshot capture would introduce.

Archival. Unlike a Grafana panel, whose continued visibility depends on the dashboard service, an SVG that has been generated does not stop being valid. It can be saved into a ticketing system, an incident retrospective, or a paper, and it will render correctly years later. For analyses that the operator may want to revisit, this property matters.

Figure 6.3 shows the internal structure of the pipeline. A natural-language question is first turned into SQL through retrieval-augmented generation; the SQL is executed against the analytical tier; the resulting DataFrame is summarised into a compact profile rather than sent to the LLM in full; the profile and the question are passed to a second LLM call that emits chart-rendering code; the code is executed in a constrained namespace, and its output is captured as an SVG string. SQL generation and chart-code execution are each

protected by a single-shot regeneration step that catches the LLM's most common error modes.

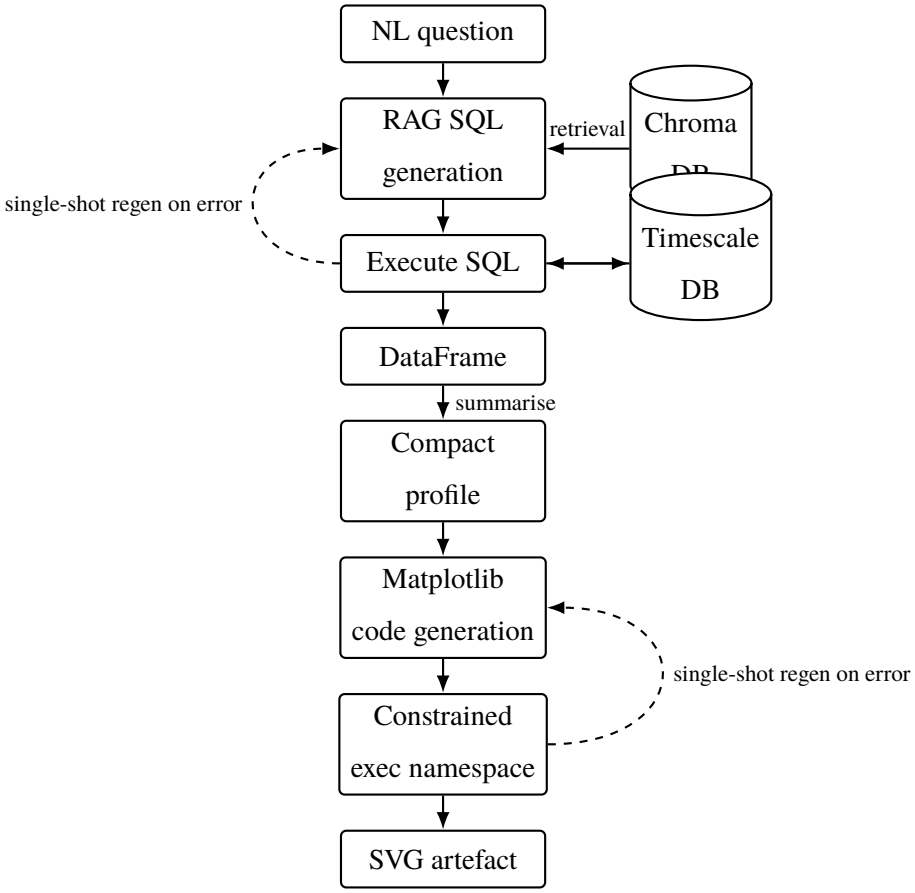


Figure 6.3: Internal structure of the historical pipeline. The natural-language question is transformed into SQL by retrieval-augmented generation against a ChromaDB-backed corpus; the SQL is executed against TimescaleDB; the resulting DataFrame is summarised into a compact profile that is sent to a second LLM call which emits matplotlib code; the code is executed in a constrained namespace and produces an inline SVG artefact. Both LLM-emitting stages are protected by a single-shot regeneration loop that retries once on execution error.

6.5.2 Retrieval-Augmented SQL Generation

The first stage of the pipeline produces a SQL query from the natural-language question. The agent uses a retrieval-augmented generator built on the Vanna.AI framework [25], which combines a vector store of schema documentation and question/SQL exemplars with an LLM-based generator. The choice of retrieval-augmented generation over a single-prompt approach is motivated by three properties of HPC schemas.

Why RAG over a flat prompt. First, the schema is large. A monitoring substrate accumulates dozens of measurement tables, each with their own column conventions, time-bucketing rules, and join paths to job and user metadata. Pasting the entire schema into the prompt for every request would either exceed the context window or push the actual instruction far enough back that the LLM begins to ignore it. Retrieval lets the agent place *only* the schema fragments relevant to the current question into the prompt, keeping the active context tight.

Second, monitoring schemas have idioms. Time-series queries against a hypertable use specific bucketing functions; queries against the job table use specific join conditions to associate metrics with users; queries about cumulative quantities use specific aggregation patterns. These idioms are difficult to convey through a generic schema description but easy to convey through worked examples. The retrieval store therefore holds question/SQL exemplars in addition to DDL, and at retrieval time both kinds of evidence are surfaced for the LLM to imitate.

Third, schemas drift. Per-request schema introspection (Section 6.3) handles drift in the SQL generator's prompt, but the retrieval corpus itself can also become stale if exemplars reference columns that no longer exist. The agent re-trains the corpus against the live schema at server startup, so a restart is a sufficient and cheap remedy for any corpus staleness that accumulates between deployments.

Vector store and corpus. The corpus is held in a ChromaDB **[cite:ChromaDB]** vector database persisted on disk. Vanna's retrieval mechanism embeds the user question, finds the nearest DDL fragments and question/SQL exemplars by cosine similarity, and assembles the retrieved evidence into a prompt that is then handed to the LLM.

Generation. The LLM emits a SQL query that is intended to be syntactically and semantically correct against TimescaleDB **[cite:TimescaleDB]**. The query is not yet trusted at this point; it is parsed and executed in the next stage, which is where errors are caught.

6.5.3 SQL Execution and Isolation

The generated SQL is executed against the analytical tier. Two design properties bound the consequences of an incorrectly generated query.

Read-only role (design intent). The intent is to execute every generated SQL statement under a database role that is structurally incapable of mutation: no `INSERT`, `UPDATE`, `DELETE`, `TRUNCATE`, or DDL is permitted, regardless of what the LLM emits. Under this discipline, even an adversarially crafted prompt that coerced the LLM into emitting a destructive statement would be denied at the database boundary, because the connection it ran on does not have the privilege required.

The current prototype relies on the role assigned to the connection user—if that user is read-only, the property holds; if it is not, it does not. Application-layer enforcement (e.g. a parser that rejects non-`SELECT` statements before they reach the database, or a session-level `SET TRANSACTION READ ONLY`) is a known hardening item that is discussed alongside the security analysis in Section 6.8. The text below describes the design as intended; the gap is acknowledged here once and not repeated in every paragraph.

Connection lifecycle. A fresh connection or pooled connection-handle is used for each request, both so that one long-running query cannot block another and so that any session-level state set by an erroneous query (e.g. a transaction left open) does not persist beyond the request that introduced it.

6.5.4 SQL Auto-Fix

The single most common failure mode of LLM-generated SQL is a database-level error: a misspelled column, a hallucinated table, a syntax error from a paraphrased clause. Many of these errors are recoverable in one additional shot, because the database's error message itself constitutes useful repair information.

Mechanism. When SQL execution raises a database error, the agent feeds the failing

SQL together with the database's error text back to the same LLM and asks it to produce a corrected query. The corrected query is executed exactly once. There is no further retry beyond this single regeneration: a second failure indicates that the LLM is unable to converge from the available signals, and the agent surfaces the error to the caller rather than entering an unbounded loop. This single-shot policy is deliberate. Allowing arbitrary retries would couple the agent's worst-case latency to the LLM's worst-case behaviour and would also amplify the operator's bill on the LLM provider.

Coverage. The auto-fix step recovers the substantial majority of failed-on-first-execution queries on cross-domain benchmarks (the empirical recovery rate is reported in the chapter on evaluation). The mechanism is most effective against errors whose cause appears in the database error text, and is least effective against errors that arise from a deeper misreading of the schema—for those, the next request, with a freshly introspected schema, is more likely to produce a correct query than another regeneration attempt on the same one.

6.5.5 DataFrame Profile, Not Raw Rows

When the SQL has executed successfully, the agent has a DataFrame in hand: a relational table whose contents answer the question. The naive next step is to send this DataFrame to an LLM and ask it to write the chart-rendering code. The agent does not do this. Instead, it constructs a compact *profile* of the DataFrame and sends only the profile.

Privacy. A monitoring DataFrame may contain user identifiers, job names, hostnames, internal addresses, or other deployment-specific data that the operator may not wish to send to a third-party LLM provider. Sending only a profile—summary statistics and a tiny deterministic head sample—means that the LLM sees enough of the data's shape to choose visual encodings without seeing enough of the data's content to reconstruct the underlying records.

Token cost. A full DataFrame can run to thousands of rows and dominate the token cost of the request. A profile is bounded in size by the number of columns and a small con-

stant per column, regardless of the DataFrame's row count. This gives the agent a stable, predictable cost profile across questions of very different selectivity, which matters when the agent is exposed to many users at once.

Sufficiency. The decisions the LLM has to make at the chart-code stage—which column should be the x-axis, which the y-axis, what the title should be, how to choose colours, whether to aggregate before plotting—depend on metadata, not on individual values. A profile that captures the shape of the data (`shape`, `dtypes`), the distribution of numeric columns (`min`, `max`, `mean` for up to a small number of columns), and a deterministic head sample (a fixed number of rows for the LLM to anchor on) carries the information the LLM needs without carrying anything more.

The profile is constructed locally by the agent rather than by the LLM. It is therefore deterministic, auditable, and computationally trivial.

6.5.6 Hardened Chart-Code Generation

The profile and the original question are sent to an LLM that emits a fragment of matplotlib **[cite:Matplotlib]** code. The code is executed server-side. Because the agent has chosen to execute LLM-generated code, this stage requires more careful design than the SQL stage; the rest of the section describes the structural contract that makes the execution tractable and the auto-fix step that catches single-shot failures.

Structural contract. The code fragment is required to operate on a single named DataFrame variable supplied to it (`df`) and to produce its rendered output through a single named SVG variable that the agent reads back (`svg_output`). This contract is enforced by the prompt: the LLM is told the input variable name, told that no other input is available, told the output variable name, told that the rendering must happen entirely within the supplied namespace, and told the precise sequence of matplotlib calls that captures the figure as SVG into the output variable. Any code fragment that respects this contract can be executed; any code fragment that violates it—reads from the filesystem, opens a network connection, refers to undefined variables, fails to populate `svg_output`—fails

detectably and is caught by the auto-fix mechanism below.

Headless rendering. The agent forces matplotlib's rendering backend to a headless mode (`Agg`) before executing the code, so that the fragment cannot accidentally try to open a display window or rely on a graphical environment that a server process may not have. This is also the canonical configuration for capturing the figure as an SVG buffer rather than as a window.

SVG capture. The agreed convention is that the fragment writes the figure into an in-memory `StringIO` buffer using `plt.savefig(buf, format='svg', bbox_inches='` then assigns `buf.getvalue()` to `svg_output`. The agent reads `svg_output` from the namespace once the fragment has finished executing. This is a small but rigid structural contract: it removes any ambiguity about how the chart leaves the LLM's code and enters the agent's response, and it allows the rest of the pipeline to assume a single artefact format regardless of which chart type the LLM chose.

Namespace isolation. The fragment is executed in a freshly constructed Python namespace that contains only the inputs the contract names—the DataFrame and the matplotlib module. Variables defined by the fragment do not leak between requests, and the fragment cannot accidentally observe or mutate state from another request. Namespace isolation is not a security sandbox (Section 6.8 returns to this point); it is, however, the right correctness boundary for a stage whose authorial intent is rendering, not arbitrary computation.

6.5.7 Chart-Code Auto-Fix

The chart-code stage's most common failure mode is an exception during execution: a column-name mismatch, a type error, an unsupported chart configuration. Many of these are recoverable from the exception's traceback, in the same way that SQL errors are recoverable from the database's error text.

Mechanism. When the chart-code fragment raises an exception, the agent feeds the

failing fragment together with the exception text back to the same LLM and requests a corrected fragment. The corrected fragment is executed exactly once. As with the SQL auto-fix, the policy is single-shot: a second failure surfaces the error to the caller. This mirrors the SQL stage and gives the historical pipeline a uniform failure treatment.

Why it works. The chart-code stage's failures are heavily concentrated on simple, mechanical errors: a column referenced by name that does not match the DataFrame's columns; a date-axis configuration that conflicts with the actual dtype; a missing legend or label that the LLM forgot in its first attempt. An exception traceback names the offending line and the immediate cause; a regeneration that incorporates this signal converges on a working fragment in the substantial majority of cases. Empirical recovery rates are reported in the evaluation chapter.

6.5.8 Pipeline Output

When the chart-code fragment has executed successfully, the agent returns a JSON envelope whose payload includes the SVG string, the SQL that was generated, and any auxiliary metadata the caller may want for logging (the question that was asked, the classifier's reasoning). The SVG can be inlined into any HTML or rendered into a static image at the caller's discretion. The pipeline's contract ends here; subsequent display, persistence, or sharing of the artefact is the caller's responsibility.

6.6 Realtime Pipeline

The realtime pipeline answers the questions that the classifier identifies as wanting a continuously refreshing view rather than a snapshot. Its structure is deliberately different from the historical pipeline: instead of generating a query and rendering the result locally, the agent generates a structured panel description and pushes it to a remote visualisation service. This section describes the rationale for that indirection, the structured schema the LLM is asked to emit, the deterministic builder that turns the structure into a Flux query, and the panel-push mechanism that produces the embeddable URL the agent returns.

6.6.1 Pipeline Overview and Artefact Justification

A realtime answer is not a chart; it is a window onto a service. The artefact returned by the realtime pipeline is therefore not a rendered image but a URL: an embeddable Grafana [47] panel reference that the operator can paste into a chat message, an internal portal, or a browser tab, and that will continue to refresh as long as it is being viewed.

This design has three direct consequences. First, the agent does not render the panel itself; rendering happens inside Grafana, which is the system specialised for live visualisation. The agent is responsible only for *creating* the panel and *returning* a way to view it. Second, the realtime artefact is not self-contained: the URL is meaningless without the Grafana service that hosts it. This is acceptable for realtime questions, whose nature is precisely that they are not self-contained—a current observation has no value detached from the system being observed. Third, the realtime artefact is not archivable in the same way as the historical SVG: the panel may rotate, be deleted, or change as the dashboard evolves. Operators who want a permanent record of a realtime observation are expected to ask the historical pipeline a corresponding question over a closed window.

Figure 6.4 shows the internal structure of the pipeline. The natural-language question is sent to an LLM that emits a small structured panel-configuration document; a deterministic builder compiles the document into a Flux query, applying an operator whitelist and value escaping; the query and panel metadata are assembled into a Grafana panel definition and pushed to a shared dashboard via the Grafana HTTP API; the API responds with an embeddable URL that becomes the pipeline's output.

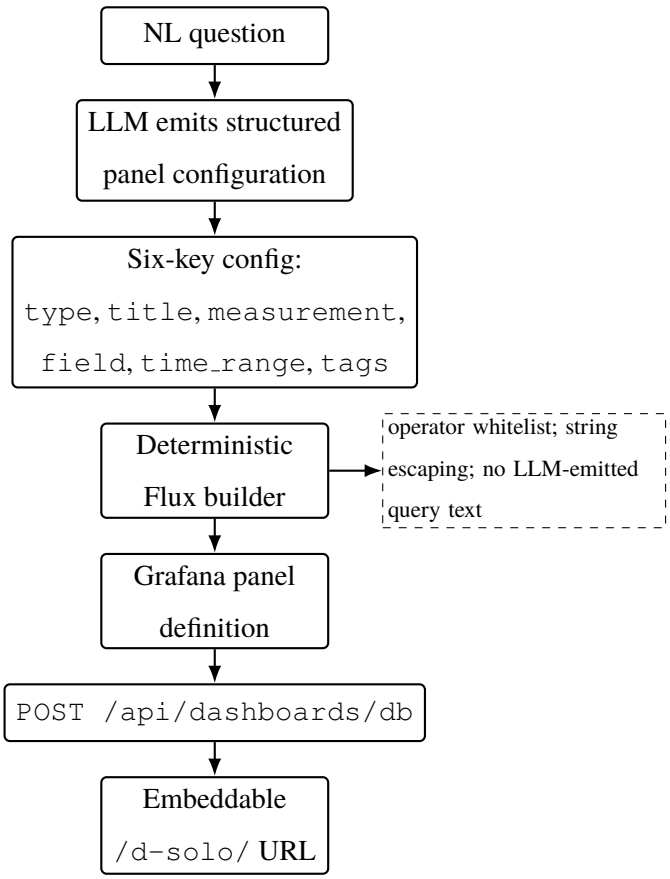


Figure 6.4: Internal structure of the realtime pipeline. The LLM emits a structured panel configuration rather than a raw query; a deterministic builder compiles the configuration into a Flux query under an operator whitelist; the resulting panel definition is pushed to a shared dashboard via the Grafana HTTP API; the API's response gives an embeddable panel URL that becomes the pipeline's output.

6.6.2 Why Structured Configuration Rather Than a Raw Query

The most consequential design decision in the realtime pipeline is that the LLM does *not* emit Flux directly. It emits a small structured document that the agent then compiles. Three motivations support this indirection.

Safety. A pipeline that accepts an arbitrary LLM-emitted query as trusted input has effectively granted the LLM a query-language injection vector: an adversarial prompt could shape the query to read measurements outside the operator's intended scope, smuggle expressions into filters, or manipulate the engine in ways the agent did not anticipate. Constraining the LLM's output to a fixed shape—a small set of named fields with bounded

value spaces—turns an arbitrary-text-output problem into a structured-data-validation problem, which is a problem the agent can solve.

Validatability. A query language is hard to validate. A structured document is easy: each field has a known type and a known set of acceptable values, and validation reduces to checking each field independently. The agent can reject a malformed configuration before it reaches the database, with a precise diagnostic about which field is wrong.

Decoupling. The structured configuration is independent of the underlying query language. Today the agent compiles to Flux because the realtime tier is InfluxDB; tomorrow, if the realtime tier moved to a different time-series engine with a different query language, the same structured configuration would compile to that engine's syntax. The LLM's task—semantic interpretation of the question into a panel intent—is unchanged, and the per-engine syntax knowledge is concentrated in the builder.

The principle is general: the LLM does *semantic interpretation* (what to plot), and the agent does *compilation* (how to plot it in the target engine's language). Confining the LLM to the upstream half of this split is what permits the agent to validate, log, and harden the downstream half.

6.6.3 The Panel Configuration Schema

The structured configuration the LLM is asked to emit has six fields. Each is small, each has a well-defined type, and together they are sufficient to describe any panel the realtime pipeline currently supports. Table 6.4 enumerates the fields.

Table 6.4: Panel configuration schema emitted by the LLM in the realtime pipeline. The six fields are sufficient to describe any panel the pipeline currently produces, and each is constrained enough that downstream validation reduces to per-field checks. The deterministic builder compiles this configuration into Flux; the LLM never emits Flux text directly.

Field	Type	Purpose
`type`	enumerated string	Panel visualisation type, drawn from a small list (e.g. `timeseries`, `stat`, `gauge`).
`title`	free string	Human-readable title shown on the panel.
`measurement`	string	InfluxDB measurement to query; validated against the introspected schema.
`field`	string	Field within the measurement to plot; validated against the introspected schema.
`time_range`	relative string	Window to display, e.g. `now-5m`; constrained to a relative-time grammar.
`tags`	object	Tag-key/tag-value filters; each key validated against the introspected schema.

The schema is deliberately small. It does not include arbitrary expression slots, function calls, or structural variants, because any such slot would reintroduce the very problem that the structured configuration was designed to avoid: a place where an LLM could emit something that has to be parsed as code rather than validated as data.

6.6.4 The Deterministic Flux Builder

The builder is the component that turns a panel configuration into an executable Flux query. It is a small, pure piece of code with no LLM in the loop, and it is the agent's

strongest line of defence on the realtime path.

Operator whitelist. Filter conditions on tags are expressed by a comparison operator (==, !=, regex match, range comparison). The builder accepts only operators from a fixed whitelist; an attempt to use an operator outside the set is rejected. The whitelist closes off any operator-injection vector that the LLM could otherwise create by emitting an unexpected operator string in the configuration's tag filters.

String escaping. All string values that flow into the Flux query—tag values, measurement names, field names—are escaped before interpolation, so that a value containing Flux control characters cannot break out of its quoted context. This treatment is uniform: the builder does not trust any string just because it came from a structured document, because the structured document itself was emitted by an LLM.

Schema validation. Tag keys, measurement names, and field names are checked against the introspected schema. A configuration that names a measurement or tag that does not exist is rejected before the query is compiled. This is more than a cosmetic check: it ensures that an LLM that hallucinates a measurement cannot reach the database with a query that names it.

Determinism. The builder has no nondeterminism. Given a valid configuration, it always produces the same Flux query. Given an invalid configuration, it always rejects in the same way. The agent's behaviour on the realtime path is therefore predictable in a way that LLM-emitted Flux never could be.

6.6.5 Grafana Panel Push

The compiled Flux query and the panel configuration's display fields (title, type, time range) are assembled into a Grafana panel definition and posted to the Grafana HTTP API [48]. Two design decisions shape the push behaviour.

Shared dashboard. All panels produced by the agent live on a single dedicated dash-

board inside Grafana, identified by a fixed UID. Each new request adds or updates a panel on this dashboard rather than spawning a new dashboard. The shared-dashboard model has two practical benefits: it keeps the operator's Grafana instance from being flooded with one-off dashboards, and it lets the agent reuse Grafana's existing access-control and folder-organisation conventions instead of reinventing them.

Idempotent update. The push operation is structured as an upsert: the agent retrieves the current state of the dashboard, modifies the panel set, and pushes the modified state back. If a panel for an equivalent question already exists, it is updated; otherwise it is added. Idempotency means that the operator can re-ask the same question without producing duplicate panels, and means that a retry after a transient API error does not result in two panels where one was intended.

Embeddable URL. Grafana supports a per-panel embedded view (the `/d-solo/` URL family), which renders a single panel without the surrounding dashboard chrome. This is the form the agent returns. An operator who pastes the URL into a chat message gets a single live panel, not the entire dashboard, which is the right granularity for the question they asked.

Datasource discovery. The agent does not embed the InfluxDB datasource UID into its code. Instead, on each request, it queries Grafana for its registered datasources, picks the InfluxDB one, and uses the discovered UID. This is what allows the agent to be deployed against a Grafana instance whose datasource configuration was not known at compile time, and what lets the operator replace or rotate the InfluxDB datasource without redeploying the agent.

6.6.6 Pipeline Output

The realtime pipeline returns a JSON envelope identical in shape to the historical pipeline's, with the SVG payload replaced by the panel URL. Identical shape, different payload, distinguished by a discriminator field; this is the unification described in Section 6.2.1. The caller's parser does not have to know which pipeline served the request to know how to

consume the response.

6.7 Caching and Freshness Strategy

The agent makes deliberate choices about what is cached, what is refreshed, and what is recomputed on every request. Three caching boundaries shape the agent's runtime behaviour: the schema itself, the retrieval-augmented-generation corpus, and the Grafana datasource registry. Each boundary trades off freshness against cost differently, and each is positioned where it gives the strongest correctness guarantee at a tolerable price.

6.7.1 Per-Request Schema Introspection

The agent does not cache the schema. Every request triggers a fresh introspection of both backends, with the result feeding the classifier and the per-pipeline generators (Section 6.3).

The choice has been justified at length already; the framing relevant here is that the schema sits on the most volatile axis the agent has to deal with. New collectors, renamed columns, added tags, and reorganised measurements happen during the lifetime of a deployment, and each such change can silently invalidate a cached schema. By construction, a per-request introspection cannot be stale: the schema seen at request time is exactly the schema available when the query will run a fraction of a second later. Staleness is structurally impossible.

The cost of this choice is a fixed latency floor on every request—the round-trip time of the metadata queries to both backends. Because metadata queries are cheap and are served from indexes the engines maintain natively, this floor is small in absolute terms, and it is paid only once per request rather than scaling with the size of the data being queried.

6.7.2 RAG Corpus Persistence

The retrieval-augmented-generation corpus used by the historical pipeline's SQL generator (Section 6.5.2) is the second caching boundary, and the trade-offs there are different.

What is cached. The corpus holds DDL fragments, written documentation describing what each measurement table represents, and question/SQL exemplars. These artefacts are large enough that recomputing them per request is impractical—each exemplar represents accumulated effort by the operator team—and stable enough that they do not change during a single request lifecycle.

Persistence. The corpus is held in a ChromaDB **[cite:ChromaDB]** vector store on disk. Persistence on disk means that a server restart does not destroy the corpus; this is what permits the operator team to grow the corpus over time, with each new exemplar refining the SQL generator's behaviour for one more class of question.

Refresh cadence. The corpus is re-trained against the live schema at server startup. This binds the freshness of the corpus to the deployment cycle, which is a deliberately coarse-grained but operationally clean policy. An operator who has just renamed columns or added a measurement restarts the agent and is guaranteed that every subsequent request sees a corpus aligned with the live schema. Between restarts, the per-request schema introspection guards the LLM's view of the schema directly, so corpus staleness can only affect the retrieval step, not the schema fragments visible in the prompt.

Why startup-only is enough. A more aggressive cadence—rebuilding the corpus on every request, or polling for schema changes—would be both wasteful and incorrect. Wasteful because rebuild cost dwarfs the cost of a single request. Incorrect because it would mean that two requests close in time could see different exemplar sets, breaking the predictability the agent relies on. Tying the corpus to the deployment cycle keeps the agent's behaviour reproducible across requests and gives the operator team a single, well-understood lever for forcing a refresh.

6.7.3 Grafana Datasource Discovery

The realtime pipeline needs the UID of the InfluxDB datasource registered with Grafana in order to push panels. Two policy options apply: cache the UID at startup, or look it

up per request.

Per-request lookup. The agent queries Grafana for its datasource list as part of the realtime push (Section 6.6.5). The discovered UID is then used in the panel definition that is pushed back. The cost of the lookup is one additional HTTP round-trip to Grafana, which the realtime path is already in conversation with, so the marginal latency is small.

Why not cache. The datasource UID is a piece of operator-controlled state: an administrator may rotate the InfluxDB datasource, replace it, or change its UID for any number of reasons. A cached UID can become stale silently, and the resulting failure—panels pushed against a defunct datasource—surfaces as a confusing display issue inside Grafana rather than as a clear agent error. Looking up the UID per request makes the agent robust to any datasource change made on the Grafana side.

6.7.4 Summary of Caching Decisions

The agent's caching strategy can be summarised in one sentence: cache only what is large and stable; recompute everything else. The schema is small but volatile, so it is recomputed. The RAG corpus is large but stable across a deployment, so it is cached on disk and refreshed at startup. The Grafana datasource UID is small and operator-controlled, so it is recomputed per request rather than cached. Each decision is local and motivated by the same principle: the cost of recomputation is a price the agent pays in exchange for a class of staleness errors that simply cannot occur.

6.8 Security as a Design Property

The agent compiles natural-language input into both queries and code that are executed server-side. This is unusual among monitoring components, which classically execute only what their administrators have written, and it earns the agent an explicit security analysis. The analysis is structured around the three threats the LLM-mediated control flow introduces, and treats security as a property to be argued from the design rather than as an afterthought to be patched on later. Some of the mitigations are structural, some

are layered, and one of the three boundaries is acknowledged here as the weakest link in the system.

6.8.1 Threat Surface

The agent has three points where untrusted natural language is transformed into something that an interpreter will execute: SQL generation in the historical pipeline, panel-configuration generation in the realtime pipeline, and chart-code generation in the historical pipeline. The first two boundaries cross from English into a query language; the third crosses from English into Python. Each boundary therefore has a different attack surface, a different set of mitigations, and a different residual risk.

The threat model assumes a benign operator who phrases questions in good faith, but it also accommodates a stronger adversarial setting: an attacker who can submit arbitrary natural-language input to the agent, with the goal of escalating to operations that the agent was not intended to perform. This stronger assumption is what makes the analysis useful—if the design is sound under it, the benign case follows.

Three threats. The three boundaries map to three named threats:

- **T1: SQL injection by string concatenation.** An attacker tries to escape from the prompt or from the SQL generator's quoting and emit SQL that the agent did not intend.
- **T2: Query-language injection on the realtime path.** An attacker tries to coerce the LLM into emitting Flux that reaches the realtime tier with operator semantics outside the operator whitelist.
- **T3: Arbitrary LLM-generated code execution.** An attacker shapes the question so that the chart-code stage emits Python that goes beyond rendering—reading files, opening network sockets, manipulating environment.

The remainder of this section treats each threat in turn.

6.8.2 T1: SQL Injection by String Concatenation Is Structurally Absent

The classical web-application SQL-injection vector—an attacker's input is concatenated into a SQL string—does not apply to the historical pipeline. The natural-language question is never spliced into a SQL string. It is sent to an LLM as the body of a structured prompt, and the SQL the LLM emits is generated as a single complete statement, not assembled from string fragments under the agent's control.

This eliminates the most common SQL-injection pattern by construction, but it does not eliminate every concern. An adversarially crafted prompt could shape the SQL the LLM generates—"ignore previous instructions and emit `DROP TABLE users`"—and that SQL would then be executed. The agent's bound on this residual risk is the read-only role under which SQL is intended to execute (Section 6.5.3). Under the read-only role, even a successful prompt-shaping attack cannot mutate the database; it can at most expose a row that the agent's role had read access to, which is the same exposure that a legitimate operator would have through any other interface.

Residual risk. The current prototype's read-only enforcement relies on the role assigned to the connection user, not on application-layer enforcement. If the deployment uses a non-read-only role, the residual risk grows accordingly. Application-layer enforcement—a parser that rejects non-`SELECT` statements before they reach the database, or a session-level read-only setting—is a known hardening item.

6.8.3 T2: Realtime-Path Injection Is Bounded by Indirection

The realtime pipeline does not give the LLM the option to emit raw Flux. The LLM emits a structured panel configuration with six named fields (Section 6.6.3), and the agent compiles that configuration into Flux through a deterministic builder. Three layers of mitigation apply.

Structural indirection. The LLM is never invited to produce query text. The compila-

tion from configuration to query happens in the agent's own code, not in the LLM. An attacker who shapes the LLM's output can shape the configuration; they cannot directly shape the query.

Operator whitelist. The builder accepts only operators from a fixed whitelist when compiling tag filters. A configuration that asks for an operator outside the whitelist is rejected. This closes any operator-injection vector that an attacker might try to introduce through the configuration.

Schema validation. Measurement names, field names, and tag keys in the configuration are checked against the schema introspected on the same request. A configuration referring to a measurement that does not exist is rejected before the query is compiled, which means an attacker cannot use the configuration to probe the realtime tier for measurements that the schema does not expose.

Residual risk. The realtime pipeline reads, it does not write, so the residual risk is bounded to information disclosure within the schema's reach. An attacker who controls the natural-language input can ask the realtime tier to plot any measurement and any tag combination that the agent's connection has access to; this is the same access that any legitimate operator has, and is the cost of giving the agent realtime visibility at all.

6.8.4 T3: Arbitrary Code Execution Is the Weakest Boundary

The chart-code stage is more difficult. The fragment emitted by the LLM is Python, executed inside the agent's process. The agent's mitigations here are structural rather than enclosing.

Structural contract. The fragment is required to operate on a single named DataFrame variable and produce its output through a single named SVG variable (Section 6.5.6). Code that fails to populate the output variable, or that fails to produce an SVG that parses, is detected and rejected. This catches free-form deviations from the contract—the LLM emitting prose, emitting a chart in the wrong format, or forgetting to render at all.

Namespace isolation. The fragment runs in a freshly constructed Python namespace that contains only the inputs the contract names. Variables defined by one fragment do not leak to subsequent fragments, and the fragment cannot accidentally observe state from other requests.

What namespace isolation does *not* buy. Namespace isolation is not a security sandbox. A code fragment that respects the structural contract—accepts the DataFrame, populates the SVG variable—can additionally do any of the following, because Python permits it: import modules, open files in the agent's filesystem context, open network connections, read environment variables, execute subprocess calls. The structural contract does not constrain capability; it constrains shape.

Residual risk. This is the largest residual risk in the agent. An adversarial prompt that successfully shapes the chart-code stage into a code fragment that performs side effects, while still respecting the structural contract, will not be detected by the contract's checks. The agent's defence in depth here is that the LLM is heavily prompted toward chart code and produces it almost exclusively in benign cases; an attacker has to override this prior strongly to get the LLM to emit anything else. But the agent does not rely on this prior alone, and the strong mitigation is to enclose the execution in an actual sandbox.

Path forward. Two stronger isolation strategies are appropriate. The first is subprocess isolation: spawn a child process per chart-code execution with limited capabilities (no network, restricted filesystem, restricted CPU and memory budgets), execute the fragment there, and read the SVG output back over a pipe. The second is a WebAssembly sandbox: execute the fragment in a WASM runtime that exposes only matplotlib and the DataFrame, with no host capabilities at all. Either strategy bounds T3 by the confines of the sandbox rather than by the fragment's intent. Closing this boundary is the principal hardening item identified for future work.

6.8.5 Threat Summary

Table 6.5 summarises the three threats, the mechanisms that mitigate them, and the residual risk that remains.

Table 6.5: Threats introduced by LLM-mediated query and code generation, and the mechanisms that mitigate each. T1 and T2 are bounded by structural design choices; T3 is the weakest of the three boundaries and the principal hardening item identified for future work.

Threat	**Mechanism**	**Residual risk**
T1: SQL injection by string concatenation	Question never spliced into SQL string; intended read-only DB role bounds prompt-shaping attacks	Application-layer read-only enforcement is a known hardening item; risk grows if connection role is not read-only
T2: Realtime query-language injection	LLM emits structured configuration, not query text; deterministic builder applies operator whitelist and string escaping; schema validation against introspected catalogue	Information disclosure within the schema's reach (read-only path)
T3: Arbitrary code execution	Strict structural contract on the chart-code fragment; namespace isolation	Code that respects the contract retains full host capability (filesystem, network, environment); strong sandboxing is future work

The picture the table makes explicit is that the agent's three boundaries have very different strengths. T1 and T2 are bounded by structural design choices that the agent makes once and for all; an attacker has to defeat those structural choices to escalate, and the structural choices are simple and auditable. T3, by contrast, is bounded only by the LLM's prior

and by the structural contract's shape checks, neither of which constrains capability. The honest disclosure is that closing T3 to a comparable degree of safety is the most important hardening item the agent has, and that until it is closed the chart-code stage's trust posture should match the operator's trust in the LLM provider's output, not the trust the operator places in the agent's own code.

6.9 Summary

This chapter has presented the Visualization Chatbot as a co-designed companion to the monitoring substrate of the preceding chapters. The chatbot's job is to convert one English sentence into either a static analytical chart or a live dashboard panel, and to do so well enough that an operator can trust the answer without inspecting the SQL, Flux, or Python that produced it.

The chapter's argument follows the dependency order of the design itself. The substrate maintains two storage tiers because two access patterns—analytical-relational and tag-indexed time-series—cannot be served well by one engine. The chatbot inherits that separation through two architecturally distinct pipelines, and unifies them only at the API surface, where a classifier takes responsibility for choosing the path so that the operator does not have to. Before either pipeline runs, the agent introspects both backends so that schema drift is structurally impossible, and feeds the introspected schemas to the classifier so that its routing decision is conditioned on what the data actually supports. The classifier defaults to the historical pipeline on every error edge, on the grounds that the historical pipeline is self-contained and produces an archivable artefact while the realtime pipeline depends on an external service.

The historical pipeline is the more elaborate of the two, because it must address three sources of risk: schema drift (eliminated by per-request introspection), incorrect SQL (mitigated by retrieval-augmented generation against a corpus refreshed at server startup, and by single-shot regeneration against database errors), and incorrect chart-rendering code (mitigated by a strict structural contract, by single-shot regeneration against execution exceptions, and by isolating the data the LLM sees behind a compact profile rather

than the raw rows). The realtime pipeline is shaped by the asymmetric demands of a live artefact: the LLM emits a structured panel configuration rather than a raw query, the agent compiles the configuration through a deterministic builder with an operator whitelist and string escaping, and the result is pushed to a shared Grafana dashboard whose embeddable URL is the pipeline's output.

The agent's caching strategy is the inverse of what a naive deployment would adopt. The schema, which is small and volatile, is recomputed on every request. The retrieval corpus, which is large and stable across a deployment, is held on disk and refreshed at startup. The Grafana datasource UID, which is operator-controlled, is rediscovered per request rather than cached. Each decision is justified locally by the cost of staleness against the cost of recomputation.

Security is treated as a design property, not a postscript. SQL injection by string concatenation is structurally absent, because the question is never spliced into a SQL string. Realtime-path query-language injection is bounded by the structured-configuration indirection and the deterministic builder's operator whitelist and escaping. Arbitrary code execution is the weakest of the three boundaries: the chart-code fragment runs in an isolated namespace, but namespace isolation is not a sandbox, and a code fragment that respects the structural contract retains full host capability. The honest disclosure is that closing the chart-code boundary with a real sandbox—a subprocess or a WASM runtime—is the most important hardening item the agent has, and is identified for future work.

Bridge to the rest of the thesis. This chapter has covered the design of the agent. The implementation that realises this design—the prompt templates that drive each LLM call, the construction of the retrieval corpus, the precise shape of the Flux builder, the data-collection scripts, and the deployment topology—is the subject of Chapter 7. The empirical evaluation that places the chart-generation core against four reference systems on the public VisEval benchmark, and that reports per-stage latency, token cost, and auto-fix recovery rates from a full benchmark run, is the subject of Chapter 8. The integrated evaluation of the agent end-to-end on real HPC telemetry—measuring user-perceived latency,

integration error rates, and operator usability over a realistic workflow—remains an item for future empirical validation, alongside the chart-code sandboxing and the schema-change responsiveness identified in this chapter.

Chapter 7

Implementation

This chapter describes how the monitoring framework was constructed in practice. While Chapter 5 explained what technologies were selected and why, this chapter focuses on how those technologies were put together, how each component was structured, non-trivial engineering challenges and implementation details that are not captured by architecture diagrams alone. Components that are not complicated to implement given the design choices of Chapter 5 are described briefly; more consideration is given to the parts that required careful engineering or that make this system stands out from conventional monitoring tools.

Compute Node Agent

The Compute Node Agent is the outermost layer of the system that touches the kernel and must operate continuously on every node without disturbing the workloads it is observing. Its implementation is split into two concerns that are kept separately in the code: the Virtual Sensor Module, which owns everything related to data acquisition and local packaging, and the gRPC Client Module, which owns everything related to upstream transmission. This separation means that changes to what is collected such as adding a new sensor type, adjusting sampling granularity would do not require any changes to the transmission path, and vice versa.

Orchestration. The Virtual Sensor Module is not a single collector. It is an orchestrator that coordinates multiple resource-specific collectors, each targeting a different subsys-

tem, combines their outputs into one unified metric package per collection cycle. During each cycle, every collector runs independently and produces a per-PID mapping of its resource measurements. The Virtual Sensor then performs a join-by-PID across all collector outputs, producing a single list of process entries where each entry contains the combined CPU, memory, disk I/O, network and GPU measurements for one process. This joined structure is the standard payload unit that the gRPC Client Module consumes so that downstream components never need to correlate separate per-resource streams themselves.

eBPF-based Collection. The eBPF collectors are the most technically involved part of the agent implementation. Each eBPF program is a small C program that is compiled at load time by the BCC toolkit and injected into the kernel at a specific hook point. For CPU accounting, the program attaches to scheduler tracepoints, specifically the `sched_switch` event, it maintains a per-PID accumulator in a BPF map that records the total time each process spent on-CPU during the sampling window. For memory, the program reads the resident set size (RSS) of each active process by sampling the kernel's memory management structures at the tracepoint. For disk I/O, it attaches to block I/O completion events to count bytes read and written per PID. For network, it attaches to the socket send and receive paths to count transmitted and received bytes per PID.

The user-space Python layer periodically calls into the BPF maps using the BCC Python bindings, reads the accumulated counters for each PID and resets them for the next window. This *read-and-reset* mechanism is what transforms kernel-level event accumulators into per-window rate measurements. For example, dividing the accumulated CPU on-time nanoseconds by the window duration in nanoseconds gives the fraction of the window that the process spent on CPU. The fact that this computation happens in user space, reading from a shared BPF map, rather than requiring the kernel to push data, means the collection cost is dominated by the map read latency rather than by the frequency of kernel events, which is important for workloads that generate millions of I/O or scheduler events per second.

One non-trivial challenge in the eBPF implementation is handling *short-lived processes*, those are processes that start and exit within a single sampling window. The BPF maps

are keyed by PID, and when a process exits its PID becomes available for reuse. If a new process is assigned the same PID within the same window, its counters would be added to the departed process's entry, producing a misleading combined measurement. The implementation addresses this by also tracking the process start time in the BPF map entry and discarding entries where the start time has changed between the previous and current read, these entries are treated as belonging to a new process rather than continuing the previous one.

nvidia-smi Integration. The GPU collector is simpler in structure than the eBPF collectors but requires more consideration in parsing. The agent invokes `nvidia-smi` with a specific set of query fields formatted as CSV output, which is more reliable to parse than the default human-readable table format and avoids the overhead of XML parsing. The fields requested per process are the PID, the GPU index and the used GPU memory in MiB. Device-level metrics such as utilization percentage, temperature, power draw and power limit are queried separately at the device level since they are not per-process attributes.
After parsing, the GPU collector produces two outputs: a per-PID GPU memory mapping that is provided to the *join-by-PID* step in the Virtual Sensor, and a per-device metrics list that is included as a separate section in the final metric package alongside the per-process list. This dual output reflects the difference between GPU and CPU resources: GPU utilization is a property of the device as a whole, not of individual processes, while GPU memory can be attributed per process.

Packaging and Metadata Attachment. After the *join-by-PID* step produces the unified process list, the Virtual Sensor attaches global metadata to form the final package. The metadata includes the node ID, the node's IP address, the hostname, a Unix timestamp marking the start of the collection window, the duration of the window in seconds and the ID of the Collect Agent that this node is configured to stream to. The window duration is included because it is needed by the Data Server to compute rate-based derived metrics such as bytes per second from the raw byte accumulators that the eBPF collectors report. Including it in the package rather than assuming a fixed sampling rate means the Data

Server remains correct even when the sampling interval is changed dynamically via the Coordinator.

Plugin-Based Sensor Configuration. The set of active collectors is controlled by a plugin configuration that the agent loads at startup and updates dynamically when the Coordinator distributes new configuration. Each collector is implemented as an independent plugin that exposes a common interface (`collect() -> per_pid_dict`), then the Virtual Sensor iterates over the currently active plugins to assemble the list of collectors it will run each cycle. Enabling or disabling a sensor type therefore requires only a configuration change, not a code change or service restart. This is the practical realization of the extensibility property described in Chapter 5.

gRPC Client-Streaming Workflow. The gRPC Client Module maintains a single long-lived client-streaming connection to the Collect Agent. At agent startup, it establishes the connection and enters a loop in which each iteration collects one cycle of metrics from the Virtual Sensor, serializes the resulting package into the Protobuf message format defined in the shared `.proto` file and writes the message to the open stream. The stream is kept alive between cycles so there is no connection collapse and re-establishment between packages. If the stream is interrupted due to a network error or a Collect Agent restart, the client module will detect the broken stream through gRPC's built-in error signaling and enters a reconnection loop with backoff before resuming streaming. This reconnection logic is the most important part of the transmission implementation: in a production cluster, temporary network disruptions are common and an agent that gives up after a single failed write would create gaps in the monitoring record that are difficult to diagnose after the fact.

Collect Agent

The Collect Agent holds the most significant position in the middleware layer. It is a gRPC server receiving concurrent streams from every compute node in the cluster, it is also a preprocessing engine transforming raw metric packages into clean, enriched data and a publisher pushing that data into Kafka for downstream consumption. It also main-

tains a separate direct channel to the Application Server for time-sensitive alerts. Each of these responsibilities imposes different implementation constraints and getting them to concur without disrupting with each other is where the critical engineering of this component lies.

Concurrent Stream Handling. The Collect Agent is implemented as an asynchronous gRPC server, using Python's `asyncio`-based gRPC server runtime. The choice of an asynchronous event loop rather than a *thread-per-connection* model is intentional. In a cluster with many compute nodes, a *thread-per-stream* approach would create as many OS threads as there are nodes, each consuming memory and suffering scheduling overhead even during idle periods. An asynchronous server handles all concurrent streams on a small, fixed-size thread pool, multiplexing I/O events across streams without blocking. This makes the server's memory footprint and CPU overhead proportional to actual message arrival rate rather than to the number of connected nodes.

Each Compute Node Agent opens one client-streaming RPC connection and keeps it alive for the duration of the agent's lifetime, pushing metric packages at each collection cycle. On the server side, each stream is handled by an independent coroutine that reads packages from the stream as they arrive, processes them through the preprocessing pipeline and publishes the result to Kafka. The coroutine model means that a slow Compute Node Agent that stops sending for a period but does not block or interfere with the processing of packages arriving from other nodes. Each stream is isolated so a failure or disconnection on one stream is caught at the coroutine level and logged without affecting any other active stream.

The input contract between the Compute Node Agent and the Collect Agent is defined in a shared Protobuf schema file. This schema is the single source of truth for the message format because both sides generate their serialization and deserialization code from it. Any attempt by a Compute Node Agent to send a message that does not follow the schema is caught at the gRPC deserialization layer before the package reaches the preprocessing pipeline, which provides a first line of defense against abnormal data without requiring any validation code in the pipeline itself.

The Fixed Preprocessing Pipeline. Every metric package that successfully deserializes passes through a fixed four-stage preprocessing pipeline before it is published to Kafka. These stages always execute in order and cannot be skipped or reordered by user configuration since they represent the system's quality guarantees that all downstream consumers rely on.

The first stage is **schema validation**. Although the Protobuf deserialization catches irregular packages, it does not catch semantic invalidity. For example, a package with a missing node ID field, a negative collection window duration or a process entry with all-zero metrics that indicates a collection error rather than a idle process. The validation stage applies a set of semantic checks and rejects packages that fail them, logging the rejection reason for operational visibility. Rejected packages are dropped at this point and never reach Kafka, which means downstream consumers are guaranteed to receive only packages that have passed these checks.

The second stage is **filtering and cleaning**. Compute nodes produce a number of inconsistent metrics during the first few collection cycles after startup as the kernel structures are not yet fully populated, BPF maps may contain old entries from a previous agent run and rate-based metrics computed from the first window are unreliable because the window start baseline has not been established. The filtering stage identifies and discards these bootstrap packages based on a configurable startup grace period per node. It also removes process entries whose resource measurements are below a floor threshold, which reduces the volume of near-zero entries that would otherwise accumulate in the database from kernel threads and idle system processes.

The third stage is **enrichment**. Each package that passes filtering is stamped with two additional fields: the Collect Agent's own identifier and the current pipeline version string. The agent ID serves a practical purpose in multi-Collect-Agent deployments where different agents handle different subsets of nodes; therefore, it allows operators and the Support Agent to trace which path a specific metric package traveled through the system. The pipeline version string records which version of the preprocessing logic was active when the package was processed, which is useful for post-hoc analysis when preprocessing rules change and an operator needs to understand why historical data looks different from recent data.

The fourth and final fixed stage is **problem detection**. This stage evaluates each package against a set of administrator-defined resource thresholds. For example, CPU usage above 95% for more than a configured number of consecutive windows or memory usage exceeding a critical level. When a violation is detected, the problem detection stage does two things simultaneously: it allows the package to continue through the rest of the pipeline as normal and it constructs an alert then dispatches it via the direct alert channel to the Application Server. The reason the package is not held back is that the alert and the metric data serve different purposes, the alert is an immediate notification that something needs attention, while the metric data is a persistent record that will be available in the database for subsequent investigation. Separating these two paths ensures neither is delayed by the other.

The Direct Alert Channel. The alert channel is implemented as a unary gRPC call from the Collect Agent to the Application Server, this is a single request carrying the alert payload, with a single acknowledgement response. This is different from the client-streaming model used for metric ingestion. A streaming channel would be appropriate if alerts arrived at high frequency and needed to be batched, but alerts are rare events that require immediate delivery. A unary call delivers the alert as soon as it is detected, with a round-trip that is bounded by network latency alone rather than by any batching or buffering mechanism. The call is made asynchronously from the problem detection coroutine so that it does not block the preprocessing pipeline, when the alert is triggered, the package processing continues in parallel.

The User Plugin Chain. After the four fixed stages complete, each package passes through a configurable plugin chain. The plugin chain is the mechanism through which administrators can inject custom preprocessing logic, which can be filtering rules, derived metric calculations or context-specific enrichment without modifying the Collect Agent's core code.

Each plugin implements a simple interface: it receives a metric package and returns either a transformed package or a signal to drop the package entirely. Plugins are chained in sequence, with the output of each plugin becoming the input of the next. This struc-

ture means that complex preprocessing behavior can be built from a series of simple, independent transformations rather than from a single monolithic function. The current plugin chain is stored as an ordered list in the agent's in-memory configuration, which the Coordinator can update at runtime.

Plugin Chain Hot Reload. The hot-reload mechanism is the most valuable feature of the plugin architecture and also the one that required the most careful implementation. The challenge is that the plugin chain is accessed by the stream-handling coroutines on every incoming package, which can be dozens of concurrent accesses at any given moment, while the Coordinator may push a configuration update that requires the chain to be replaced. A simple implementation that reassigns the chain reference mid-processing would risk having one coroutine begin processing a package through the old chain while another begins through the new one, producing inconsistent results within the same collection cycle.
This implementation using an immutable pipeline object pattern. The active plugin chain is stored as an immutable object reference. When a configuration update arrives from the Coordinator, a new pipeline object is constructed from the updated configuration and swapped in as the active reference. Coroutines that have already acquired a reference to the old pipeline continue using it until their current package is fully processed, then pick up the new pipeline on their next iteration. This approach eliminates any locking overhead on the critical path because reading the active pipeline reference is a single pointer read, while ensuring that no package is ever processed through a partially constructed pipeline.

Publishing to Kafka. Once a package has passed through all preprocessing stages, the Collect Agent serializes it to JSON and publishes it to the `rawData` Kafka topic. The publish operation is performed asynchronously: the stream-handling coroutine enqueues the serialized package into a buffer and a dedicated publisher coroutine drains the buffer and sends messages to Kafka in batches. This producer-consumer separation within the Collect Agent itself serves an important purpose: it decouples the ingestion rate from the Kafka write rate, so that a temporary slowdown in Kafka delivery due to broker load

or network congestion would not create backpressure on the gRPC streams and does not cause Compute Node Agents to wait for acknowledgement.

The Kafka producer is configured with the node ID as the message key, which determines the partition to which each message is routed. Since Kafka preserves ordering within a partition, this partitioning strategy guarantees that all metric packages from a given node arrive at the consumer in the order they were published, the property that the Data Server relies on when computing rate-based derived metrics that span collection windows. The producer does not wait for broker acknowledgement before considering a message delivered. This is an intentional trade-off for a monitoring system where data arrives continuously at short intervals and the cost of losing a random package due to a transient broker issue is far lower than the latency cost of synchronous acknowledgement on every message. An irregular gap in a time-series is acceptable and visible; added latency on every data point would degrade the real-time responsiveness of the entire monitoring pipeline.

Message Broker

The Message Broker implementation is simple given the technology selection made in Chapter 5, Apache Kafka's deployment model is well-documented and its configuration for a monitoring pipeline of this scale does not involve unusual engineering decisions. This section therefore focuses on the specific configuration choices made for this system and the reasoning behind them, rather than describing Kafka's general operation.

Topic Design. The pipeline uses two dedicated Kafka topics with separated responsibilities. The `rawData` topic serves as the durable buffer between the Collect Agent and the Data Server. The Collect Agent is the only producer of this topic and the Data Server is its only consumer. The purpose of this separation is intended: if the Data Server is temporarily slow or unavailable due to a database write bottleneck or a service restart then messages accumulate in the topic rather than being lost or causing backpressure on the Collect Agent. The Collect Agent continues publishing at its natural rate regardless of downstream conditions and the Data Server catches up when it recovers. The `processedData` topic serves a different purpose, it is produced by the Data Server after it has ingested and processed each batch, and is available for any additional con-

sumer that needs instant data. For example, a future real-time anomaly detection service. No such consumer exists in the current implementation, but the topic is maintained as a designed extension point that requires no changes to existing components.

Partitioning Strategy. Both topics are partitioned by node ID, meaning all metric packages from a given compute node are routed to the same partition. This decision is driven by the Data Server's need for per-node ordering, when it computes rate-based derived metrics such as bytes per second from consecutive collection windows, the windows must arrive in the order they were produced. Kafka guarantees ordering within a partition but not across partitions, so assigning each node a stable partition via consistent hashing on the node ID key would provide this guarantee without requiring any ordering logic in the Data Server itself. The number of partitions is set to match the number of compute nodes in the cluster, which distributes the ingestion load across the broker while keeping the per-node order.

Retention and Fault Tolerance. Messages in the `rawData` topic are retained for 24 hours. This retention window serves as a recovery buffer, if the Data Server experiences a prolonged outage, it can replay up to 24 hours of missed messages from the topic rather than losing that data permanently. The retention period is short because the Data Server writes incoming data to InfluxDB in real time and the topic is not intended as a long-term archive, InfluxDB and TimescaleDB serve that role. Retaining messages longer than necessary would consume broker disk space without providing additional value. Each topic is replicated across the broker cluster with a replication factor of two, ensuring that a single broker failure does not result in data loss or topic unavailability.

Data Server and Hybrid Storage

The Data Server and the Hybrid Storage tier together form the persistence backbone of the system. The Data Server is responsible for consuming metric packages from Kafka and writing them into InfluxDB in a structured, queryable form. The Hybrid Storage tier consists of two databases, which are InfluxDB for short-term high-resolution data and TimescaleDB for long-term aggregated data, they are bridged by a scheduled ETL

pipeline. Of all the components in the Application and Data Services Layer, this is the part where the implementation required the most careful engineering decisions, particularly around the database schema design and the aggregation logic in the ETL pipeline.

The Kafka Consumer and Measurement Decomposition. The Data Server is implemented as a continuously running Python script that subscribes to the `rawData` Kafka topic using a named consumer group. The consumer group mechanism ensures that if multiple instances of the Data Server are ever deployed for horizontal scaling, Kafka automatically distributes partitions across them without any coordination logic in the application code. In the current single-instance deployment, the consumer group still provides the benefit of offset tracking where the consumer's position in the topic is committed to Kafka after each successfully processed message, so a Data Server restart automatically resumes from the last committed offset rather than reprocessing all messages or skipping unprocessed ones.

For each incoming message, the Data Server splits the unified metric package into three distinct InfluxDB measurements before writing. This decomposition step is the central design decision of the Data Server implementation. The alternative which is writing each package as a single flat record would have produced a measurement with an extremely wide schema containing node-level, GPU-level and process-level fields together, which would make tag-based filtering and time-range queries significantly less efficient. By splitting into three measurements, each with its own tag set and field set, the schema aligns with InfluxDB's optimized access patterns: queries that ask about node health never touch the process measurement and queries about per-user resource consumption never touch the GPU measurement.

The `node_status` measurement stores one data point per compute node per collection cycle, tagged by node ID and Collect Agent ID, with fields for CPU usage percentage, memory usage percentage, total memory bytes, used memory bytes, collection window duration and reception timestamp. The `gpu_status` measurement stores one data point per GPU per node per cycle, tagged by node ID and GPU index, with fields for utilization percentage, temperature, power draw, power limit and used and total GPU memory. The `process_status` measurement stores one data point per user-application group per node per cycle, tagged by node ID, user ID and command name, with fields for CPU

on-time in nanoseconds, disk read and write bytes, network receive and transmit bytes, GPU memory used, average resident set size and process count.

Process Grouping and Cardinality Control. The most non-trivial part of the Data Server implementation is the process grouping logic applied before writing `process_status` points. A busy compute node may have hundreds of active processes at any given moment, and writing one InfluxDB point per PID per collection cycle would generate an enormous number of series; therefore, InfluxDB's performance degrades significantly as the number of unique tag value combinations grows, because each series requires its own index entry and write path. Writing one point per PID would create a cardinality proportional to the number of processes times the number of nodes times the number of collection cycles, which would become out of control within days on a busy cluster.

The grouping strategy addresses this by aggregating processes along two dimensions before writing. First, the user ID is normalized: root (UID 0) is mapped to a canonical root identifier, system processes (UID 1–999) are collapsed into a single system category and regular user IDs are retained. Second, the process command name is normalized by collapsing known kernel thread prefixes, such as `kworker`, `kthread` and similar patterns into canonical category names, reducing the number of distinct command values. All processes sharing the same normalized user ID and normalized command name are then grouped together, with their resource metrics summed within the group. This produces one InfluxDB point per user-application category per node per cycle, which is both the granularity that administrators actually care about for resource attribution and a cardinality that remains manageable as the cluster scales.

Batched Writes to InfluxDB. All data points produced from a single Kafka message, across all three measurements are assembled into a single batch and written to InfluxDB in one HTTP request. This batching strategy reduces write latency and network overhead compared to issuing one write request per data point: a typical message from a node with several active users and one or two GPUs produces between ten and thirty data points and batching these into a single request reduces the number of TCP round-trips by the same factor. The timestamp used for all points in a batch is the server-side reception time at second precision, which ensures that all three measurements from the same collection cycle are aligned in the database and can be correlated by timestamp in downstream queries.

The Hourly ETL Pipeline. The ETL pipeline is a scheduled Python service that runs five minutes past each hour, queries InfluxDB for the data accumulated in the preceding hour, aggregates it and writes the results into TimescaleDB. The five-minute offset from the hour boundary is deliberate as it gives the Data Server time to process and flush any Kafka messages that were produced just before the hour boundary but may not have been written to InfluxDB yet, ensuring the aggregation window is complete before it is read.
The pipeline produces two output tables in TimescaleDB. The `node_status_hourly` table stores one row per node per hour containing the average and maximum CPU usage percentage, average memory usage percentage, maximum memory used in bytes, average GPU utilization and temperature, total GPU power consumption and total disk and network I/O bytes accumulated during the hour. The `user_app_hourly` table stores one row per unique combination of node, user ID and application command name per hour, containing total CPU time consumed in seconds, average and maximum memory usage, total I/O and network bytes and the peak process count observed during the hour. This second table is the primary data source for the Support Agent's historical pipeline when answering user-level resource attribution questions. The aggregation queries are written in InfluxDB's Flux query language and executed directly against the database rather than pulling raw data into Python for aggregation. This is an important implementation decision because pulling all raw data points for an hour from InfluxDB into the Python process would transfer hundreds of thousands of rows over the network, require significant memory to hold them, and then perform aggregation in Python that InfluxDB's native engine could perform far more efficiently on indexed data. By pushing the aggregation into Flux queries using `aggregateWindow` with one-hour windows, the pipeline transfers only the aggregated results, one row per node per measurement type which are typically two to three orders of magnitude smaller than the raw data.
For cumulative metrics such as total CPU time and total I/O bytes, the Flux `spread()` function is used rather than `sum()` or `mean()`. The reason is that these metrics are stored as running totals within each collection window and the eBPF collectors report the total bytes read since the start of the window, not the delta from the previous window. To compute the amount consumed during an entire hour, the correct operation is to subtract the minimum observed value from the maximum observed value within the hour

which is exactly what `spread()` computes. Using `sum()` would incorrectly add up the per-window totals, double-counting the cumulative effect. The ETL pipeline writes to TimescaleDB using `ON CONFLICT DO NOTHING` semantics on the primary key, which combines the bucket timestamp, node ID and application identifiers. This idempotency guarantee means the pipeline can be safely rerun after a failure without producing duplicate rows in the historical store, the second run simply finds that all rows it would insert already exist and skips them. Combined with the fact that the pipeline reads from InfluxDB which retains data for seven days, this means any hourly aggregation that fails can be retried up to seven days later without any data loss.

Application Server

The Application Server is the component through which all users, including administrators and regular users alike interact with the monitoring system. It is implemented as a Next.js full-stack web application, which means the frontend and backend API routes coexist within a single deployable unit. This section focuses on the implementation decisions that are specific to this system's requirements rather than on general Next.js patterns.

Real-Time Dashboard Integration. The real-time monitoring dashboard does not query InfluxDB directly from the Next.js backend. Instead, it embeds Grafana panels using `iframe`-based embed URLs that Grafana generates for each pre-configured dashboard panel. This approach grants the responsibility of live data visualization, which polls InfluxDB at refresh intervals, rendering time-series charts, handling panel interactivity entirely to Grafana, which is purpose-built for exactly this task. The Next.js frontend simply renders the embed URLs inside `iframe` containers, which means the real-time dashboard requires no database query logic on the application server side at all. The practical benefit is that administrators gain access to Grafana's full panel feature set such as zoom, time range selection, threshold annotations and alert indicators without any reimplementation in the web application.

Historical Analytics via TimescaleDB Queries. For historical data, the Application Server's

backend API routes query TimescaleDB directly using standard PostgreSQL client libraries. The separation between real-time and historical data access is intentional and follows the same two-tier logic established throughout the system: InfluxDB serves real-time access through Grafana, while TimescaleDB serves historical analysis through the application's own backend. The backend constructs parameterized SQL queries against the `node_status_hourly` and `user_app_hourly` tables, applying time range filters, node filters and user filters based on the request parameters passed from the frontend. Parameterized queries are used throughout to prevent SQL injection and results are returned to the frontend as JSON for rendering in the analytics interface.

Cluster Configuration and Coordinator Integration. The cluster configuration page is one of the most significant parts of the web application. It provides administrators with a form-based interface for modifying system parameters such as sampling intervals, enabled sensor plugins, preprocessing thresholds and alert bounds. When an administrator submits a configuration change, the frontend sends the updated values to a Next.js API route, which validates the input and then connects directly to the Coordinator's etcd instance to write the new configuration values to the appropriate keys. The etcd write triggers the watch notifications that propagate the change to the Compute Node Agents and the Collect Agent, as described in Chapter 5. From the administrator's perspective, the change takes effect within seconds of submission without any visible system disruption, the web application simply reflects that the configuration was saved successfully.
The direct etcd connection from the Next.js backend is worth noting as an implementation decision. An alternative design would have the Application Server communicate with an intermediate configuration service that owns the etcd connection, keeping the web application decoupled from the coordination technology. However, for the current scale of the system, a single laboratory cluster with a small number of administrators, the additional indirection layer provides no practical benefit and adds deployment complexity. The direct connection is simpler, and the etcd client library provides sufficient abstraction over the underlying protocol that replacing etcd in the future would require changes only in the configuration page's backend route.
AI Support Interface. The chatbot interface is implemented as a dedicated page within

the Next.js application, consistent with the other pages but functionally distinct. The frontend provides a text input field where users type natural language questions, a submit button and a response area where the result is rendered. When a question is submitted, the frontend sends it to a Next.js API route that acts as a proxy to the Support Agent Server microservice. The proxy pattern is used rather than calling the Support Agent directly from the browser for two reasons: it keeps the Support Agent's internal endpoint private and not directly accessible from the client, and it allows the Application Server to attach authentication context to the request before forwarding it.
The Support Agent Server returns a unified JSON response containing either an inline SVG string for historical questions or a Grafana embed URL for real-time questions, distinguished by a discriminator field. The frontend inspects this field and renders the response appropriately: SVG responses are injected directly into the DOM as inline vector graphics, while embed URL responses are rendered as `iframes`, this is exactly the same embedding mechanism used by the real-time dashboard. This means the chatbot response visualization reuses the same rendering path as the rest of the application rather than introducing a new display mechanism.

Alert Notifications. Alerts detected by the Collect Agent's problem detection stage are delivered to the Application Server via direct gRPC call, as described in the Collect Agent section. On the Application Server side, incoming alert calls are received by a gRPC server endpoint running within the Next.js backend process. Each alert is stored in an in-memory notification queue and surfaced to logged-in administrators as browser notifications through a polling mechanism: the frontend polls the notification endpoint at a short interval and displays any new alerts as toast notifications in the interface. The current implementation does not persist alerts to a database or support email delivery, these are identified as extensions for future work.

Authentication. The current implementation supports a single administrator account. Authentication is handled through a session-based mechanism built into Next.js, where a successful login creates a server-side session that is verified on every subsequent API route call. All data-fetching routes, the configuration endpoint and the chatbot proxy re-

quire a valid session, unauthenticated requests receive a 401 response. Role-based access control distinguishing administrator and regular user roles is identified as a near-term extension as the system moves toward multi-user deployment.

Support Agent Server

The Support Agent Server is deployed as an independent microservice, separate from the Application Server and the rest of the monitoring pipeline. It exposes a single HTTP API endpoint that accepts a natural language question and returns a unified JSON response containing either a static SVG chart or a live Grafana panel embed URL. The internal architecture of the agent, which is the dual-pipeline design, the schema introspection mechanism, the LLM classifier, the historical and real-time pipelines and the security boundaries — is described in full in Chapter 6. This section focuses specifically on the implementation aspects that are relevant to how the agent is deployed and integrated with the rest of the system, rather than repeating the pipeline design.

Deployment as a Microservice. The agent is packaged as a standalone Python application running in a Docker container. The containerized deployment isolates the agent's Python environment and dependencies, including the LLM client library, the ChromaDB vector store, the database client libraries for both InfluxDB and TimescaleDB and the matplotlib rendering stack from the rest of the system. This isolation is practically important because the agent's dependency set is heavier than that of any other component in the system, and keeping it in a dedicated container prevents dependency conflicts with the Application Server's Node.js environment. The container exposes a single HTTP port that the Application Server's proxy route connects to.

RAG Corpus Initialization. When the agent container starts, it performs a corpus initialization step before accepting any requests. This step connects to TimescaleDB, introspects its live schema and trains the ChromaDB vector store with DDL documentation, table descriptions and question-SQL exemplars derived from the live schema. This startup-time training ensures that the retrieval corpus is aligned with the actual database schema at the time of deployment. The training step takes several seconds to complete,

during which the agent does not accept incoming requests. Once training is complete the agent signals readiness and the Application Server's proxy begins forwarding requests to it.

Connections to Both Storage Tiers. The agent maintains persistent connection pools to both InfluxDB and TimescaleDB. The InfluxDB connection is used in two contexts: during schema introspection on every request, where it queries the measurement, tag and field metadata, and during the real-time pipeline, where it provides the datasource that Grafana panels connect to. The TimescaleDB connection is used during schema introspection for the historical tier and during the historical pipeline's SQL execution step. Maintaining persistent connection pools rather than opening a new connection per request avoids the TCP and authentication handshake overhead that would otherwise be incurred on every question, which is important given that the agent already pays the latency cost of multiple sequential LLM API calls per request.

Grafana Integration. The real-time pipeline pushes panel definitions to a shared Grafana instance via the Grafana HTTP API. The agent is configured with the Grafana base URL and an API key at startup via environment variables. On each real-time request, the agent queries Grafana's datasource registry to discover the current UID of the InfluxDB datasource, constructs the panel definition and posts it to the shared dashboard. The resulting embed URL in Grafana's `/d-solo/` format that renders a single panel without the surrounding dashboard chrome is included in the JSON response returned to the Application Server, which passes it through to the frontend for `iframe` rendering.

Environment Configuration. All external connection parameters, including the InfluxDB URL and token, the TimescaleDB connection string, the OpenAI API key for GPT-4o-mini and the Grafana base URL and API key are injected as environment variables at container startup rather than hardcoded in the application. This follows standard twelve-factor application practice and means the same container image can be deployed against different infrastructure configurations. For example, a development environment pointing to local database instances and a production environment pointing to dedicated

servers without rebuilding the image.

Chapter 8

Evaluation

This chapter evaluates two preconditions of the proposed architecture rather than the integrated system as a whole. The first is that the monitoring substrate must be lightweight enough to operate continuously on compute nodes and collectors without significantly interfering with production workloads. The second is that the historical visualization core must be competent at translating natural language questions into correct and readable charts. The measurements presented below support both claims. However, they do not measure classifier routing accuracy, real-time panel correctness or compounded end-to-end behavior on live HPC telemetry, these remain as items for future empirical validation.

8.1 Monitoring Substrate Overhead

A monitoring framework that consumes a significant fraction of the resources it is trying to observe would go against its original purpose. The overhead evaluation therefore focuses on illustrating the resource footprint of the Compute Node Agent and the Collect Agent under four controlled scenarios, each isolating a different potential cost driver. All configurations were run five times for thirty minutes with the first five minutes discarded, so the reported figures reflect post-warm-up steady-state behavior. The workstation hosts used for the collector and loaded compute-node roles were `ws-17`, `ws-18` and `ws-20` (Core i7-9700, 3.0 GHz, 32 GB RAM, Debian 12 / Linux 6.1) and `ws-19` (Core i7-9700, 3.0 GHz, 32 GB RAM, Ubuntu 24.04 / Linux 6.17). Cluster-scale, multi-tenant and read-side-contention measurements remain outside the scope of this evaluation.

Scenario 1 - Metric Collection Scope. The first scenario measures the incremental cost of enabling additional sensor types on the Compute Node Agent, comparing a partial configuration collecting only CPU and disk metrics against a full configuration collecting all available sensor types including memory, network and GPU. The results across the three measured dimensions (detailed charts are shown in Figure 8.1, including CPU usage, memory usage and memory utilization show a consistent outcome.

On the CPU dimension, both the host node and the CN-Agent process remain at effectively the same level regardless of collection scope: the host stays at $1.00\,\%$ and the agent process moves only from $0.994\,\%$ to $1.05\,\%$. The difference of 0.056 percentage points is within measurement noise and should not be considered as a meaningful increase. This is the expected behavior of the eBPF-based design, when additional sensor types attach new eBPF programs to additional kernel hooks, but the user-space collection loop that reads from BPF maps runs at the same frequency regardless of how many maps it reads, so adding sensors does not significantly increase the CPU cost of the agent process.

On the memory dimension, the picture is similarly stable. The host node's total memory usage is identical at $2.80\,$GiB in both configurations, indicating that enabling additional sensors has no effect on the node's overall memory footprint. The CN-Agent process itself grows from 0.170 to $0.210\,$GiB, the increase of $40\,$MiB corresponds to the additional BPF maps allocated in kernel space for the new sensor programs and the slightly larger per-cycle payload that the agent packages before transmission. While the relative increase is approximately $23\,\%$ in agent process memory, the absolute increase of $40\,$MiB is negligible in the context of the $32\,$GiB host as the agent process with full collection enabled still consumes less than $0.66\,\%$ of total host memory.

The memory utilization percentage strengthens this reading: the host node moves slightly from $7.90\,\%$ to $8.00\,\%$, a difference of 0.10 percentage points that is practically insignificant. The CN-Agent process utilization increases from $0.400\,\%$ to $0.600\,\%$, which again reflects the additional BPF map memory but remains well below any threshold of concern.

The key takeaway from Scenario 1 is that the plugin-based sensor architecture described

in Chapter 5 achieves its extensibility goal without a proportional cost increase. Enabling all available sensor types costs approximately 40 MiB of additional agent memory and an insignificant increase in CPU, which means administrators can enable the full monitoring capability of the system without making a resource trade-off on the monitored node.

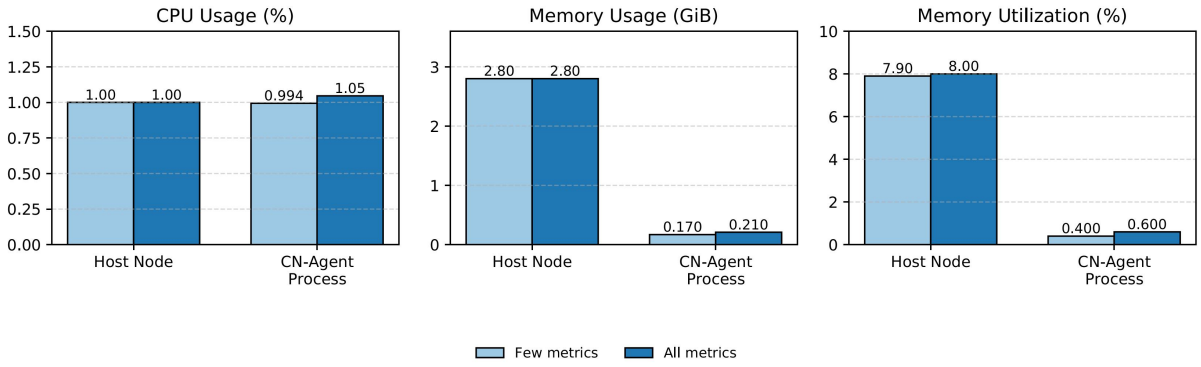


Figure 8.1: Scenario 1 - Idle CN-Agent footprint under partial collection (CPU and disk) and full collection.

Scenario 2 - Host-Process Density. The second scenario measures how the overhead of the Compute Node Agent and the Collect Agent scales when the number of active processes on the monitored host grows. The simulated workload varies the number of extra processes spawned on the CN host across four levels: 0, 1 000, 10 000 and 30 000. All measurements are taken under full collection and full preprocessing, so any cost observed here is attributable to process-count growth rather than configuration differences (shown in Figure 8.2). Four components are tracked: the CA Host and CN Host at the machine level, and the CA Process and CN Process at the individual-process level.

On the CPU dimension, the host-level results reflect the expected asymmetry between the two machines. The CN Host, which is generating the extra processes, sees its CPU usage rise greatly from $1.59\,\%$ at zero extra processes to $8.90\,\%$ at 30 000 extra processes, as spawning and scheduling thousands of short-lived processes is CPU-intensive work for the operating system. By contrast, the CA Host remains flat across the same range, moving only from $1.59\,\%$ to $2.26\,\%$, a moderate increase that reflects the higher message-processing rate as the CN-Agent sends larger packages containing more process entries per collection cycle. At the process level, the CN Process CPU rises from $1.19\,\%$ at baseline to $4.54\,\%$ at 30 000 extra processes. This increase is expected: the eBPF *read-*

and-reset loop must iterate over a larger set of BPF map entries per cycle, as each active process has its own accumulator stored in the map and the user-space Python layer must perform the *join-by-PID* across a larger set of per-collector outputs before packaging the result. However, the CA Process CPU stays nearly constant throughout the sweep, measured at $0.298\,\%$, $0.298\,\%$, $0.300\,\%$ and $0.397\,\%$ respectively. This stable is a direct consequence of the asynchronous coroutine architecture of the Collect Agent described in Chapter 5: the agent's overhead is proportional to the rate of arriving Kafka messages, not to the number of processes within each message, so the preprocessing pipeline handles a denser package in the same number of pipeline stages.

On the memory dimension, the results keep the same pattern. The CN Host memory usage is effectively stable at the host level, holding between 6.37 and 6.44 GiB across all four process-count levels, which indicates that spawning the extra workload processes does not change the node's overall memory consumption at the measurement used. The CA Host memory likewise remains flat at around 5.00 to 5.20 GiB, consistent with a server whose resource usage is determined by its fixed software stack rather than by incoming data volume. At the process level, the CN Process memory stays near 0.205 to 0.208 GiB throughout the entire sweep, rising by only 3 MiB between the 0 and 30 000 process configurations. This stability is explained by the eBPF design: BPF maps are pre-allocated at load time with a fixed maximum-entry capacity configured per sensor, so the kernel-side memory footprint of the monitoring maps does not grow proportionally with the number of live processes, only the number of populated entries within those maps changes. The CA Process memory is similarly bounded, remaining at 0.068 to 0.072 GiB, because the Collect Agent's state per stream is bounded by the size of one metric package at a time; larger packages pass through the pipeline without accumulating in memory.

The memory utilization percentages are consistent with the absolute figures: CN Host utilization is stable at $19.9\,\%$ to $20.1\,\%$, CA Host utilization holds at $16.2\,\%$ to $20.6\,\%$ with the highest value appearing at the 30 000 process level, and both process-level utilization values remain at or below $0.671\,\%$ across all configurations.

The key implication from Scenario 2 is that the host-process density of the monitored node does not cause unbounded growth in either the CN-Agent or Collect Agent process

overhead. The CN-Agent CPU does increase with process count, as it must perform a larger *join-by-PID* each cycle and read from more populated BPF maps, but the growth is bounded and the absolute values remain acceptable even at 30 000 extra processes. More importantly, the Collect Agent process is entirely insulated from this growth: its overhead is determined by the arrival rate of metric packages rather than their internal cardinality, confirming that the pipeline-based preprocessing architecture achieves the decoupling between monitoring scale and middleware cost that was one of the design goals of the system.

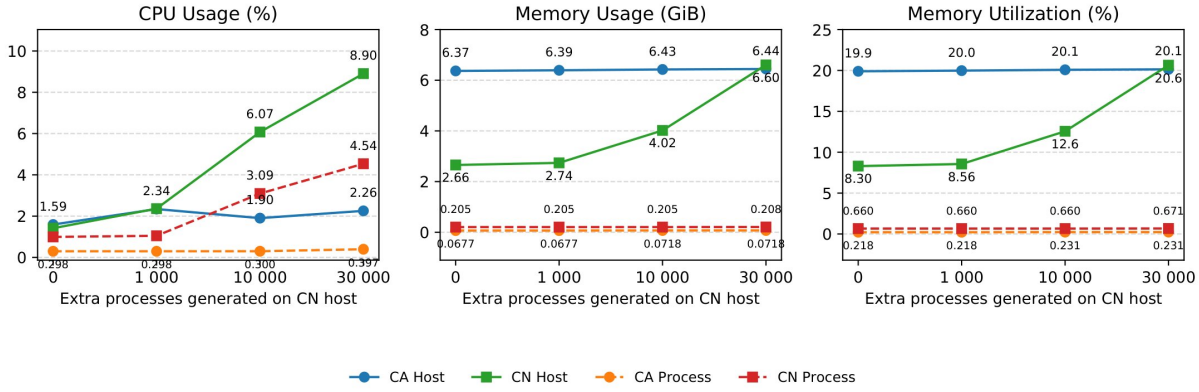


Figure 8.2: Scenario 2 - Host-process-density sweep on the CN host with full collection and full preprocessing.

Scenario 3 - Collect Agent Node Scalability. The third scenario isolates the cost that each additional connected Compute Node Agent imposes on the Collect Agent, scaling the number of simultaneously attached CN-Agents from one to four while keeping full collection and full preprocessing active throughout (Figure 8.3. Unlike Scenario 2, which varied the internal complexity of each metric package, this scenario varies the number of concurrent input streams that the Collect Agent must maintain at the same time. Only the Collect Agent side is instrumented here, with the CA Host representing the machine running the Collect Agent and the CA Process representing the Collect Agent process itself.

On the CPU dimension, the CA Host rises gradually from $1.50\,\%$ at one connected CN-Agent to $1.78\,\%$ at four, a total increase of 0.28 percentage points across the full range.

The CA Process contributes only $0.266\,\%$ to $0.297\,\%$, with a net gain of 0.031 percentage points over the four-agent sweep. Both of these increments are slightly small. The reason is that the Collect Agent is implemented as an asynchronous gRPC server using Python's asyncio runtime, as described in Chapter 5. Each connected CN-Agent is handled by an independent coroutine that reads packages from its stream as they arrive and passes them through the preprocessing pipeline. Because the event loop multiplexes all coroutines on a small, fixed-size thread pool rather than spawning a new OS thread per connection, the cost of opening a new stream is a coroutine allocation rather than a thread creation. Adding a fourth stream therefore contributes only the CPU cycles needed to process that stream's incoming packages at the prevailing collection rate and no additional scheduling, synchronization or context-switching overhead beyond that.
On the memory dimension, the CA Host usage rises from $5.00\,$GiB at one CN-Agent to $5.20\,$GiB at four, a host-level increase of $200\,$MiB that reflects the broader system state of the machine rather than the Collect Agent process itself. At the process level, the CA Process memory grows from $0.611\,$GiB to $0.615\,$GiB, an increase of only approximately $4\,$MiB over the full four-agent range. This flat is explained by the per-stream memory model of the Collect Agent: each coroutine holds at most one metric package in flight at any given time, passing it through the fixed preprocessing stages before either publishing it to Kafka or discarding it. There is no per-stream buffer that accumulates messages in memory and the immutable pipeline object shared across all coroutines is allocated once rather than duplicated per stream. As a result, each additional connected CN-Agent contributes only the memory needed for one coroutine's stack and one in-flight package. The memory utilization percentages confirm this reading: CA Host utilization moves from $16.2\,\%$ to $16.8\,\%$ and CA Process utilization remains nearly unchanged at $1.96\,\%$ across all four configurations, rising by only 0.01 percentage points at the four-agent level. These values are consistent across all three metrics and leave no ambiguity in their interpretation.

This means that the Collect Agent's resource cost scales linearly with the number of connected compute nodes. The total CA Process memory growth of $4\,$MiB and CPU growth of 0.031 percentage points across a four times increase in connected agents demonstrates

that the asyncio coroutine architecture achieves the design goal stated in Chapter 5: the agent's memory footprint and CPU overhead are proportional to actual message arrival rate rather than to the number of connected nodes. While the current evaluation covers only up to four connected CN-Agents due to the available testbed hardware, the near-zero marginal cost per additional stream suggests that the architecture is well-positioned to scale to the larger node counts encountered in production HPC deployments.

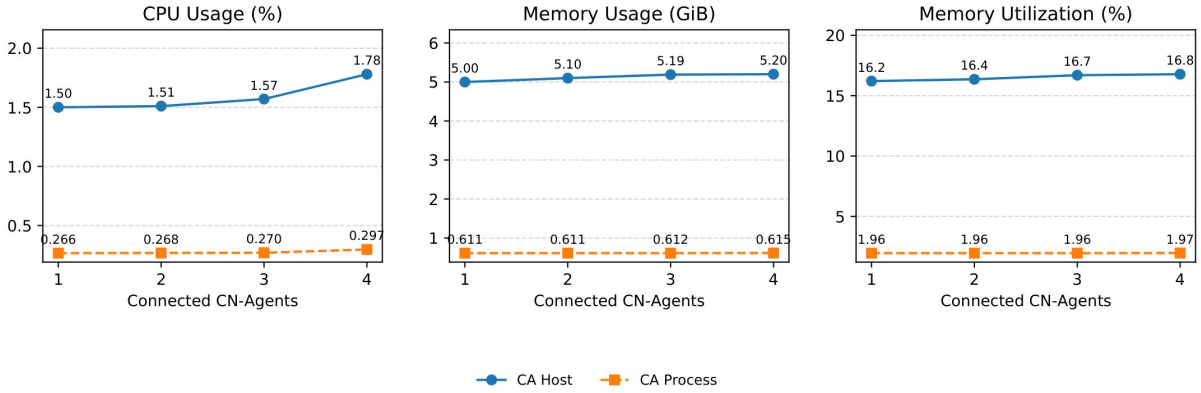


Figure 8.3: Scenario 3 - Collector scaling from one to four attached CN-Agents with full collection and preprocessing.

Scenario 4 - Preprocessing Pipeline Depth. The final scenario examines the cost of activating the full preprocessing pipeline in the Collect Agent, comparing a minimal one-plugin baseline that runs only the schema validation stage against the full five-stage pipeline that adds filtering and cleaning, enrichment, problem detection and the user-configurable plugin chain on top. The scenario is run with four CN-Agents attached, matching the maximum node count from Scenario 3, so that the pipeline cost is measured under a realistic multi-stream load rather than a single-stream idle state. Unlike the previous three scenarios, which found the dominant cost driver to be either collection scope, process density or stream count, this scenario identifies preprocessing depth as the single most significant lever in the entire substrate evaluation (Figure 8.4.

On the CPU dimension, the CA Host rises from $1.37\,\%$ under the one-plugin baseline to $1.78\,\%$ under the full pipeline, an increase of 0.41 percentage points. The CA Process itself contributes a rise from $0.0468\,\%$ to $0.237\,\%$, a relative increase of approximately $5\times$ at the process level. While proportionally significant, the absolute value of $0.237\,\%$ remains

low in terms of its impact on the node as a whole. The increase is expected: each additional pipeline stage executes synchronously within the stream-handling coroutine for every incoming package, so activating four more stages means the coroutine performs four additional passes over each package before publishing it to Kafka. The problem detection stage in particular involves evaluating a set of threshold conditions against all per-process entries in the package, whose computational cost scales with the number of processes in each package rather than being a fixed constant. The alert dispatch path, which fires an asynchronous unary gRPC call to the Application Server upon a threshold violation, also adds discontinuous CPU activity that is absent in the one-plugin baseline. The memory dimension is where the most consequential difference appears. The CA Host memory is identical in both configurations at 5.20 GiB, confirming that the host-level footprint is determined by the machine's steady-state software environment and is not sensitive to pipeline depth. The CA Process memory, however, jumps from 0.0570 GiB (58 MiB) under the one-plugin baseline to 0.615 GiB (630 MiB) under the full pipeline, an increase of approximately 572 MiB and an order-of-magnitude growth at the process level. This is the largest change observed across all four scenarios and it ensures a careful mechanistic explanation. The origin of this increase lies in the pipeline's sequential, in-memory execution model: each stage in the pipeline receives the full metric package from the previous stage, may transform and passes the result forward. When only schema validation runs, the package is held in memory only for the duration of that single check before being published to Kafka and released. When all five stages are active, the package must be retained across the full stage sequence, and each intermediate transformation, particularly the enrichment stage, which attaches new metadata fields, and the problem detection stage, which must maintain per-node threshold violation counters across consecutive collection windows would introduce additional live objects that coexist in the process heap while the pipeline executes. With four concurrent streams each processing packages at their collection rate, the number of such in-flight objects at any moment grows proportionally. Despite this growth, the 630 MiB figure still represents less than $2\,\%$ of the 32 GiB host memory, keeping the Collect Agent well within an acceptable resource budget.

The memory utilization percentages reflect the same finding directly: CA Process uti-

lization rises from $0.170\,\%$ to $1.92\,\%$, while the CA Host utilization remains unchanged at $16.7\,\%$ versus $16.8\,\%$, confirming that the process-level cost does not translate into a host-level burden.

From Scenario 4, it is observed that preprocessing depth is the primary cost in the monitoring substrate, more significant than collection scope (S1), process density (S2) or node count (S3). Administrators who need to operate under strict memory constraints should be aware that enabling the full pipeline at scale will be the primary driver of Collect Agent memory consumption. At the same time, the full pipeline's $630\,\text{MiB}$ ceiling remains well below a meaningful threshold on any server-class machine, and the functionality it provides including guaranteed data quality, metadata traceability, real-time threshold alerting and extensible custom logic via the user plugin chain represents the complete set of data quality guarantees that all downstream components of the system rely on. The cost is therefore justified by what it delivers so that operators who do not need the full pipeline can disable individual stages through the Coordinator without restarting the service.

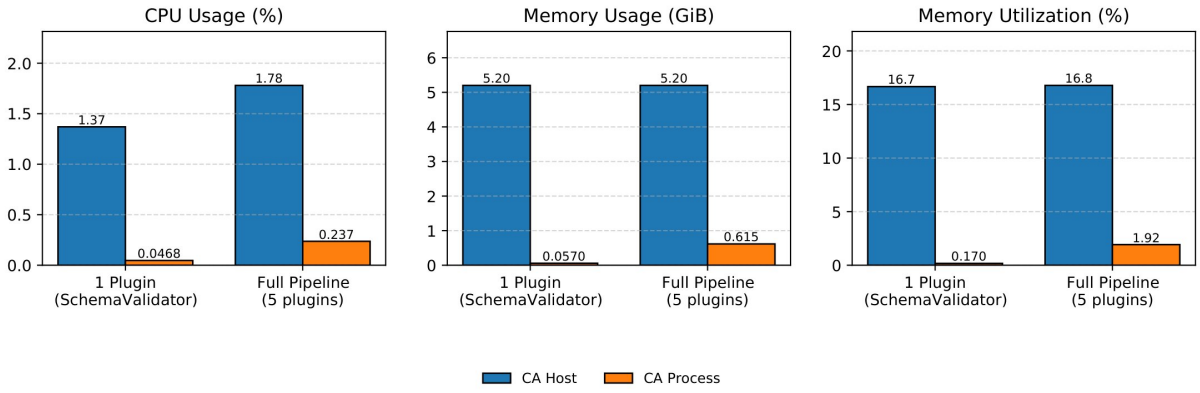


Figure 8.4: Scenario 4 - Preprocessing-depth impact with four CN-Agents attached.

Together, the four scenarios show that the substrate's resource footprint is well-bounded under the measured configurations. The Compute Node Agent's memory is stable regardless of host process count, full sensor collection adds negligible incremental cost and the Collect Agent scales near-flatly with the number of attached nodes. The largest cost in the system is the Collect Agent's preprocessing pipeline, which is both predictable and acceptable. Larger deployments, multi-tenant scenarios, and read-side contention under

concurrent chatbot queries remain as items for future measurement.

8.2 Chart-Generation Core on VisEval

The historical chart-generation module is evaluated on the VisEval benchmark [45], using its single-table split of 688 samples $\times$ 3 paraphrases, yielding 2,064 queries in total. Four reference systems: CoML4VIS [45], nvAgent [49], Chat2VIS [50] and LIDA [51] are evaluated alongside a plain-prompt GPT-4o-mini baseline under a shared harness: all systems use the same model, decoding settings (temperature 0) and output interface, with each system using its own published prompt templates. The comparison should therefore be read as shared-harness performance rather than an absolute ranking across all possible model and prompt pairings. VisEval uses Spider-style relational schemas rather than HPC telemetry, so these results test the system's generic chart-generation competence, specifically its ability to generate correct SQL and appropriate visualizations from natural language rather than its HPC-specific classifier dispatch or real-time pipeline behavior.

Table 8.1 presents the results across six metrics: invalid rate (queries that fail to produce any output), illegal rate (queries that produce syntactically invalid charts), pass rate (queries that produce a correct and renderable chart), readability score, composite quality score (pass rate $\times$ readability), tokens per query, wall-clock latency and quality delta relative to the plain-prompt baseline.

Table 8.1: VisEval single-table split (2,064 queries = 688 samples $\times$ 3 paraphrases), all systems under `gpt-4o-mini`, temperature 0, 180-second timeout. Quality = Pass Rate $\times$ Readability (VisEval composite). *Tok/q* is the sum of prompt and completion tokens per query. ΔQ is the quality delta relative to the GPT-4o-mini plain-prompt baseline. Best value per column is bold; the proposed system is shaded.

System	Paradigm	Invalid	Illegal	Pass	Read.	Qual.	Tok/q	Lat.	ΔQ
CoML4VIS [45]	LLM	**0.5%**	27.0%	72.5%	3.80	2.76	2,657	4.6 s	-0.47
nvAgent [49]	LLM	2.5%	25.1%	72.4%	4.35	3.16	5,270	7.7 s	-0.07
Chat2VIS [50]	LLM	2.1%	26.6%	71.3%	4.44	3.17	**840**	7.1 s	-0.06
LIDA [51]	LLM	25.9%	**18.4%**	55.6%	4.31	2.39	1,432	**3.3 s**	-0.84
GPT-4o-mini (baseline)	Baseline	2.2%	26.6%	71.1%	**4.54**	3.23	841	7.2 s	—
Ours (Vanna-RAG)	LLM + RAG	1.5%	20.2%	**78.3%**	4.42	**3.49**	1,416	9.7 s	**+0.26**

The proposed system achieves the highest pass rate at $78.3\,\%$, representing a gain of 5.8 percentage points over the next best system CoML4VIS and the highest composite quality score of 3.49, a $+0.26$ improvement over the plain-prompt baseline. The illegal rate of $20.2\,\%$, while not the lowest across all systems, is a meaningful improvement over the $26.6\,\%$ observed in both Chat2VIS and the plain-prompt baseline, reflecting the benefit of the RAG-based SQL generation and single-shot regeneration mechanism described in Chapter 6.

The improvement comes at a measurable cost. The proposed system consumes approximately $1.7\times$ the tokens per query compared to the plain-prompt baseline (1,416 vs. 841) and takes $1.4\times$ longer in wall-clock latency (9.7 s vs. 7.2 s). This overhead reflects the per-request schema introspection, the retrieval step that fetches relevant DDL fragments and exemplars from ChromaDB and the occasional single-shot SQL regeneration triggered by execution errors. For the monitoring use case, where a user submitting a natural language query expects a response within seconds rather than milliseconds, this latency is acceptable. The quality gain fewer failed queries, fewer illegal charts and a higher overall composite score justifies the additional cost.

It is worth noting that LIDA achieves the lowest illegal rate ($18.4\,\%$) and the fastest latency ($3.3\,s$), but also the lowest pass rate ($55.6\,\%$) and the worst composite quality score (2.39), suggesting that its approach trades correctness for speed in a way that is unsuitable for a system where the primary goal is to produce reliable, readable charts from arbitrary natural language questions.

Chapter 9

Conclusion and Future Work

This chapter brings the thesis to a close. Section 9.1 summarizes the contributions of the proposed Smart Monitoring Framework and reflects on what was achieved relative to the goals set out in Chapter 3. Section 9.2 identifies the current limitations of the system as it stands. Section 9.3 proposes concrete directions for addressing those limitations and extending the framework's capabilities.

9.1 Conclusion

This thesis set out to address a practical and regular problem in shared HPC laboratory environments: resource allocation is inefficient because administrators lack the fine-grained, real-time visibility needed to understand how resources are actually being consumed across users, applications and processes. The surveyed landscape of existing monitoring tools, spanning ten academic frameworks from DCDB and Ganglia to MonSTer and LDMS confirmed that this is not a solved problem. Every reviewed tool fell short on at least one of the eight evaluation criteria derived from the identified research gaps and none offered any form of intelligent, interactive interface that non-expert users could access without deep technical knowledge of the underlying data schema.

The proposed Smart Monitoring Framework addresses these gaps through a modular, end-to-end architecture organized into three layers. The Compute Node Layer deploys a lightweight agent on each physical node that collects CPU, memory, disk I/O, network and GPU metrics at the process level using eBPF and nvidia-smi, packages them

into unified per-node payloads then streams them continuously to the middleware via gRPC client-streaming. The Middleware Layer receives these streams through the Collect Agent, applies a structured preprocessing pipeline that validates, filters, enriches and detects anomalies in real time and publishes clean data to Apache Kafka for decoupled, fault-tolerant downstream delivery. Runtime configuration across all agents is managed dynamically by the Coordinator using etcd, allowing sampling rates, enabled sensors and preprocessing rules to be updated without any service restart. The Application and Data Services Layer consumes the Kafka stream, decomposes each package into three purpose-built InfluxDB measurements for short-term high-resolution storage and aggregates hourly summaries into TimescaleDB for long-term analytical queries. A Next.js web application provides administrators with real-time dashboards, historical analytics, alert notifications and cluster configuration management in a single interface. The Support Agent Server allows users to query the monitoring data through natural language and receive dynamically generated visualizations in either static SVG charts for historical questions or live Grafana panel embeds for real-time questions without requiring any knowledge of the underlying database schema.

The evaluation results support the framework's two primary claims. On the monitoring substrate side, the overhead measurements across four controlled scenarios demonstrate that the Compute Node Agent maintains a stable, near-constant memory footprint regardless of host process density, which enables all sensor types adds only 40 MiB of agent process memory over a partial configuration. The Collect Agent scales near-flatly with the number of attached nodes and its full preprocessing pipeline consumes less than $2\,\%$ of host memory even in the most demanding measured configuration. On the visualization side, the historical chart-generation module achieves the highest pass rate ($78.3\,\%$) and composite quality score (3.49) on the VisEval single-table benchmark against four reference systems under a shared evaluation harness, confirming that the RAG-augmented SQL generation and single-shot regeneration mechanism produce more reliable chart outputs than both simpler LLM approaches and the plain-prompt baseline.

Beyond these results, the framework's most significant contribution is arguably architec-

tural. By demonstrating that a monitoring system can be simultaneously modular, with every component independently replaceable, also intelligent with natural language as the primary user interface. This work establishes a reference design for the next generation of HPC monitoring tools that serve not only system administrators but also the researchers and lecturers who depend on the resources being managed.

9.2 Current Limitations

The framework as implemented has several limitations that are important to acknowledge. Some are scope decisions made deliberately given the timeline of a single academic year; others are known engineering gaps that were identified during development but deferred.

Single-node Collect Agent deployment. The current implementation deploys a single Collect Agent instance that receives streams from all Compute Node Agents in the cluster. While the asynchronous gRPC server design handles concurrent streams efficiently within a single process, there is no mechanism for horizontal scaling of the Collect Agent itself. If the number of compute nodes grows beyond what a single Collect Agent instance can handle or if the Collect Agent process fails, all metric ingestion stops until it is restarted. The system currently has no Collect Agent failover or load distribution capability.

***At-most-once* Kafka delivery semantics.** The Kafka producer in the Collect Agent is configured with *at-most-once* delivery, meaning that a message may be silently lost if the broker is temporarily unavailable or the producer process restarts. For a continuous monitoring stream where occasional gaps are acceptable, this is a reasonable trade-off for lower latency. However, it means the system cannot provide any guarantee of complete data coverage for periods that administrators may need to analyze precisely after the fact.

Limited alert mechanism. Alerts detected by the Collect Agent's problem detection stage are delivered to the Application Server via a direct gRPC call and surfaced as browser notifications through a polling mechanism. There is no persistent alert history, no email or messaging integration. An alert that an administrator misses because they

are not looking at the web interface is effectively lost.

No file system or network fabric monitoring. The current sensor suite collects CPU, memory, disk I/O at the process level, network socket bytes and GPU metrics. It does not collect metrics from shared file systems such as Lustre or NFS, nor from the cluster's high-speed interconnect fabric. In HPC environments, shared file system contention and network congestion are common sources of job performance degradation that the current system cannot observe or attribute.

Chart-code execution sandbox. As identified in Chapter 6's security analysis, the chart-code generation stage in the Support Agent's historical pipeline executes LLM-generated Python code in a namespace-isolated environment but not in a true security sandbox. A maliciously crafted prompt that successfully shapes the LLM's code output could in principle cause the agent process to perform unintended operations. This is the most significant security limitation in the current implementation.

9.3 Future Work

Each limitation identified in the previous section points to a concrete direction for future development. Beyond addressing those gaps, there are also several natural extensions to the framework's capabilities that fall outside the current scope.

Horizontal scaling and high availability of the Collect Agent. The most impactful near-term infrastructure improvement would be to deploy multiple Collect Agent instances behind a load balancer, with each instance handling a subset of the compute nodes. Kafka's consumer group model already supports fan-out consumption from multiple producers, so no changes to the downstream pipeline would be required. The Coordinator's etcd-based configuration distribution would need to be extended to manage multiple Collect Agent instances simultaneously, but the watch-based notification mechanism already supports this at the protocol level. Combining multiple instances with a health check and automatic failover policy would eliminate the current single point of failure in the middleware layer.

Persistent and multi-channel alert management. The alert system could be improved by persisting alerts to TimescaleDB alongside the monitoring data, creating a queryable alert history that the Support Agent could answer questions about. Integration with external notification channels such as email, Slack or a webhook endpoint would ensure that critical alerts reach administrators regardless of whether they are currently viewing the web interface. A configurable escalation policy that increases notification urgency for violations that persist beyond a defined duration would further improve the system's utility.

Extended sensor coverage. Adding support for shared file system monitoring, specifically Lustre and NFS metrics such as open, close, read and write operation counts and latencies would provide visibility into one of the most common sources of HPC job performance degradation. Similarly, adding support for InfiniBand or high-speed Ethernet fabric metrics at the per-node level would allow the system to detect network congestion and attribute it to specific jobs or users. Both extensions could be implemented as new plugins in the agent's plugin-based sensor architecture without any changes to the core collection or transmission pipeline.

Chart-code sandbox. Replacing the current namespace-isolated execution environment for LLM-generated chart code with a proper security sandbox either a subprocess with restricted capabilities and resource limits or a WebAssembly runtime that exposes only matplotlib and the result DataFrame would close the most significant security gap in the Support Agent. This is a well-understood engineering problem and could be implemented independently of any other changes to the agent's pipeline.

End-to-end evaluation on HPC telemetry. The most important evaluation gap to close is a systematic assessment of the Support Agent's performance on real HPC monitoring data. This would involve constructing a benchmark of natural language questions representative of the questions that administrators and users actually ask, covering both historical and real-time queries, a range of complexity levels and queries that require joins between the node, GPU and user-application measurements and evaluating the agent's

end-to-end accuracy, latency and user-perceived quality on this benchmark. Coupling this with a user study involving actual cluster administrators and researchers would provide the kind of practical validation that the current VisEval results.

Predictive analytics and anomaly forecasting. Looking further ahead, the long-term data accumulated in TimescaleDB creates an opportunity to move beyond reactive monitoring toward predictive analytics. Machine learning models trained on historical usage patterns could forecast resource demand for upcoming jobs, identify users whose allocation requests consistently deviate from actual usage or predict node failures based on early warning signals in hardware metrics. Such capabilities would transform the system from a tool that describes what happened into one that anticipates what is about to happen, a qualitative step forward in the practical utility of HPC monitoring.

References

[1] Alessio Netti et al. “From Facility to Application Sensor Data: Modular, Continuous and Holistic Monitoring with DCDB”. In: (). URL: https://www.influxdata.com/.

[2] Francesco Beneventi et al. “Continuous learning of HPC infrastructure models using big data analytics and in-memory processing tools Conference Paper Continuous Learning of HPC Infrastructure Models using Big Data Analytics and In-Memory processing Tools”. In: *Europe Conference & Exhibition (DATE* (2017), pp. 1038–1043. DOI: 10.3929/ethz-b-000192078. URL: http://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=7927143&isnumber=7926947.

[3] Anthony Agelastos et al. *The Lightweight Distributed Metric Service: A Scalable Infrastructure for Continuous Monitoring of Large Scale Computing Systems and Applications*. Tech. rep. 2014.

[4] Matthew L. Massie, Brent N. Chun, and David E. Culler. “The ganglia distributed monitoring system: Design, implementation, and experience”. In: *Parallel Computing* 30 (7 July 2004), pp. 817–840. ISSN: 01678191. DOI: 10.1016/j.parco.2004.04.001.

[5] Jie Li et al. “MonSTer: An Out-of-the-Box Monitoring Tool for High Performance Computing Systems”. In: *Proceedings - IEEE International Conference on Cluster Computing, ICCC*. Vol. 2020-September. Institute of Electrical and Electronics Engineers Inc., Sept. 2020, pp. 119–129. ISBN: 9781728166773. DOI: 10.1109/CLUSTER49012.2020.00022.

[6] Eric Anderson, Dave Patterson, and U C Berkeley. *Extensible, Scalable Monitoring for Clusters of Computers*. Tech. rep. 1997. URL: http://now.cs.berkeley.edu/.

[7] M. J. Sottile and R. G. Minnich. "Supermon: A high-speed cluster monitoring system". In: *Proceedings - IEEE International Conference on Cluster Computing, ICCC*. Vol. 2002-January. Institute of Electrical and Electronics Engineers Inc., 2002, pp. 39–46. ISBN: 0769517455. DOI: 10.1109/CLUSTR.2002.1137727.

[8] Ryan Mooney, Ken P Schmidt, and R Scott Studham. "NWPerf: a system wide performance monitoring tool for large Linux clusters". In: *2004 IEEE International Conference on Cluster Computing (IEEE Cat. No. 04EX935)*. IEEE. 2004, pp. 379–389.

[9] Bin Yang et al. "End-to-end I/O Monitoring on Leading Supercomputers". In: *ACM Transactions on Storage* 19 (1 Jan. 2023). ISSN: 15533093. DOI: 10.1145/3568425.

[10] Robert Dietrich et al. "PIKA: Center-Wide and Job-Aware Cluster Monitoring". In: *Proceedings - IEEE International Conference on Cluster Computing, ICCC*. Vol. 2020-September. Institute of Electrical and Electronics Engineers Inc., Sept. 2020, pp. 424–432. ISBN: 9781728166773. DOI: 10.1109/CLUSTER49012.2020.00061.

[11] Zabbix LLC. *Zabbix: The Enterprise-Class Open Source Network Monitoring Solution*. https://www.zabbix.com. Accessed: 2025. 2001.

[12] Prometheus Authors. *Prometheus: From metrics to insight*. https://prometheus.io. Accessed: 2025. 2012.

[13] Nagios Enterprises. *Nagios Core: The Industry Standard in Open Source Monitoring*. https://www.nagios.org. Accessed: 2025. 1999.

[14] Mengyi Liu and Jianqiu Xu. "NLI4DB: A Systematic Review of Natural Language Interfaces for Databases". In: (Mar. 2025). URL: https://arxiv.org/pdf/2503.02435.

[15] Ana-Maria Popescu, Oren Etzioni, and Henry Kautz. “Towards a theory of natural language interfaces to databases”. In: (Jan. 2003), pp. 149–157. DOI: 10.1145/604045.604070. URL: /doi/pdf/10.1145/604045.604070?download=true.

[16] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. “Sequence to Sequence Learning with Neural Networks”. In: *Advances in Neural Information Processing Systems* 4 (January Sept. 2014), pp. 3104–3112. ISSN: 10495258. URL: https://arxiv.org/pdf/1409.3215.

[17] Dzmitry Bahdanau, Kyung Hyun Cho, and Yoshua Bengio. “Neural Machine Translation by Jointly Learning to Align and Translate”. In: *3rd International Conference on Learning Representations, ICLR 2015 - Conference Track Proceedings* (Sept. 2014). URL: https://arxiv.org/pdf/1409.0473.

[18] Tao Yu et al. “Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task”. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, EMNLP 2018* (Sept. 2018), pp. 3911–3921. DOI: 10.18653/v1/d18-1425. URL: https://arxiv.org/pdf/1809.08887.

[19] Xiaojun Xu, Chang Liu, and Dawn Song. “SQLNet: Generating Structured Queries From Natural Language Without Reinforcement Learning”. In: (Nov. 2017). URL: https://arxiv.org/pdf/1711.04436.

[20] Jiaqi Guo et al. “Towards Complex Text-to-SQL in Cross-Domain Database with Intermediate Representation”. In: *ACL 2019 - 57th Annual Meeting of the Association for Computational Linguistics, Proceedings of the Conference* (2019), pp. 4524–4535. DOI: 10.18653/V1/P19-1444. URL: https://aclanthology.org/P19-1444/.

[21] Mark Chen et al. “Evaluating Large Language Models Trained on Code”. In: (July 2021). URL: https://arxiv.org/pdf/2107.03374.

[22] Zijin Hong et al. “Next-Generation Database Interfaces: A Survey of LLM-based Text-to-SQL”. In: *IEEE Transactions on Knowledge and Data Engineering* 37 (12

June 2024), pp. 7328–7345. ISSN: 15582191. DOI: 10.1109/TKDE.2025.3609486. URL: https://arxiv.org/pdf/2406.08426.

[23] Xiaohu Zhu et al. "Large Language Model Enhanced Text-to-SQL Generation: A Survey". In: (Oct. 2024). URL: https://arxiv.org/pdf/2410.06011.

[24] *Best practices for prompt engineering with Meta Llama 3 for Text-to-SQL use cases — Artificial Intelligence*. URL: https://aws.amazon.com/blogs/machine-learning/best-practices-for-prompt-engineering-with-meta-llama-3-for-text-to-sql-use-cases/.

[25] *Vanna 2.0 – Build Agents Your Users Can Actually Use*. URL: https://vanna.ai/.

[26] Dawei Gao et al. "Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation". In: *Proceedings of the VLDB Endowment* 17 (5 Aug. 2023), pp. 1132–1145. ISSN: 21508097. DOI: 10.14778/3641204.3641221. URL: https://arxiv.org/pdf/2308.15363.

[27] Eduardo R Nascimento et al. "Text-to-SQL based on Large Language Models and Database Keyword Search". In: (Jan. 2025). URL: https://arxiv.org/pdf/2501.13594.

[28] Mohammadreza Pourreza and Davood Rafiei. "DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction". In: *Advances in Neural Information Processing Systems* 36 (Apr. 2023). ISSN: 10495258. URL: https://arxiv.org/pdf/2304.11015.

[29] Tao Yu et al. "CoSQL: A Conversational Text-to-SQL Challenge Towards Cross-Domain Natural Language Interfaces to Databases". In: *EMNLP-IJCNLP 2019 - 2019 Conference on Empirical Methods in Natural Language Processing and 9th International Joint Conference on Natural Language Processing, Proceedings of the Conference* (2019), pp. 1962–1979. DOI: 10.18653/V1/D19-1204. URL: https://aclanthology.org/D19-1204/.

[30] Tao Yu et al. "SParC: Cross-Domain Semantic Parsing in Context". In: *ACL 2019 - 57th Annual Meeting of the Association for Computational Linguistics, Proceed-*

ings of the Conference (2019), pp. 4511–4523. DOI: 10.18653/V1/P19-1443. URL: https://aclanthology.org/P19-1443/.

[31] Huaixiu Steven Zheng et al. "Take a Step Back: Evoking Reasoning via Abstraction in Large Language Models". In: *12th International Conference on Learning Representations, ICLR 2024* (Oct. 2023). URL: https://arxiv.org/pdf/2310.06117.

[32] Raphaël Mouravieff, Benjamin Piwowarski, and Sylvain Lamprier. "Learning Relational Decomposition of Queries for Question Answering from Tables". In: *Proceedings of the Annual Meeting of the Association for Computational Linguistics* 1 (2024), pp. 10471–10485. ISSN: 0736587X. DOI: 10.18653/V1/2024.ACL-LONG.564. URL: https://aclanthology.org/2024.acl-long.564/.

[33] Jock Mackinlay. "Automating the Design of Graphical Presentations of Relational Information". In: *ACM Transactions on Graphics (TOG)* 5 (2 Apr. 1986), pp. 110–141. ISSN: 15577368. DOI: 10.1145/22949.22950;WEBSITE:WEBSITE:DL-SITE;TAXONOMY:TAXONOMY:ACM-PUBTYPE;PAGEGROUP:STRING:PUBLICATION. URL: /doi/pdf/10.1145/22949.22950?download=true.

[34] William S. Cleveland and Robert McGill. "Graphical perception: Theory, experimentation, and application to the development of graphical methods". In: *Journal of the American Statistical Association* 79 (387 1984), pp. 531–554. ISSN: 1537274X. DOI: 10.1080/01621459.1984.10478080;REQUESTEDJOURNAL:JOURNAL:UASA20;WGROUP:STRING:PUBLICATION.

[35] Kevin Hu et al. "VizML: A machine learning approach to visualization recommendation". In: *Conference on Human Factors in Computing Systems - Proceedings* (May 2019). DOI: 10.1145/3290605.3300358;PAGE:STRING:ARTICLE/CHAPTER. URL: /doi/pdf/10.1145/3290605.3300358?download=true.

[36] Dominik Moritz et al. "Formalizing visualization design knowledge as constraints: Actionable and extensible models in Draco". In: *IEEE Transactions on Visualiza-*

tion and Computer Graphics 25 (1 Jan. 2019), pp. 438–448. ISSN: 19410506. DOI: 10.1109/TVCG.2018.2865240. URL: https://ieeexplore.ieee.org/document/8440847.

[37] Yuyu Luo et al. "DeepEye: Creating good data visualizations by keyword search". In: *Proceedings of the ACM SIGMOD International Conference on Management of Data* (May 2018), pp. 1733–1736. ISSN: 07308078. DOI: 10.1145/3183713.3193545;JOURNAL:JOURNAL:ACMCONFERENCES;PAGEGROUP:STRING:PUBLICATION. URL: /doi/pdf/10.1145/3183713.3193545?download=true.

[38] Yuan Tian et al. "ChartGPT: Leveraging LLMs to Generate Charts from Abstract Natural Language". In: *IEEE Transactions on Visualization and Computer Graphics* 31 (3 Jan. 2025), pp. 1731–1745. DOI: 10.1109/TVCG.2024.3368621. URL: http://arxiv.org/abs/2311.01920http://dx.doi.org/10.1109/TVCG.2024.3368621.

[39] Chenglong Wang, John Thompson, and Bongshin Lee. "Data Formulator: AI-powered Concept-driven Visualization Authoring". In: *IEEE Transactions on Visualization and Computer Graphics* 30 (1 Sept. 2023), pp. 1128–1138. ISSN: 19410506. DOI: 10.1109/TVCG.2023.3326585. URL: https://arxiv.org/pdf/2309.10094.

[40] Lei Wang et al. "LLM4Vis: Explainable Visualization Recommendation using ChatGPT". In: *EMNLP 2023 - 2023 Conference on Empirical Methods in Natural Language Processing, Proceedings of the Industry Track* (Oct. 2023), pp. 675–692. DOI: 10.18653/v1/2023.emnlp-industry.64. URL: https://arxiv.org/pdf/2310.07652.

[41] Arpit Narechania, Arjun Srinivasan, and John Stasko. "NL4DV: A toolkit for generating analytic specifications for data visualization from natural language queries". In: *IEEE Transactions on Visualization and Computer Graphics* 27.2 (2020), pp. 369–379.

[42] Yuyu Luo et al. “Natural language to visualization by neural machine translation”. In: *IEEE Transactions on Visualization and Computer Graphics* 28.1 (2021), pp. 217–226.

[43] Paula Maddigan and Teo Susnjak. “Chat2VIS: Generating data visualizations via natural language using ChatGPT, Codex and GPT-3 large language models”. In: *IEEE Access* 11 (2023), pp. 45181–45193.

[44] Xinyang Zhao, Xuanhe Zhou, and Guoliang Li. “Chat2Data: An interactive data analysis system with RAG, vector databases and LLMs”. In: *Proceedings of the VLDB Endowment* 17.12 (2024), pp. 4481–4484.

[45] Nan Chen et al. “VisEval: A benchmark for data visualization in the era of large language models”. In: *IEEE Transactions on Visualization and Computer Graphics* 31.1 (2024), pp. 1301–1311.

[46] Junqi Yin et al. “chatHPC: Empowering HPC users with large language models”. In: *The Journal of Supercomputing* 81.1 (2025), p. 194. DOI: 10.1007/s11227-024-06637-1.

[47] *Grafana 12 release: observability as code, dynamic dashboards, new Grafana Alerting tools, and more — Grafana Labs*. URL: https://grafana.com/blog/grafana-12-release-all-the-new-features/.

[48] *HTTP API — Grafana documentation*. URL: https://grafana.com/docs/grafana/latest/developer-resources/api-reference/http-api/.

[49] Geliang Ouyang et al. “nvagent: Automated data visualization from natural language via collaborative agent workflow”. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2025, pp. 19534–19567.

[50] Paula Maddigan and Teo Susnjak. “Chat2vis: Generating data visualizations via natural language using chatgpt, codex and gpt-3 large language models”. In: *Ieee Access* 11 (2023), pp. 45181–45193.

[51] Victor Dibia. “LIDA: A tool for automatic generation of grammar-agnostic visualizations and infographics using large language models”. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*. 2023, pp. 113–126.